

UNIVERSIDAD DE ALMERIA

ESCUELA SUPERIOR DE INGENIERÍA

SmartFreezer:
Aplicación inteligente
para la Detección,
Clasificación y Gestión
de Frutas y Verduras”

Curso 2024/2025

Alumno/a:

Jesús David Martínez Valdés

Director/es:

Francisco de Asís Guindos Rojas

DEDICATORIA

Este logro no es únicamente mérito propio debido al apoyo incondicional recibido tanto por mi familia, como por mi pareja y amigos, todo esto no hubiese sido posible por su presencia y ayuda oportuna, incluso sin necesidad de pedirles explícitamente, han hecho posible este camino.

Deseo destacar también el extraordinario esfuerzo y continuo interés de mis queridos abuelos en mi progreso académico. Es ahora donde quiero demostrarles y agradecerles todo lo que he sido capaz y seré durante mi vida.

Finalmente, quisiera agradecer a los distintos docentes de la Universidad de Almería que han despertado en mí infinitas ilusiones en continuar mi camino como Ingeniero Informático. Ha sido todo un honor poder haber sido estudiante de cada uno de ellos.

RESUMEN

En la actualidad, la gestión eficiente de productos perecederos es un desafío en muchos hogares, lo que conlleva al desperdicio de alimentos y compras innecesarias. Gracias a *SmartFreezer* brindará a los usuarios una solución innovadora que optimiza la organización de frutas y verduras mediante un sistema de reconocimiento basado en redes neuronales. El objetivo de esta aplicación es reducir el malgasto, mejorar la planificación de compras y proporcionar información interesante de cada alimento, facilitando así un consumo más eficiente y sostenible. Además, integrará funcionalidades como notificaciones de compras, información nutricional, recomendación de recetas con los productos disponibles, etc.

Esta aplicación ha sido desarrollada principalmente mediante el lenguaje de programación de código abierto *Kotlin*, preferido por muchos desarrolladores de Android. Por otro lado, se ha integrado la función de detección y reconocimiento de varias frutas y verduras mediante un modelo previamente entrenado por el modelo de Deep Learning *YOLOv8* “*You Only Look Once*”. Cabe mencionar el uso del servicio de Backend-as-a-Service (BaaS) *Firebase* tanto para el sistema de autenticación de usuario, como sistema de notificaciones y uso de base de datos NoSQL.

Esta idea surge de la amplia demanda de dispositivos inteligentes en el hogar, por ejemplo, un frigorífico inteligente que solucione todas las peticiones del usuario para su compra y conservación de alimentos de forma automática. Sin embargo, *SmartFreezer* recoge todas estas características sin la inversión monetaria de un dispositivo nuevo al hogar.

Palabras claves: [YOLOv8](#), [Kotlin](#), [Firebase](#), [Red Neuronal](#), [Android](#)].

ABSTRACT

Currently, the efficient management of perishable products is a challenge in many households, leading to food waste and unnecessary purchases. Thanks to SmartFreezer, users will be provided with an innovative solution that optimizes the organization of fruits and vegetables through a recognition system based on neural networks. The objective of this application is to reduce waste, improve purchase planning, and provide interesting information about each food item, thus facilitating more efficient and sustainable consumption. Furthermore, it will integrate functionalities such as shopping notifications, nutritional information, recipe recommendations based on available products, etc.

This application has been primarily developed using the open-source programming language Kotlin, preferred by many Android developers. On the other hand, the detection and recognition function for various fruits and vegetables has been integrated using a model previously trained by the Deep Learning model YOLOv8 "You Only Look Once." It is worth mentioning the use of the Backend-as-a-Service (BaaS) Firebase for the user authentication system, notification system, and NoSQL database usage.

This idea arises from the widespread demand for smart home devices, for example, a smart refrigerator that automatically solves all user requests for food purchasing and preservation. However, SmartFreezer encompasses all these features without the monetary investment of a new home appliance.

Keywords: [YOLOv8](#), [Kotlin](#), [Firebase](#), [Red Neuronal](#), [Android](#)].

Índice del TFG

1. Introducción.....	15
1.1 Contextualización y Justificación del Proyecto	15
1.2 Objetivos del proyecto.....	16
1.3 Estructura del documento	17
1.4 Metodología empleada.....	19
1.4.1 Ciclo de vida del software	19
1.4.2 Herramientas empleadas.....	21
1.4.3 Modelado formal	25
1.4.4 Modelado conceptual.....	41
1.5 Planificación temporal	42
1.6 Diseño del prototipo	44
2. Estado del arte / Antecedentes y Marco teórico.....	52
2.1 Gestión de Alimentos y Desperdicio Alimentario	52
2.2 Tecnologías de Visión Artificial y Deep Learning	54
2.2.1 Modelo YOLO (You Only Look Once) (17)	57
2.3 Desarrollo de Aplicaciones Móviles Android	59
2.3.1 Kotlin como lenguaje nativo (19).....	59
2.3.2 Android Studio como IDE (20)	60
2.4. Servicios Backend-as-a-Service (BaaS): Firebase (22)	64
2.5 API Externa para Enriquecimiento de funcionalidades.....	68
3. Diseño y Desarrollo del Modelo de Detección de Frutas y Verduras	68
3.1 Estructura y herramientas	68
3.1.1 Dataset utilizado	68
3.1.2 Preparación de la estructura del dataset.....	70

3.2 Entrenamiento del modelo YOLOv8.....	80
3.2.1 Gráfico de Pérdida.....	83
3.2.2 Gráfico de Detección	84
3.2.3 Matriz de confusión.....	84
3.3 Adaptación del modelo para Android	87
4. Diseño y Desarrollo de la Aplicación "SmartFreezer"	88
4.1 Arquitectura de la Aplicación.....	88
4.2 Diseño y herramientas utilizadas de Firebase para la app	91
4.3 Funcionamiento de la aplicación	98
4.3.1 Model.....	98
4.3.2 View.....	101
4.3.3 ViewModel	117
4.4 Configuración del entorno de desarrollo.....	119
4.5 Diagrama de la arquitectura de la aplicación	122
5. Pruebas y Evaluación de la Aplicación.....	124
5.1 Pruebas del Modelo de Detección	124
5.1.1 Pruebas de Precisión y Rendimiento	124
5.1.2 Pruebas de Tiempo Medio de Escaneo	126
5.2 Pruebas de Funcionalidad de la Aplicación.....	127
5.3 Pruebas de Aceptación.....	128
5.4 Pruebas No Funcionales de la Aplicación	129
6. Resultados y discusión	132
6.1 Cumplimiento de objetivos.....	132
6.2 Presentación de la Aplicación Final	132
6.3 Discusión de los resultados obtenidos	133
7. Conclusiones y Trabajo futuro	134

7.1 Líneas de Trabajo Futuro.....	135
Bibliografía	137
Anexo A	140
Manual de Usuario.....	140
A.1 Descripción	140
A.2 Manual de usuario SmartFreezer	140

Índice de Ilustraciones

Ilustración 1: Desperdicio total de productos sin elaborar	15
Ilustración 2: Metodología SCRUM	20
Ilustración 3: Repositorio GitHub SmartFreezer	22
Ilustración 4: Logotipo Firebase.....	23
Ilustración 5: Logotipo Android Studio	23
Ilustración 6: Logotipo Pycharm	24
Ilustración 7: Tarjeta gráfica NVIDIA.....	25
Ilustración 8: Diagrama de casos de uso sobre la aplicación.....	42
Ilustración 9: Logotipo SmartFreezer.....	44
Ilustración 10: Prototipo pantalla de Inicio	45
Ilustración 11: Prototipo pantalla Inicio Sesión	45
Ilustración 12: Prototipo pantalla Registro	46
Ilustración 13: Prototipo pantalla Inventario	46
Ilustración 14: Prototipo pantalla Detalles Producto.....	47
Ilustración 15: Prototipo pantalla Desperdicios	47
Ilustración 16: Prototipo pantalla Añadir Producto Inventario.....	48
Ilustración 17: Prototipo pantalla Lista de Compra.....	48
Ilustración 18: Prototipo pantalla Añadir Producto Lista de Compra	49
Ilustración 19: Prototipo pantalla Recetas Disponibles	49
Ilustración 20: Prototipo pantalla Detalles Receta.....	50
Ilustración 21: Prototipo pantalla Alertas	50
Ilustración 22: Prototipo pantalla Escáner	51
Ilustración 23: Prototipo pantalla Perfil	51
Ilustración 24: Prototipo pantalla Ajustes.....	52
Ilustración 25: Función ReLU	55
Ilustración 26: Pooling funcionamiento	55
Ilustración 27: Arquitectura CNN	57

Ilustración 28: Arquitectura YOLOv8	58
Ilustración 29: Estructura AndroidStudio.....	61
Ilustración 30: Diagrama ciclo de vida Activity	64
Ilustración 31: Estructura Dataset.....	69
Ilustración 32: dataset.yaml.....	70
Ilustración 33: Creación train/ y val/.....	71
Ilustración 34: Archivo configuración YOLOv8.....	72
Ilustración 35: Ejemplo de bounding box XML	73
Ilustración 36: Estructura COCO segment_generator.py	74
Ilustración 37: Proceso de segmentación	75
Ilustración 38: Funciones de segmentación OpenCV	76
Ilustración 39: Imagen resultado de la segmentación	76
Ilustración 40: Código sobre anotaciones COCO	77
Ilustración 41: Código sobre guardar archivo COCO	77
Ilustración 42: Formato YOLO imagen	78
Ilustración 43: Lectura archivo COCO	79
Ilustración 44: Calcular coordenadas normalizadas.....	79
Ilustración 45: Instalador NVIDIA CUDA.....	81
Ilustración 46: Entrenamiento modelo script.....	82
Ilustración 47: Gráficas obtenidas del entrenamiento	83
Ilustración 48: Matriz de confusión (Fuente: Rune Hylsberg Jacobsen).....	85
Ilustración 49: Matriz de confusión frutas/verduras	86
Ilustración 50: Matriz de confusión normalizada	86
Ilustración 51: Lote de predicción val/.....	87
Ilustración 52: export_to_onnx.py	88
Ilustración 53: Diagrama BBDD de la app	92
Ilustración 54: Gestión de usuarios Firebase.....	93
Ilustración 55: Plantillas Firebase	94
Ilustración 56: Personalización notificación	95

Ilustración 57: Ajustes notificación con valores adicionales	95
Ilustración 58: Función onMessageReceived	96
Ilustración 59: Función sendNotification	97
Ilustración 60: Función subscribeToDietTopics	97
Ilustración 61: Función unsubscribeFromAllDietTopics	98
Ilustración 62: JSON to Kotlin Class	99
Ilustración 63: Result.kt	99
Ilustración 64: RecipeRepository.kt	100
Ilustración 65: RemoteDataSource.kt	100
Ilustración 66: NetworkModule.kt	100
Ilustración 67: FoodRecipseApi.kt	101
Ilustración 68: Lógica de inicio de sesión	102
Ilustración 69: Lógica para restablecer contraseña	103
Ilustración 70: createUserWithEmailAndPassword	104
Ilustración 71: sendEmailVerfication	104
Ilustración 72: NavController	105
Ilustración 73: nav_graph.xml	106
Ilustración 74: Código ProfileActivity método loadUserData()	107
Ilustración 75: Código SettingsActivity.kt método setupNotificationSwitch()	109
Ilustración 76: Código SettingsActivity.kt método setupDarkModeSwitch()	109
Ilustración 77: Código SettingsActivity.kt método changeAppLanguage()	109
Ilustración 78: BaseActivity.kt	110
Ilustración 79: Método loadItemsFromFirestore InventoryFragment.kt	111
Ilustración 80: Método applyFilters de InvetoryFragment.kt	111
Ilustración 81: Método requestApiData de RecipesFragment.kt	112
Ilustración 82: Método processImage parte 1 de ScannerFragment	114
Ilustración 83: Método processImage parte 2 de ScannerFragment	114
Ilustración 84: recipes_row_layout.xml	116
Ilustración 85: RecipesRowBinding	117

Ilustración 86: Función getRecipes de RecipesViewModel.....	118
Ilustración 87: Función getRecipesDetails de RecipesViewModel	118
Ilustración 88: Función de applyQueries de RecipesViewModel.....	119
Ilustración 89: Clase NetworkResult.....	119
Ilustración 90: Plugins Gradle.....	120
Ilustración 91: Dependencias Gradle.....	121
Ilustración 92: MyApplication.....	122
Ilustración 93: AndroidManifest.xml.....	122
Ilustración 94: Diagrama arquitectura Smartfreezer.....	123
Ilustración 95: Resultados detección 1 escenario favorable	125
Ilustración 96: Resultados detección 2 escenario favorable 2.....	125
Ilustración 97: Resultados detección escenario no favorable 1.....	126
Ilustración 98: Resultados detección escenario no favorable 2.....	126
Ilustración 99: Métricas de rendimiento para ScannerFragment.....	130
Ilustración 100: Firebase Performance Monitoring.....	130
Ilustración 101: Consola Crashlytics.....	131
Ilustración 102: Pantalla inicio	140
Ilustración 103: Pantalla inicio sesión	141
Ilustración 104: Dialogo restablecer contraseña	142
Ilustración 105: Pantalla registrar	142
Ilustración 106: Pantalla inventario	143
Ilustración 107: Pantalla seleccionar categoría.....	144
Ilustración 108: Pantalla seleccionar producto inventario	144
Ilustración 109: Pantalla añadir producto al inventario.....	145
Ilustración 110: Pantalla inventario con producto.....	145
Ilustración 111: Pantalla ventana producto del inventario	146
Ilustración 112: Pantalla dialogo para eliminar producto del inventario.....	146
Ilustración 113: Pantalla desperdicios gráfico de barras.....	147
Ilustración 114: Pantalla desperdicio gráfico circular	148

Ilustración 115: Pantalla lista de la compra vacía	149
Ilustración 116: Pantalla seleccionar producto lista de compra.....	149
Ilustración 117: Pantalla lista de compra con productos	150
Ilustración 118: Dialogo de confirmación para mover lista de compra al inventario	150
Ilustración 119: Pantalla recetas disponibles	151
Ilustración 120: Filtros para Recetas	151
Ilustración 121: Pantalla recetas guardadas.....	152
Ilustración 122: Diálogo de confirmación para eliminar todas las recetas	153
Ilustración 123: Pantalla alertas.....	154
Ilustración 124: Diálogo de confirmación de alertas.....	154
Ilustración 125: Pantalla perfil.....	155
Ilustración 126: Pantalla configuración	155
Ilustración 127: Pantalla tema oscuro con idioma inglés.....	156
Ilustración 128: Pantalla escáner.....	157
Ilustración 129: Pantalla producto escaneado agregar a inventario	158

Índice de Tablas

Tabla 1: Especificación del actor no autenticado	27
Tabla 2: Especificación del actor autenticado	27
Tabla 3: Especificación del usuario autenticado	27
Tabla 4: Especificación del usuario no autenticado	27
Tabla 5: RF-1 – Registro de nuevos usuarios.....	28
Tabla 6: RF-2 - Autenticación de usuarios existentes	29
Tabla 7: RF-3 - Recuperación de contraseñas	30
Tabla 8: RF-4 - Gestión del perfil de usuario	31
Tabla 9: RF-5 - Cierre de sesión y eliminación de cuenta.....	32
Tabla 10: RF-6 - Escaneo de frutas y verduras	33
Tabla 11: RF-8 - Añadir productos al inventario.....	34
Tabla 12: RF-9 - Creación y gestión de lista de la compra	34
Tabla 13: RF-9 - Registro de productos desperdiciados	35
Tabla 14: RF-10 - Filtrar recetas.....	36
Tabla 15: RF-11 - Guardar recetas.....	37
Tabla 16: RF-12 - Gestionar alertas.....	37
Tabla 17: RF-13 - Añadir valoración	38
Tabla 18: RF-14 - Recibir notificación nueva receta recomendada.....	38
Tabla 19: RF-15: Configuración del sistema.....	39
Tabla 20: RNF-1 – Mantenibilidad	39
Tabla 21: RNF-2 – Seguridad.....	40
Tabla 22: RNF-3 – Rendimiento	40
Tabla 23: RNF-4 - Accesibilidad	40
Tabla 24: RNF-5 – Usabilidad	40
Tabla 25: RNF-6 – Portabilidad	41
Tabla 26 : Java o Kotlin	60
Tabla 27: Ventajas Firebase.....	65

Tabla 28: Comparación Realtime Database y Cloud Firestore	66
---	----

1. Introducción

1.1 Contextualización y Justificación del Proyecto

En la era digital, los dispositivos móviles y los sistemas embebidos se han convertido en una pieza clave para optimizar los procesos cotidianos. En este contexto, la gestión de productos perecederos en el hogar sigue siendo un reto no resuelto a nivel general, provocando un elevado desperdicio alimentario y por consecuencia, un consumo energético ineficiente.

A pesar de la disminución del 6% en el desperdicio alimentario en hogares españoles durante 2022, según el Ministerio de Agricultura, Pesca y Alimentación (1), la cifra total desperdiciada aún fue considerable, alcanzando los 1.170,45 millones de kilos o litros. Dentro de los productos sin elaborar, las frutas, verduras y hortalizas destacaron como los más desechados, con un porcentaje de desperdicio cinco veces mayor que otros alimentos y, preocupantemente, experimentando un incremento respecto al año anterior como se refleja en siguiente gráfica, Ilustración 1: Desperdicio total de productos sin elaborar, sacada del informe generado por el Ministerio de Agricultura, Pesca y Alimentación.

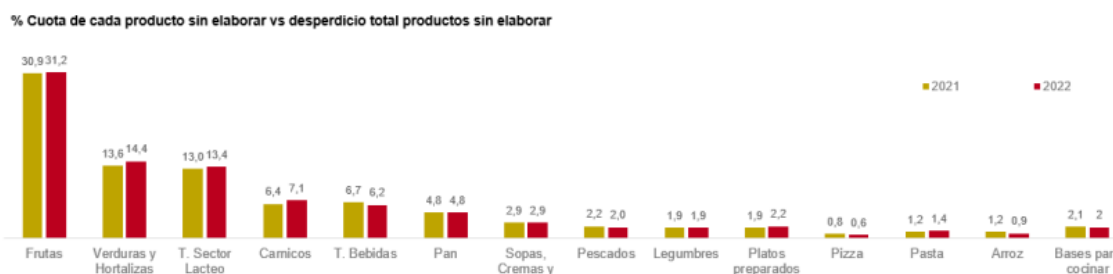


Ilustración 1: Desperdicio total de productos sin elaborar

Desde la perspectiva de la Ingeniería Informática, este escenario plantea cubrir esta necesidad mediante el diseño de arquitecturas de software que logren integrar visión artificial, aprendizaje automático y servicios en la nube de forma ágil, segura y escalable.

SmartFreezer surge como respuesta a este problema, proponiendo una solución basada en una aplicación Android desarrollada en Kotlin que combina la capacidad de reconocimiento de objetos mediante un modelo YOLOv8 optimizado para dispositivos móviles. La aplicación no solo realiza el inventario automático de frutas y verduras,

sino que detecta las imperfecciones de cada una de ellas para evitar deteriorar los demás productos, además de planificar compras y ofrecer numerosas recetas con su información detallada, todo ello sin depender de hardware especializado.

Desde el punto de vista técnico, el principal desafío radica en lograr que dicho modelo se integre de forma nativa con Android Studio, garantizando su conversión a formatos compatibles (por ejemplo, TFLite u ONNX), la cuantización necesaria y su carga eficiente en la aplicación móvil sin sacrificar rendimiento ni precisión.

Paralelamente, resulta esencial diseñar una arquitectura de software modular y segura, empleando la arquitectura de diseño MVVM (Model, View, ViewModel) y aplicando patrones de diseño, además de implementar mecanismos de autenticación y autorización robustos en Firebase Auth, así como garantizar la consistencia y escalabilidad de los datos en la nube a través de Firestore.

Desde un punto de vista metodológico, el proyecto se ha desarrollado siguiendo una aproximación ágil basada en **SCRUM** recogiendo algunos de sus principios como definir las tareas principales de la aplicación y planificarlas en *sprints* semanales para poder implementar las funcionalidades y evaluarlas cada fin de semana por si surgiera la necesidad de reajustar algún objetivo, además el uso de herramientas como GitHub y Firebase Console se han implementado para el control de versiones, seguimiento de tareas y despliegue de servicios.

1.2 Objetivos del proyecto

El objetivo principal de este proyecto consiste en diseñar y desarrollar una aplicación inteligente (para sistemas operativos Android) permitiendo optimizar la gestión doméstica de frutas y verduras mediante la integración de un sistema de reconocimiento de productos basado en redes neuronales, gestión de inventario en la nube y recomendaciones personalizadas, con el fin de reducir el desperdicio alimentario y mejorar la planificación de consumo.

A continuación, se describen los siguientes objetivos específicos del proyecto:

- **Diseñar e implementar un sistema de escaneo de productos** basado en visión por computador utilizando un modelo de Deep Learning (YOLOv8) [1] optimizado para su uso en dispositivos móviles Android.

- **Entrenar y evaluar un modelo de detección de objetos** con una precisión superior al 80%, aplicando técnicas de optimización como la cuantización y conversión a ONNX o TFLite.
- **Integrar servicios en la nube mediante Firebase**, incluyendo:
 - Autenticación de usuarios.
 - Base de datos NoSQL (Firestore) (2) para gestión de productos.
 - Envío de notificaciones push mediante Firebase Cloud Messaging (FCM).
- **Diseñar una interfaz de usuario intuitiva y responsiva**, utilizando Jetpack Compose, que permita al usuario gestionar su inventario, recibir alertas y visualizar recomendaciones.
- **Implementar un sistema de recomendación de recetas**, conectado con una API externa (Spoonacular) (3), que sugiere platos basados en los productos disponibles en el inventario del usuario.
- **Evaluar el rendimiento y la usabilidad de la aplicación**, considerando métricas como la precisión del modelo, el tiempo medio de escaneo, la fluidez de la interfaz y la experiencia del usuario.
- **Documentar el proceso completo de desarrollo**, justificando las decisiones técnicas, describiendo la arquitectura del sistema, y analizando los resultados obtenidos en términos de funcionalidad, rendimiento y escalabilidad.

1.3 Estructura del documento

El presente Trabajo Fin de Grado se ha estructurado en varios capítulos para facilitar la comprensión del proyecto "SmartFreezer", desde su concepción hasta la evaluación de sus resultados. A continuación, se describe brevemente el contenido de cada uno de ellos:

- **Capítulo 1: Introducción.** Se presenta el contexto del proyecto, justificando la necesidad de una solución como *SmartFreezer* ante el problema del desperdicio alimentario. Se definen los objetivos principales y específicos que guían el desarrollo, se describe la metodología ágil adaptada seguida, y, finalmente, se detalla la estructura de este documento. Además, se ha realizado un **análisis de requisitos**, identificando las funcionalidades clave del sistema desde el punto de

vista del usuario. Como complemento, se ha elaborado un **diagrama de casos de uso**, que representa gráficamente las interacciones entre los actores y las funcionalidades del sistema. Por último, se mostrará el diseño aplicado para la aplicación junto a la planificación temporal para el desarrollo de esta.

- **Capítulo 2: Estado del Arte / Antecedentes y Marco Teórico.** Este capítulo profundiza en los conceptos y tecnologías fundamentales para el proyecto. Se revisa la problemática de la gestión de alimentos, se introducen las bases de la visión artificial, el Deep Learning con especial énfasis en el modelo YOLOv8, y las técnicas de optimización para dispositivos móviles. Además, se aborda el desarrollo de aplicaciones Android con Kotlin, el uso de Firebase como Backend-as-a-Service (BaaS) y la integración de APIs externas como Spoonacular para la recomendación de recetas.
- **Capítulo 3: Diseño y Desarrollo del Modelo de Detección de Frutas y Verduras.** Aquí se detalla todo el proceso relacionado con el componente de inteligencia artificial del proyecto. Se describe el dataset empleado, el entrenamiento del modelo YOLOv8, la evaluación de su rendimiento mediante diversas métricas y, finalmente, la optimización y exportación del modelo para su correcta integración en la aplicación Android.
- **Capítulo 4: Diseño y Desarrollo de la Aplicación "SmartFreezer".** Este es el núcleo del desarrollo software, donde se explica la arquitectura de la aplicación móvil, la configuración del entorno y la implementación detallada de cada una de sus funcionalidades: autenticación de usuarios, gestión de inventario mediante escaneo y adición manual, sistema de notificaciones, recomendación de recetas, gestión de listas de compras y visualización de estadísticas de desperdicio. También se aborda el diseño de la interfaz y la experiencia de usuario.
- **Capítulo 5: Pruebas y Evaluación de la Aplicación.** En este apartado se presentan las pruebas realizadas para validar tanto el modelo de detección integrado en la app como la funcionalidad, usabilidad y rendimiento general de "SmartFreezer".
- **Capítulo 6: Resultados y Discusión.** Se analiza el grado de cumplimiento de los objetivos planteados, se presenta la aplicación final y se discuten los resultados obtenidos, destacando fortalezas, limitaciones y los diversos problemas durante el desarrollo del proyecto.

- **Capítulo 7: Conclusiones y Trabajo Futuro.** Se exponen las conclusiones principales derivadas del desarrollo del TFG, el aprendizaje adquirido y se proponen posibles líneas de trabajo futuro para continuar mejorando y expandiendo "SmartFreezer".
- **Bibliografía y Referencias:** Se listan todas las fuentes consultadas y citadas a lo largo del documento.
- **Anexos:** Se incluye material complementario que puede ser de utilidad para una comprensión más profunda de aspectos específicos del proyecto.

1.4 Metodología empleada

El desarrollo de *SmartFreezer* se ha llevado a cabo siguiendo una metodología ágil, adaptada a un entorno individual. Aunque SCRUM está orientado a equipos, se ha utilizado como marco de referencia para organizar el trabajo de forma iterativa e incremental, dividiendo el desarrollo en pequeñas entregas funcionales a lo largo del tiempo.

1.4.1 Ciclo de vida del software

He seguido una adaptación del marco **SCRUM** (4) en solitario, donde he asumido todos los roles (product owner, scrum master y desarrollador). Esto ha permitido una planificación flexible del trabajo, revisiones periódicas de avances y reajustes de prioridad en función de la dificultad técnica de cada fase. Las tareas se han dividido en "semanales", con objetivos claros como:

- Diseño de prototipos mediante Figma (5)
- Análisis de requisitos y diagrama de casos de uso
- Entrenamiento del modelo YOLOv8
- Diseño de UI y UX
- Desarrollo del sistema de inventario
- Integrar Firebase (Auth, Firestore, FCM)
- Implementación del sistema de recetas con Spoonacular API
- Integración del modelo de visión artificial
- Pruebas y ajustes de rendimiento



Ilustración 2: Metodología SCRUM

En la anterior Ilustración 2: Metodología SCRUM se refleja el proceso de metodología ágil **SCRUM**.

Los pilares fundamentales de **SCRUM** son su transparencia, es decir, todos los aspectos proyecto deben ser visibles para todos, además de su inspección y adaptación a lo que se refiere que el proyecto debe ser revisado regularmente para realizar ajustes basados en la inspección continua para mejorar.

Los roles claves son:

- **Product Owner:** Hace de cliente y debe decir qué es lo más importante del **Product Backlog** (la lista de todo lo que se necesita en el producto).
- **Scrum Master:** Facilita el proceso de Scrum, apoya al equipo a quitar obstáculos y se asegura que se sigan las prácticas de Scrum.
- **Equipo de Desarrollo:** Es el grupo de profesionales que realizan el trabajo de construir el producto. Se organizan solos y saben hacer de todo un poco.

Sobre cómo fluye el trabajo, se empieza con la **Planificación del Sprint**, donde se define el trabajo que hay que realizar. Diariamente, el equipo se pone al día en el **Daily Scrum**. Al final del Sprint, se revisa el incremento en la **Sprint Review** y se reflexiona sobre el proceso en la **Retrospectiva del Sprint**.

En resumen, SCRUM se enfoca en la entrega temprana y continua de valor, en la adaptación a los cambios en lugar de seguir un plan fijo y en la continua mejora continua del proceso.

1.4.2 Herramientas empleadas

Para facilitar la organización y trazabilidad del proyecto, se han empleado diversas herramientas:

- **GitHub:** Control de versiones del código fuente, permitiendo retroceder a versiones anteriores y documentar cambio.
- **Firebase Console:** Configuración y pruebas de servicios en la nube como Firestore, Firebase Auth y Firebase Cloud Messaging.
- **Android Studio:** Entorno de desarrollo principal para el código Kotlin y la integración de librerías ML.
- **Figma:** Plataforma de diseño en línea de interfaces de usuario y experiencia de usuario, que permite crear prototipos de forma simultanea desde cualquier lugar.
- **Pycharm:** Entorno de desarrollo para preparación y entrenamiento del modelo ML diseñado por JetBrains para el lenguaje de programación Python.
- **Spoonacular API:** Interfaz de programación de aplicaciones (API) que proporciona acceso a una amplia base de datos de información sobre recetas, alimentos y nutrición.

GitHub



Ilustración: Logotipo GitHub

GitHub (6) es una plataforma web esencial para desarrolladores, adquirida por Microsoft en 2018. Su principal función es servir de hogar y centro de gestión para el código fuente de nuestras apps, todo ello gracias al sistema de control de versiones **Git**. Básicamente, permite a los desarrolladores subir sus proyectos, de forma organizada y mantener su código en repositorios tanto públicos para código abierto como privados con suscripción, y así controlar las distintas ediciones de su programa, lo que previene que se borren versiones antiguas al actualizar. La plataforma ofrece diversas herramientas propias que complementan Git, como wikis para documentación, sistemas de seguimiento de problemas para reportar errores y

sugerencias, y herramientas de revisión de código para analizar y comentar el código fuente.

A través de los IDE, puedo implementar un control de versiones Git mediante mi token único de Github.

El control de Versiones con Git se estructura de la siguiente forma:

- **Repositorios:** GitHub aloja proyectos en forma de repositorios Git. Un repositorio contiene todos los archivos del proyecto y el historial de todos los cambios realizados en ellos.
- **Commits:** Los cambios en los archivos se guardan como "commits", cada uno con un mensaje que describe la modificación. Esto permite rastrear la evolución del código a lo largo del tiempo.
- **Branches (Ramas):** Permiten trabajar en paralelo en diferentes funcionalidades o correcciones sin afectar la rama principal del proyecto. Luego, los cambios de una rama se pueden fusionar ("merge") con la rama principal.
- **Pull Requests (Solicitudes de extracción):** Son la forma principal de proponer cambios a un repositorio. Permiten discutir el código propuesto, revisarlo y realizar comentarios antes de integrarlo a la rama principal.

En la siguiente Ilustración 3: Repositorio GitHub SmartFreezer se observa el repositorio de mi proyecto

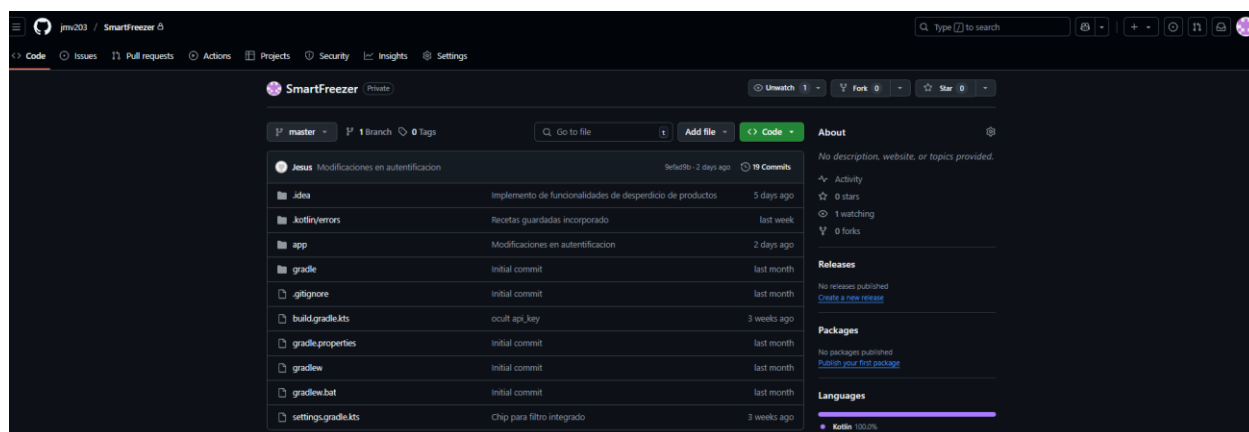


Ilustración 3: Repositorio GitHub SmartFreezer

Firestore Console



Ilustración 4: Logotipo Firebase

Firebase es un servicio en la nube creado por Google que facilita la creación de software al usar un servicio que no necesita “servidores”, por lo que no hay necesidad de cuidar su propia configuración de servidor, lo que significa que los desarrolladores no tienen que usar PHP o gestionar bases de datos relaciones como MySQL, permitiéndoles poder concentrarse en la apariencia y la funcionalidad de sus aplicaciones móviles y web.

Firebase utiliza el **servicio de backend (BAAS)**, este se refiere a un sistema que supervisa las operaciones estándar de backend como almacenamiento de datos, autenticación del usuario, notificaciones push y características adicionales.

Android Studio (7)



Ilustración 5: Logotipo Android Studio

Google lanzó Android Studio como un novedoso entorno de desarrollo integrado, con el objetivo de hacer más fácil la creación de apps para el sistema operativo Android y con la idea de reemplazar a Eclipse, que hasta ese momento era el IDE predilecto. Los proyectos en Android Studio poseen una estructura donde casi todos los ficheros se hallan en la carpeta SRC, lo cual constituye parte de un sistema de compilación fundamentado en Gradle que aporta más adaptabilidad al curso del desarrollo. Aparte, Android Studio engloba un emulador incorporado que hace posible visualizar las modificaciones en la app al instante y verificar su apariencia en diversos dispositivos Android con distintas configuraciones y tamaños de pantalla a la vez.

Pycharm (8)



Ilustración 6: Logotipo Pycharm

PyCharm se presenta como un entorno de desarrollo integrado (IDE) orientado para Python, muy popular en proyectos de IA y visión artificial (8), como el entrenamiento de esos modelos avanzados tipo YOLOv8. Fue creado por JetBrains, esta herramienta nos da un entorno sólido y flexible que simplifica cada paso al desarrollar modelos de Machine Learning, desde preparar los datos, hasta entrenar y evaluar el modelo. Una de las cosas que mejor hace es integrarse con un montón de librerías y frameworks de inteligencia artificial, incluyendo **TensorFlow**, **PyTorch**, **OpenCV**, **scikit-learn**, **Pandas**, **NumPy**, y muchísimas más. Estas librerías son clave para preprocesar imágenes, construir redes neuronales, analizar datos estadísticos y mostrar los resultados de forma visual.

Además, PyCharm trae herramientas avanzadas que hacen que el desarrollado sea más productivo y sencillo:

- Un gestor de paquetes para instalar y manejar librerías sin complicaciones.
- Control de versiones integrado con Git para que el trabajo en equipo y el manejo de versiones del código sean más sencillos.
- Terminal integrado, autocompletado inteligente y una forma eficiente de moverse entre archivos.

Con todo esto, PyCharm hace más fácil desarrollar scripts complejos para preparar datasets, ajustar hiperparámetros, entrenar modelos como YOLOv8 y exportarlos a formatos que se pueden usar en móviles, como ONNX.

Spoonacular API

La **Spoonacular API** es una interfaz RESTful (9) que proporciona acceso a una base de datos de recetas, ingredientes y de información nutricional. Mediante esta API es posible consultar recetas filtrando por ingredientes, un tipo de dieta específico, sus intolerancias alimentarias, el tiempo de preparación de una receta y el nivel calórico, entre otros parámetros.

Para la app SmartFreezer, la API de Spoonacular permite obtener recetas adaptadas a los ingredientes disponibles del usuario, lo que contribuye a reducir el desperdicio de alimentos. Esta integración se realiza mediante peticiones HTTP que devuelven datos en formato JSON, los cuales se procesan en la aplicación para presentarlos al usuario de manera clara e interactiva.

Tarjeta gráfica NVIDIA

Para el entrenamiento del modelo YOLOv8 es recomendado utilizar herramientas NVIDIA. Por lo que la tarjeta gráfica Geforce GTX 1060 6GB será utilizada para el proceso.



Ilustración 7: Tarjeta gráfica NVIDIA

1.4.3 Modelado formal

En este apartado detalla el proceso de análisis y diseño seguido para el desarrollo de la aplicación “SmartFreezer”, cubriendo aspectos funcionales, de viabilidades, del sistema, la metodología del desarrollo y el diseño de la interfaz de usuario.

Análisis funcional

Definir capacidades y comportamiento esperado del sistema desde la perspectiva del usuario.

“Smartfreezer” se centra en el objetivo de facilitar la gestión de alimentos en un hogar de forma automática, si es cierto que no se ha abordado la posibilidad de instalar una cámara para poder registrar estos productos, sin embargo, para simular este proceso, de forma manual se ha implementado el uso de la cámara del dispositivo móvil para la

detección y reconocimiento de tales productos. Por otro lado, se destaca las distintas funcionalidades que posee la aplicación dividida en cinco módulos principales. La primera a destacar es la posibilidad de añadir, modificar y eliminar tus productos en distintas ubicaciones, teniendo un control de su estado y además de poder supervisar el desperdicio obtenido de estos productos mediante sencillas gráficas.

También, se podrá añadir y eliminar productos a una lista de la compra para satisfacer el control de las visitas al supermercado. Además, se destaca las consultas personalizadas de diversas recetas con la funcionalidad de guardar cualquier receta a una lista. Por antepenúltimo, en la app habrá una función de alertas que notificará al usuario de productos en mal estado como de recomendaciones de productos en temporada que podrá añadir a su lista de la compra. Y, por último, la función de escáner anteriormente mencionada en la que un usuario registrará un producto y podrá añadirlo a su inventario. Cabe mencionar los siguientes aspectos, el usuario podrá modificar sus credenciales o personalizar su experiencia de navegación mediante los apartados de perfil y configuración, además de ajustar sus preferencias dietéticas como intolerancias para recibir notificaciones personalizadas a sus preferencias. Otros aspectos, es la capacidad de modificar sus credenciales, aplicar un tema oscuro, cambiar de idioma (inglés o español), establecer una reseña, cerrar sesión, ver información acerca de la app y eliminar su cuenta.

Puntos a tener en cuenta:

- El usuario deberá estar autenticado en la app antes de poder acceder a todas las funciones mencionadas previamente.
- El usuario deberá verificar su correo electrónico antes de iniciar sesión en la app.
- El usuario no podrá escanear más de una fruta o verdura simultáneamente.

Análisis de requisitos

Este análisis se enfoca en los componentes principales del sistema "SmartFreezer" y sus interacciones, además de obtener un catálogo de los requisitos del sistema e identificar a los actores que intervienen en el sistema. Para ello haré uso de las practicas aprendidas en la asignatura de Desarrollo de Interfaces de Usuario. En cada una de las siguientes plantillas se reflejará una descripción que describe la funcionalidad del requisito, una precondition que detalla un evento previo para satisfacer el requisito, la secuencia de interacción paso a paso de cada acción junto a

los escenarios de éxito o error (excepción), y finalmente, los resultados de cada acción (postcondición).

Actores del sistema

A-001	Usuario no autenticado
Descripción	Tiene la posibilidad de iniciar sesión o darse de alta en la aplicación

Tabla 1: Especificación del actor no autenticado

A-002	Usuario autenticado
Descripción	Tiene la posibilidad de navegar a todas las funcionalidades de la aplicación

Tabla 2: Especificación del actor autenticado

Usuarios del sistema

U-001	Usuario no autenticado
Descripción	Puede iniciar sesión o registrarse en la aplicación
Comentarios	No tiene acceso a las funcionalidades principales de la aplicación hasta que inicie sesión o se registre

Tabla 3: Especificación del usuario autenticado

U-002	Usuario autenticado
Descripción	Tiene acceso completo a todas las funcionalidades de la aplicación
Comentarios	Para llegar a este estado, el usuario debe haberse autenticado previamente

Tabla 4: Especificación del usuario no autenticado

Requisitos funcionales

RF-1	Registro de nuevos usuarios	
Descripción	El sistema permitirá a un nuevo usuario crear una cuenta proporcionando un nombre, una dirección de correo electrónico válida y una contraseña. El sistema también solicitará al usuario aceptar los términos y condiciones y la política de privacidad.	
Precondición	El usuario no tiene una cuenta existente con el correo electrónico proporcionado. El usuario se encuentra en la pantalla de registro de la aplicación.	
Secuencia normal	Paso	Acción
	1	El usuario introduce su nombre, dirección de correo electrónico, contraseña en los campos designados y marca casilla de términos y condiciones
	2	El usuario pulsa el botón "Registrar".
	3	El usuario recibe un correo de verificación en la dirección proporcionada
	4	El sistema lo redirige a la pantalla de "Iniciar sesión"
Postcondición	Se crea una nueva cuenta de usuario en Firebase Authentication pendiente de verificación de correo.	
Excepciones	Paso	Acción
	2	El formato de las credenciales no es válido. El sistema muestra un mensaje de error
	2	El correo electrónico ya está en uso por otra cuenta
	2	El usuario no acepta los términos y condiciones. Muestra un aviso

Tabla 5: RF-1 – Registro de nuevos usuarios

RF-2	Autenticación de usuarios existentes	
Descripción	El sistema permitirá a un usuario registrado iniciar sesión con su correo electrónico y contraseña.	
Precondición	El usuario tiene una cuenta existente y se encuentra en la pantalla de inicio de sesión	
Secuencia normal	Paso	Acción
	1	El usuario introduce su correo electrónico y contraseña. Marca la opción “Mantener sesión iniciada” (opcional)
	2	El usuario pulsa el botón "Iniciar Sesión".
	3	El sistema valida que los campos no estén vacíos y verifica las credenciales contra Firebase Auth
	4	Si las credenciales son correctas y el email está verificado, el sistema guarda y carga las preferencias del usuario de Firestore (para suscripción a topics de FCM). Si el usuario selecciono la opción “Mantener sesión iniciada”, el sistema guarda esta preferencia
Postcondición	El usuario ha iniciado sesión en la aplicación y tiene acceso a sus funcionalidades. Las preferencias de sesión se guardan si se indicó. Además, el sistema guarda en un documento Firestore los valores del usuario	
Excepciones	Paso	Acción
	2	El formato de las credenciales no es válido. El sistema muestra un mensaje de error
	2	La cuenta de correo electrónico no ha sido verificada. El sistema muestra un mensaje indicando que se debe verificar el correo y no permite el inicio de sesión.

Tabla 6: RF-2 - Autenticación de usuarios existentes

RF-3	Recuperación de contraseñas	
Descripción	El sistema permitirá a un usuario que ha olvidado su contraseña solicitar un enlace para restablecerla a través de su correo electrónico	
Precondición	El usuario se encuentra en la pantalla de inicio de sesión y pulsa el enlace "¿Has olvidado tu contraseña?"	
Secuencia normal	Paso	Acción
	1	El sistema muestra un diálogo solicitando el correo electrónico del usuario
	2	El usuario introduce su correo electrónico y pulsa "Enviar"
	3	El sistema valida que el campo de correo no esté vacío y tenga un formato válido
	4	El sistema utiliza Firebase Authentication para enviar un correo de restablecimiento de contraseña a la dirección proporcionada. El sistema informa al usuario que se ha enviado un correo si la cuenta existe
Postcondición	Se envía un correo electrónico al usuario (si la cuenta asociada a ese correo existe) con instrucciones para restablecer su contraseña	
Excepciones	Paso	Acción
	2	El formato de las credenciales no es válido. El sistema muestra un mensaje de error
	2	No existe ninguna cuenta asociada al correo electrónico proporcionado (Firebase no da esta información directamente por seguridad, pero el correo no se enviará)

Tabla 7: RF-3 - Recuperación de contraseñas

RF-4	Gestión del perfil de usuario	
Descripción	El sistema permitirá al usuario ver y modificar la información de su perfil, incluyendo nombre y preferencias dietéticas (vegano, vegetariano, sin gluten, sin lactosa). También permitirá cambiar el correo electrónico asociado a la cuenta y restablecer la contraseña.	
Precondición	El usuario ha iniciado sesión y navega a la pantalla de perfil	
Secuencia normal	Paso	Acción
	1	El usuario pulsa una opción de modificación
	2	El sistema muestra un diálogo para introducir los nuevos datos, posteriormente muestra un diálogo de confirmación donde el usuario debe introducir su contraseña
	3	El sistema utiliza Firebase Auth para validar que sea válido los datos enviados y muestra un dialogo de confirmación
	4	El sistema actualiza los datos correspondientes del usuario en Firebase. Si el usuario ha modificado sus preferencias dietéticas el sistema actualiza las suscripciones a los topics de FCM según las nuevas preferencias.
Postcondición	La información del perfil del usuario (nombre, email, preferencias) se actualiza en Firestore y, si aplica, en Firebase Authentication. Las suscripciones a notificaciones se actualizan. La UI refleja los cambios.	
Excepciones	Paso	Acción
	2	El formato de las credenciales no es válido. El sistema muestra un mensaje de error
	2	Error al intentar cambiar el correo electrónico (ej. nuevo email ya en uso, contraseña actual incorrecta para reautenticación). El sistema muestra un mensaje de error detallado.

Tabla 8: RF-4 - Gestión del perfil de usuario

RF-5 Cierre de sesión y eliminación de cuenta		
Descripción	El sistema permitirá al usuario cerrar su sesión actual. El sistema también permitirá al usuario eliminar permanentemente su cuenta y todos los datos asociados	
Precondición	El usuario ha iniciado sesión y navega a la pantalla de perfil	
Secuencia normal	Paso	Acción
	1	El usuario pulsa la opción "Cerrar Sesión"
	2	El sistema muestra un diálogo de confirmación
	3	El sistema cierra la sesión del usuario en Firebase Auth y elimina cualquier preferencia de "Mantener sesión iniciada" guardada
Secuencia normal	1	El usuario pulsa la opción "Eliminar Cuenta"
	2	El sistema muestra un diálogo advirtiéndole que la acción es irreversible y solicitando confirmación.
	3	El sistema solicita al usuario que reintroduzca su contraseña para confirmar su identidad
	4	El sistema verifica la contraseña y elimina todos los datos del usuario de Cloud Firestore
Postcondición	El usuario ya no está autenticado en la aplicación. Si la opción ha sido eliminar cuenta la cuenta del usuario y todos sus datos asociados son eliminados permanentemente del sistema.	
Excepciones	Paso	Acción
	2 (Eliminar cuenta)	Contraseña incorrecta durante la reautenticación para eliminar cuenta. El sistema no procede con la eliminación y muestra un mensaje de error.

Tabla 9: RF-5 - Cierre de sesión y eliminación de cuenta

RF-6	Escaneo de frutas y verduras	
Descripción	El sistema permitirá al usuario utilizar la cámara del dispositivo móvil para escanear frutas y verduras. El sistema las detectará y clasificará automáticamente utilizando un modelo YOLOv8	
Precondición	El usuario ha iniciado sesión y navega a la pantalla de escáner	
Secuencia normal	Paso	Acción
	1	El usuario enfoca una fruta o verdura con la cámara del dispositivo.
	2	El modelo YOLOv8 integrado analiza los frames para detectar y clasificar objetos
	3	El sistema presenta la información del producto detectado y permite al usuario confirmar o corregir la información antes de añadirlo al inventario
Postcondición	El producto detectado se muestra al usuario, listo para ser confirmado y añadido al inventario	
Excepciones	Paso	Acción
	1	El permiso de cámara no ha sido concedido. El sistema solicita el permiso o informa al usuario de la necesidad de concederlo

Tabla 10: RF-6 - Escaneo de frutas y verduras

RF-7	Añadir productos al inventario	
Descripción	El sistema permitirá al usuario añadir productos a su inventario de forma manual, seleccionando la categoría, el producto específico dentro de esa categoría (si aplica), e introduciendo detalles como la cantidad y la fecha de caducidad.	
Precondición	El usuario ha iniciado sesión y se encuentra en la sección de inventario, seleccionando la opción para añadir un nuevo producto	
Secuencia normal	Paso	Acción
	1	El usuario selecciona la categoría del producto
	2	El usuario selecciona un producto específico y añade su ubicación, estado y cantidad
	3	El usuario pulsa el botón “Guardar”
	4	El sistema guarda el nuevo producto en Cloud Firestore en el documento del usuario. El sistema muestra un mensaje de confirmación
Postcondición	Un nuevo producto es añadido al inventario del usuario en Firestore y es visible en la interfaz de la aplicación	
Excepciones	Ninguna	

Tabla 11: RF-8 - Añadir productos al inventario

RF-8	Creación y gestión de lista de la compra	
Descripción	El sistema permitirá a los usuarios crear y gestionar listas de la compra, añadiendo productos o eliminándolos	
Precondición	El usuario ha iniciado sesión y navega a la sección de lista de la compra	
Secuencia normal	Paso	Acción
	1	El usuario pulsa el botón para añadir un nuevo producto a la lista
	2	El usuario selecciona un producto específico
	3	El sistema guarda el nuevo producto en Cloud Firestore en el documento del usuario. El sistema muestra un mensaje de confirmación
Postcondición	Un nuevo producto es añadido a la lista de compra del usuario en Firestore y es visible en la interfaz de la aplicación	
Excepciones	Ninguna	

Tabla 12: RF-9 - Creación y gestión de lista de la compra

RF-9	Registro de productos desperdiciados	
Descripción	El sistema permitirá al usuario marcar productos de su inventario como "desperdiciados" o registrar productos que se han desperdiciado. Esta información se utilizará para generar estadísticas	
Precondición	El usuario tiene productos en su inventario y selecciona uno en específico	
Secuencia normal	Paso	Acción
	1	El usuario selecciona un producto del inventario y elige la opción "Marcar como desperdiciado"
	2	El sistema registra la información del producto desperdiciado en Cloud Firestore en el documento del usuario. El sistema muestra un mensaje de confirmación
	3	El registro se visualizará en la pantalla de desperdicios
Postcondición	El producto desperdiciado queda registrado en Firestore para su posterior análisis. El inventario principal se actualiza si corresponde	
Excepciones	Ninguna	

Tabla 13: RF-9 - Registro de productos desperdiciados

RF-10	Filtrar recetas	
Descripción	El sistema permitirá al usuario filtrar la lista de recetas por diversos criterios como tipo de plato (ej. plato principal, ensalada, postre), tipo de dieta (ej. vegano, vegetariano, sin gluten, sin lactosa), e ingredientes específicos que desea incluir o excluir de su inventario	
Precondición	El usuario se encuentra en la sección de recetas y hay una lista de recetas visible o se va a realizar una búsqueda	
Secuencia normal	Paso	Acción
	1	El usuario abre el diálogo o panel de filtros y selecciona los criterios de filtros deseados
	2	El usuario aplica los filtros.
	3	El sistema actualiza la lista de recetas mostrada para reflejar los filtros aplicados. Además, muestra un indicador visual del número de filtros aplicados y un botón para limpiarlos
Postcondición	La lista de recetas se actualiza mostrando solo aquellas que cumplen con los criterios de filtro seleccionados por el usuario. Los indicadores de filtro se actualizan	
Excepciones	Ninguna	

Tabla 14: RF-10 - Filtrar recetas

RF-11	Guardar recetas	
Descripción	El sistema permitirá a los usuarios marcar recetas como favoritas y acceder a una lista de sus recetas guardadas	
Precondición	El usuario ha iniciado sesión y está viendo una lista de recetas o los detalles de una receta	
Secuencia normal	Paso	Acción
	1	El usuario pulsa el icono o botón "Guardar" en una receta.
	2	El sistema registra la información de la receta en Cloud Firestore en el documento del usuario. El sistema muestra un mensaje de confirmación
Postcondición	El estado de guardado de la receta se actualiza en Firestore y en la UI. El usuario puede acceder a su lista de recetas favoritas	
Excepciones	Ninguna	

Tabla 15: RF-11 - Guardar recetas

RF-12	Gestionar alertas	
Descripción	El sistema podrá sugerir añadir productos a la lista de la compra basados en su fecha de temporada o podrá sugerir eliminar productos del inventario en caso de ser detectado en mal estado	
Precondición	El usuario ha iniciado sesión y está viendo una lista de alertas	
Secuencia normal	Paso	Acción
	1	El usuario pulsa el botón "Añadir" en la alerta de productos en temporada y se añade a la lista de la compra
	2	El usuario pulsa el botón "Eliminar" en la alerta de productos en mal estado y se elimina del inventario
	3	Ambas opciones modifican los datos del documento del usuario en Cloud Firestore
Postcondición	El estado se refleja en la UI	
Excepciones	Ninguna	

Tabla 16: RF-12 - Gestionar alertas

RF-13	Añadir valoración	
Descripción	El sistema permitirá a los usuarios proporcionar una valoración general de la aplicación	
Precondición	El usuario ha iniciado sesión y navega a la pantalla de envío de feedback	
Secuencia normal	Paso	Acción
	1	El usuario rellenar el formulario
	2	La valoración se incluirá como parte del feedback general cuando el usuario pulse "Enviar Feedback"
Postcondición	La valoración general seleccionada por el usuario se almacena como parte de su envío de feedback en Firestore	
Excepciones	Ninguna	

Tabla 17: RF-13 - Añadir valoración

RF-14	Recibir notificación nueva receta recomendada	
Descripción	El sistema enviará notificaciones push (utilizando Firebase Cloud Messaging) a los usuarios para notificaciones sobre nuevas recetas recomendadas basadas en sus preferencias	
Precondición	El usuario ha iniciado sesión y ha concedido permisos para recibir notificaciones	
Secuencia normal	Paso	Acción
	1	El sistema genera un mensaje de notificación para el usuario
	2	El dispositivo del usuario recibe la notificación y la muestra, incluso si la aplicación no está en primer plano
	3	Al pulsar la notificación, la aplicación se abre y dirige al usuario a la receta nueva sugerida
Postcondición	El usuario recibe una notificación push en su dispositivo	
Excepciones	El usuario ha deshabilitado las notificaciones para la aplicación en los ajustes del dispositivo	

Tabla 18: RF-14 - Recibir notificación nueva receta recomendada

RF-15	Configuración	
Descripción	El sistema permitirá al usuario acceder a una sección de configuración desde el menú principal, donde podrá cambiar el idioma de la interfaz, activar o desactivar el modo oscuro, y habilitar o deshabilitar las notificaciones.	
Precondición	El usuario ha iniciado sesión y navega a la pantalla de configuración	
Secuencia normal	Paso	Acción
	1	El usuario accede a la sección de configuración desde el menú
	2	Elige una de las opciones: cambiar idioma, activar/desactivar modo oscuro, o notificaciones
	3	El sistema guarda las preferencias y actualiza la interfaz en tiempo real
Postcondición	La configuración seleccionada por el usuario queda guardada y se aplica inmediatamente o en el siguiente inicio de sesión	
Excepciones	Ninguna	

Tabla 19: RF-15: Configuración del sistema

Requisitos no funcionales

RNF-1	Mantenibilidad
Descripción	El código debe estar estructurado siguiendo una arquitectura limpia (MVVM) que permita fácil mantenimiento, escalado y depuración. Además, de utilizar un repositorio de control de versiones
Comentarios	Se favorece la extensión del proyecto a futuro y la corrección de errores sin comprometer otras funcionalidades.

Tabla 20: RNF-1 – Mantenibilidad

RNF-2	Seguridad
Descripción	La autenticación de usuarios debe realizarse mediante Firebase Auth , con cifrado seguro para contraseñas y datos sensibles
Comentarios	Se protegerán los datos del usuario, cumpliendo buenas prácticas de ciberseguridad.

Tabla 21: RNF-2 – Seguridad

RNF-3	Rendimiento
Descripción	El tiempo de respuesta debe ser mínimo para que el usuario no espere demasiado
Comentarios	Se intentará reducir los tiempos de carga al mínimo posible. No se contabilizarán retrasos derivados de la conexión a Internet.

Tabla 22: RNF-3 – Rendimiento

RNF-4	Accesibilidad
Descripción	La aplicación debe ofrecer un contraste adecuado de colores y tipografías legibles.
Comentarios	Se usarán colores accesibles y fuentes claras para personas con dificultades visuales. También se evaluará el tamaño de botones y espacios interactivos.

Tabla 23: RNF-4 - Accesibilidad

RNF-5	Usabilidad
Descripción	La interfaz debe ser intuitiva y accesible para usuarios no técnicos. Todos los textos e iconos deben seguir principios de diseño UX, con botones claramente identificables y navegación fluida
Comentarios	Se busca una experiencia de usuario óptima, asegurando que cualquier persona pueda usar la app sin formación previa

Tabla 24: RNF-5 – Usabilidad

RNF-6	Portabilidad
Descripción	La app debe ser compatible con una amplia gama de dispositivos Android con diferentes resoluciones y versiones del sistema operativo
Comentarios	Se utilizarán técnicas de diseño adaptable (responsive design) para garantizar funcionalidad y apariencia en distintos dispositivos.

Tabla 25: RNF-6 – Portabilidad

1.4.4 Modelado conceptual

Se ha realizado un modelado conceptual utilizando diagramas de casos de uso, Ilustración 8: Diagrama de casos de uso sobre la aplicación, para representar las interacciones entre los usuarios y la aplicación. Este modelo permite identificar los principales actores y las funcionalidades clave del sistema. Facilita la planificación y validación de los requisitos funcionales del proyecto.

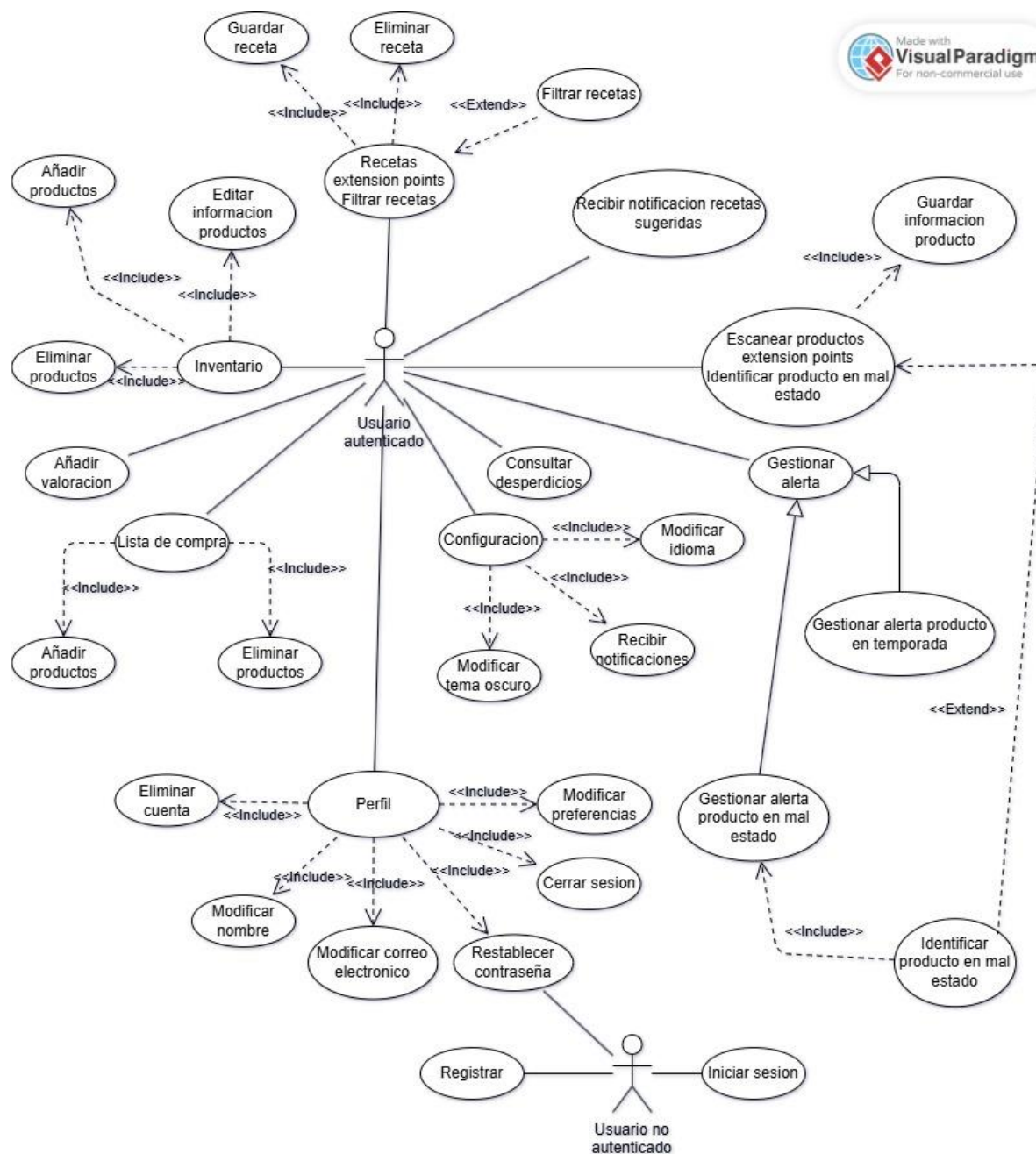
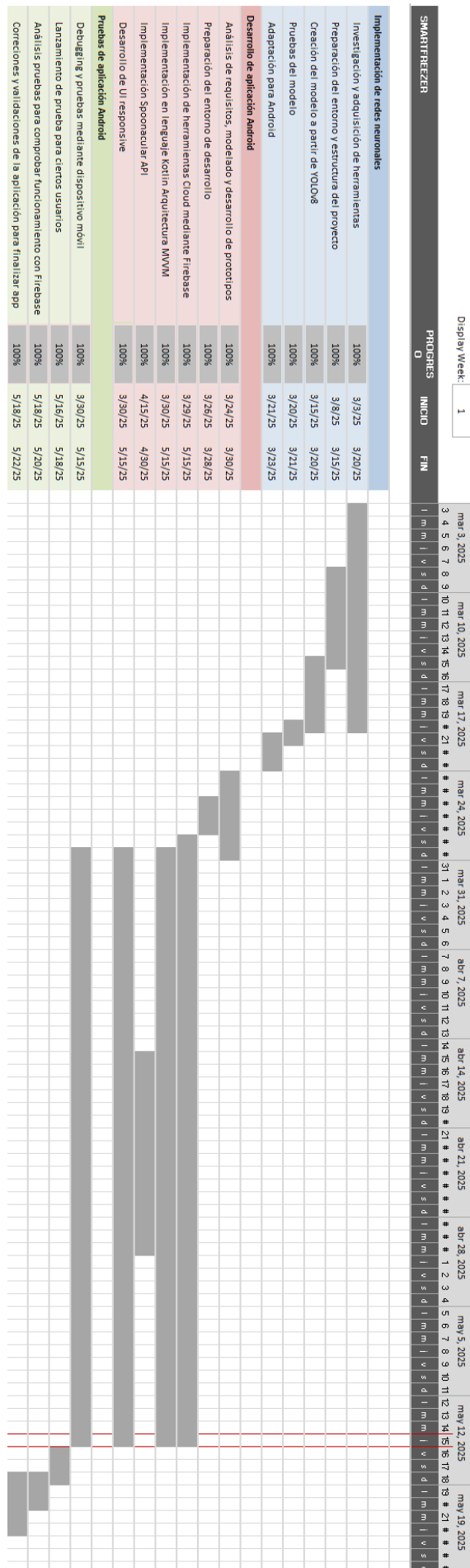


Ilustración 8: Diagrama de casos de uso sobre la aplicación

1.5 Planificación temporal

En la siguiente página, se podrá observar un diagrama de Gantt que representa la planificación temporal en días con la que se ha supuesto llevar a cabo este proyecto de fin de grado.



1.6 Diseño del prototipo

En esta sección se expondrá los diversos diseños de prototipos realizados previamente con la herramienta Figma, con la finalidad de garantizar una interfaz intuitiva, moderna y centrada al usuario. Además, de establecer ciertos objetivos como el flujo de navegación entre distintas pantallas y la decisión sobre los estilos visuales, tipografías, colores e iconografía.

En estos diseños he intentado lograr una interfaz clara y limpia, pero a la vez visualmente atractiva, con colores azules y morados representando un frigorífico. Esta idea empezó con el diseño del logotipo de la aplicación que aplica estos colores anteriores mencionados, este logotipo representa un cajón frigorífico haciendo referencia a la función de la aplicación Ilustración 9. Asimismo, otros ciertos aspectos han sido productos de fuente de inspiración de otras aplicaciones de recetas, nutrición, deporte...



Ilustración 9: Logotipo SmartFreezer

A continuación, se mostrará cada una de las pantallas realizadas para el prototipo de la aplicación, cabe resaltar que estos prototipos no representan el resultado final de la aplicación puesto que se han ampliado numerosas mejoras a esta:



Ilustración 10: Prototipo pantalla de Inicio



Ilustración 11: Prototipo pantalla Inicio Sesión



Ilustración 12: Prototipo pantalla Registro



Ilustración 13: Prototipo pantalla Inventario



Ilustración 14: Prototipo pantalla Detalles Producto



Ilustración 15: Prototipo pantalla Desperdicios



Ilustración 16: Prototipo pantalla Añadir Producto Inventario

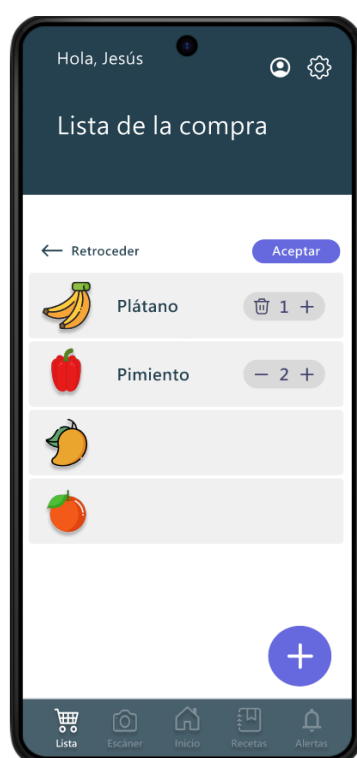


Ilustración 17: Prototipo pantalla Lista de Compra



Ilustración 18: Prototipo pantalla Añadir Producto Lista de Compra



Ilustración 19: Prototipo pantalla Recetas Disponibles



Ilustración 20: Prototipo pantalla Detalles Receta



Ilustración 21: Prototipo pantalla Alertas



Ilustración 22: Prototipo pantalla Escáner



Ilustración 23: Prototipo pantalla Perfil

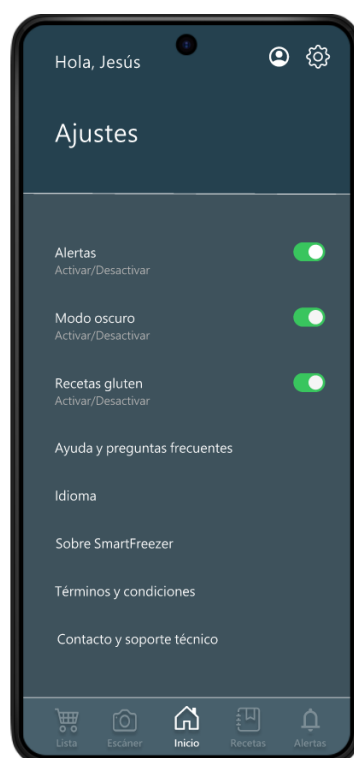


Ilustración 24: Prototipo pantalla Ajustes

2. Estado del arte / Antecedentes y Marco teórico

En este capítulo voy a comentar los aspectos teóricos y tecnológicos que se abordan en el proyecto, presentando los conceptos clave y las herramientas empleadas. Además, de poner en contexto los problemas que se pretenden resolver.

2.1 Gestión de Alimentos y Desperdicio Alimentario

Hoy en día a nivel mundial, resulta muy complicado la tarea de gestionar los alimentos y su desperdicio. Según el informe, la Organización de las Naciones Unidas para la Alimentación y Agricultura y el Programa de las Naciones Unidas para el Medio Ambiente (10), ha registrado que aproximadamente el 17% de los alimentos producidos se desperdician, con un resultado de 931 millones de toneladas de comida desperdiciada en restaurantes, supermercado, o a nivel de consumidor. Esto ha implicado una pérdida tanto económica como medioambiental, representando un nivel del 10% en emisiones globales como los gases de efecto invernadero.

Entrando en contexto con la pandemia del COVID-19 esto afectó a los sistemas de producción, distribución y consumo debido a restricciones de los sistemas de agroalimentarios, con la consecuencia de una pérdida de ingresos potencial. En 2021, 828 millones de personas parecieron hambre aumentando así en 46 millones respecto al año anterior. Además 2.300 millones enfrentaron una inseguridad alimentaria grave, 350 millones más que antes que la pandemia. Por otro lado, la guerra de Ucrania y Rusia interrumpió exportaciones clave, elevando precios y desestabilizando mercados agrícolas globales. En 2024, un número mayor de 295 millones de personas en 53 distintos países se enfrentan con crisis alimentarias. Conflictos, desastres climáticos y choques económicos son unas de las causas principales de este gran problema que nos engloba. El desperdicio de alimentos agrava el cambio climático, afectando a la producción y a la disponibilidad de los alimentos a nivel global, así generando un ciclo negativo para la seguridad alimentaria. Todos estos datos han sido recogidos según FAO (11).

Está claro que resulta difícil tener un control ante todo estos tipos de sucesos que engloban nuestro mundo, sin embargo, uno de los aspectos más comunes del desperdicio de alimentos se trata a la hora de gestionar el inventario doméstico. Esto se suele deberse a la falta de herramientas que permitan al usuario tener un control efectivo sobre los productos almacenados. Diversas iniciativas tecnológicas han intentado abordar este problema mediante el uso aplicaciones móviles de intercambio de alimentos, un ejemplo es la funcionalidad de “Too Good To Go” (12) que conecta a los usuarios con establecimientos que tienen un exceso de comida al final del día el cual los usuarios puedan salvar mediante “packs sorpresas” a precios reducidos. Otro ejemplo, aunque aún no masivo, son los refrigeradores inteligentes como algunos modelos de Samsung (13). Gracias a sus cámaras internas, el usuario puede verificar el inventario de su nevera en tiempo real, incluso estando en el supermercado. No obstante, es posible que un refrigerador sea una solución con una alta inversión económica para los usuarios.

Ante este contexto, este proyecto propone una solución basada en visión artificial y detección automática de productos, lo cual resulta más accesible para el usuario y reduce su esfuerzo al mínimo. Además, se complementa con funcionalidades de recomendación inteligente de recetas y notificaciones, con el fin de fomentar el consumo eficiente de los alimentos disponibles y reducir así el desperdicio.

2.2 Tecnologías de Visión Artificial y Deep Learning

La visión artificial es una rama de la inteligencia artificial que permite a los sistemas interpretar y comprender el contenido visual del entorno, como imágenes y vídeos. Una de sus aplicaciones más destacadas es la **detección de objetos**, que consiste en identificar y localizar múltiples elementos dentro de una imagen, asignándoles una clase específica y delimitándolos mediante cuadros o máscaras.

El **Deep Learning** (14), o aprendizaje profundo, es un subcampo del aprendizaje automático que utiliza **redes neuronales artificiales** para modelar y resolver tareas complejas. Estas redes están compuestas por múltiples capas que transforman progresivamente los datos de entrada en representaciones más abstractas, permitiendo reconocer patrones complejos y realizar tareas como la clasificación y detección de objetos.

Las imágenes son **datos estructurados en rejilla**. Para procesarlos, se emplean **redes neuronales convolucionales (CNN)**, un tipo de red profunda. La arquitectura básica de una CNN se organiza en varias etapas (15):

- Capas de convolución (16)

Cada capa aplica un conjunto de filtros pequeños denominados **kernels** – por ejemplo, de 3x3 o 5x5 píxeles – que se desplazan dependiendo de un número de píxeles en cada paso (**stride**) sobre la imagen de entrada de tamaño $H \times W$ (alto x ancho). Cada filtro realiza una operación de correlación que multiplica elemento a elemento sus pesos (m, n) por los valores de píxel y suma el resultado, produciendo un **mapa de activación**. Estos mapas resaltan características locales, bordes, texturas y esquinas; y al usar múltiples filtros, la red aprende a reconocer patrones variados en cada región.

La salida (mapa de activación) F se calcula mediante:

$$F(i, j) = \sum_{u=0}^{k-1} \sum_{v=0}^{k-1} K(u, v) \times I(i + u, j + v)$$

Es decir, el filtro K se desliza sobre cada posición (i, j) de la imagen, y multiplica elemento a elemento y los suma.

- Funciones de activación

Tras la convolución se aplica una función no lineal que introduce la capacidad de modelar relaciones complejas y evita que la red se comporte como un simple filtro lineal. La más común es ReLU donde “corta” los valores negativos a cero y deja pasar los positivos. Esto introduce no linealidad y acelera el entrenamiento Ilustración 25.

ReLU Function

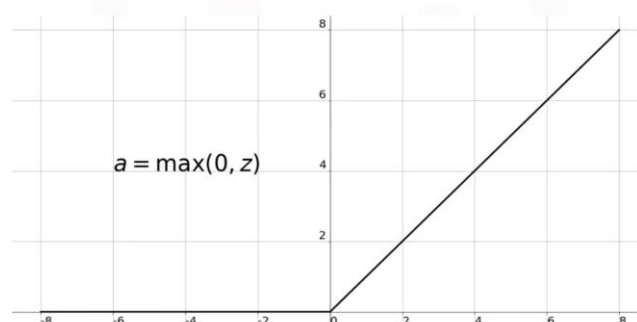


Ilustración 25: Función ReLU

- Capa de pooling

Entre bloques convolucionales se suelen intercalar operaciones de **pooling** (máximo o promedio), que reducen la resolución espacial de los mapas de activación, es decir de cada bloque ($p \times p$) se reemplaza por su valor máximo reduciendo el coste computacional. En la siguiente Ilustración 26 se observa su funcionamiento de forma visual.

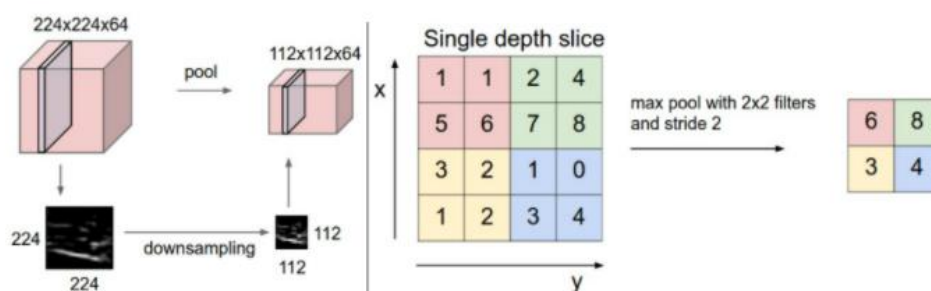


Ilustración 26: Pooling funcionamiento

- Bloques repetitivos y profundización

La combinación de varias capas de convolución, activación y pooling forma un **bloque convolucional**. Es entonces que, al apilar muchos bloques, la red “profundiza”: las primeras capas extraen rasgos muy básicos (líneas, bordes),

mientras que las capas intermedias y finales capturan patrones cada vez más abstractos como formas complejas, texturas, características de frutas o verduras.

- Capas completamente conectadas y salida

Tras la serie de bloques convolucionales y de pooling, los mapas de activación se “aplanan” (**flatten**) y se conectan a una o más capas densas que integran toda la información global.

En detección de objetos, estas capas densas suelen culminar en cabezales especializados que predicen, para cada región, las coordenadas de las cajas (**bounding boxes**) y las probabilidades de pertenencia a cada clase. Las cajas se refieren al recuadro delimitador que el ordenador dibuja para detectar al objeto.

- Entrenamiento por retro propagación

Por último, la red ajusta sus pesos minimizando una función de pérdida (**loss**), que en detección combina errores de la localización (p.ej. IoU) y de clasificación. Además, mediante **backpropagation**, se calculan los gradientes y se actualizan los filtros y pesos de cada capa para que, gradualmente, los mapas de activación respondan cada vez mejor a los patrones reales, en este ejemplo, frutas y verduras tanto en buen estado como en mal estado.

Cabe concluir que, gracias a esta arquitectura jerárquica Ilustración 27, las CNN son capaces de entender representaciones espaciales robustas de los objetos en la imagen, lo que las convierte en la base teórica y práctica de modelos de última generación como YOLOv8.

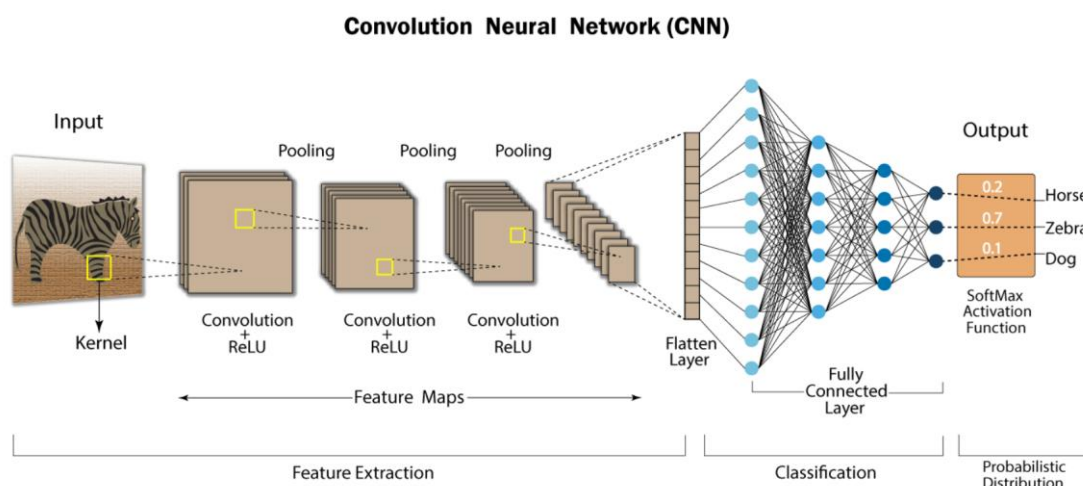


Ilustración 27: Arquitectura CNN

2.2.1 Modelo YOLO (You Only Look Once) (17)

YOLO pertenece a una familia de modelos de detección de objetos que se caracteriza por su capacidad de realizar predicciones en tiempo real con alta precisión. La versión más reciente, YOLOv8, representa la última generación desarrollada por **Ultralytics**, diseñada para ofrecer un equilibrio óptimo entre precisión, velocidad y flexibilidad en tareas de visión artificial. A continuación, se describen sus aspectos más relevantes:

- Evolución del diseño

Desde sus orígenes en el framework Darknet (YOLOv1-v4) hasta la transición a PyTorch con YOLOv5, la familia YOLO siempre ha perseguido la detección “You Only Look Once” – es decir, procesar toda la imagen de una sola pasada – para maximizar la velocidad. YOLOv8 recoge esta herencia e introduce innovaciones clave, pasando de un enfoque basado en “anclas” (anchor-based) a uno **anchor-free**, simplificando la arquitectura del modelo y mejorando su capacidad para predecir ubicaciones de objetos con mayor precisión. Esto es particularmente beneficioso en escenarios donde el conjunto de datos incluye objetos de formas y tamaños variados.

- Funcionamiento

El funcionamiento de YOLOv8 se resume en tres etapas principales:

- **Entrada (Input):** Se introduce una imagen, normalmente redimensionada a un tamaño estándar (640x640 píxeles), para que el modelo pueda procesarla uniformemente

- Extracción de características (**Backbone**): Las imágenes pasan por una serie de capas convolucionales profundas que extraen características jerárquicas: bordes, texturas, formas o combinaciones de estas.

También utiliza un **backbone** moderno llamado **CSPDarknet** o variantes mejoradas de este, que permiten mejorar el flujo de información manteniendo una red liviana y eficiente. Estas arquitecturas están optimizadas para detectar incluso frutas o verduras pequeñas en condiciones variadas de luz o fondo.

- Cabeza del modelo (**Head**) – Predicción sin anclas: Aquí es donde el modelo realiza su tarea principal: detectar objetos.

El modelo no parte de suposiciones sobre el tamaño o posición de los objetos. En su lugar, predice directamente: posición central del objeto, tamaño del cuadro delimitador, clase del objeto y confianza de que hay un objeto allí.

Finalmente, el modelo termina devolviendo una lista de detecciones con sus coordenadas del cuadro delimitador, la clase del objeto detectado y el porcentaje de confianza (probabilidad de que ese objeto pertenezca a una clase específica). En la siguiente Ilustración 28 se puede observar su arquitectura.

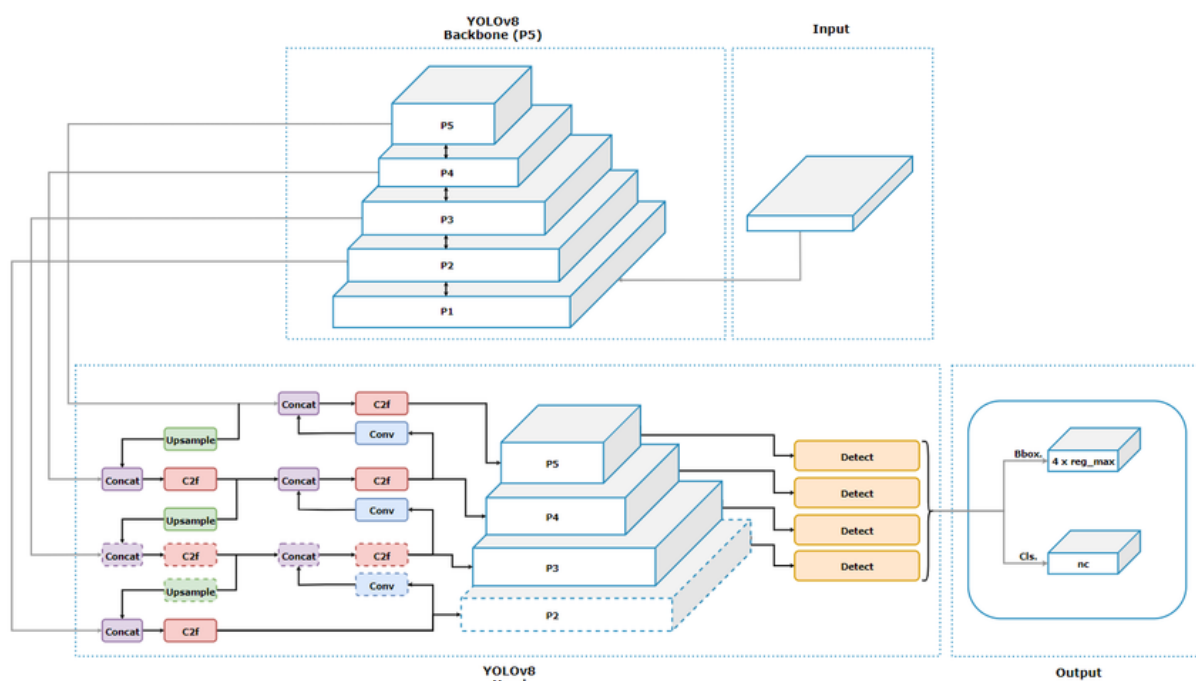


Ilustración 28: Arquitectura YOLOv8

- Multitarea unificada (18)

A diferencia de versiones anterior, YOLOv8 no solo cubre la detección de objetos, sino también segmentación de instancias, clasificación de imágenes y detección de puntos clave, todo a partir de la misma arquitectura base.

- Interoperabilidad

YOLOv8 se exporta fácilmente a ONNX, un estándar abierto que habilita inferencia en múltiples plataformas (CPU, GPU, NNAPI, TensorRT) y facilita la **cuantización post-entrenamiento** (INT8) para reducir tamaño y latencia, estas técnicas de cuantización se realizan convirtiendo los números del modelo a una versión más pequeña (de 32 bits a 8 bits) permitiendo un mejor rendimiento en formatos como teléfonos móviles.

2.3 Desarrollo de Aplicaciones Móviles Android

2.3.1 Kotlin como lenguaje nativo (19)

Kotlin es el lenguaje oficial de Android al ser un lenguaje moderno que permite escribir código más conciso y con menos boilerplate que Java, es decir con menos líneas de código, por ejemplo, una simple clase de datos en Java puede ocupar de 20 a 30 líneas; sin embargo, en Kotlin, con `data class User(val nombre: String, val edad: Int)` se resumen en una línea. Su sistema de tipos integrado con manejo de nulabilidad reduce drásticamente los `NullPointerException`, de modo que las apps en Kotlin tiene hasta un 20% menos de fallos crítico por esta causa. Además, Kotlin se integra de forma nativa con Android Studio: la IDE ofrece soporte de primera clase ofrece soporte de primera clase (autocompletado, refactorizaciones, herramientas de conversión desde Java, etc.). Es decir, un proyecto Kotlin arranca rápidamente en Android Studio sin configuraciones extras y dispone de un SDK optimizado para Kotlin. Kotlin además soporta tanto programación orientada a objetos como funcionalidades funcionales (lambdas, coroutines), lo que facilita operaciones asíncronas y programación reactiva de forma nativa. En conjunto, estas características hacen de Kotlin un lenguaje robusto, productivo y preparado para el desarrollo Android moderno. Por otro lado, sería interesante realizar una comparativa entre Java y Kotlin ver Tabla 28:

Característica	Java	Kotlin
Sintaxis	Verbosa, más boilerplate	Concisa, menos código repetitivo
Null Safety	Nulo por defecto (Nullpointer)	Sistema de tipos que previene nulls
Corutinas / Hilos	Thread, AsyncTask, callbacks	Corutinas nativas para asincronía sencilla
Data Classes	Manual (get/set, equals, etc.)	data class genera todo automáticamente
Extensiones	No disponibles	Funciones y propiedad de extensión

Tabla 26 : Java o Kotlin

2.3.2 Android Studio como IDE (20)

Basado en IntelliJ IDEA, Android studio cuenta con un editor de código inteligente, emulador integrado y herramientas avanzadas (memoria, detectores de rendimiento, etc.) que aumenta la productividad. Su sistema de compilación **Gradle** es flexible para gestionar múltiples módulos y sus variantes de build.

Además, permite la recarga de cambios instantánea (APK changes) sin reiniciar la app, simulación de múltiples dispositivos simultáneamente, y diseño visual de **layouts** (editor de gráfico de vistas). Como IDE oficial de Android, está optimizado para todas las bibliotecas y plugins de Google (como Navigation Component, Hilt, Jetpack Compose, etc.), lo que asegura compatibilidad y soporte continuo. Otro aspecto a mencionar es su capacidad de adaptación en distintas pantallas en el editor de diseños mediante el uso de **ConstraintLayouts** y de las unidades de medidas **sp** (scale-independent pixels) o **dp** (density-independent pixels)

Estructura aplicada en Android Studio para proyectos (21)

Al iniciar un nuevo proyecto en Android Studio, el IDE genera automáticamente una estructura de carpetas organizada, como la que se describe a continuación, para facilitar el desarrollo de la aplicación. Ver Ilustración 29: Estructura AndroidStudio

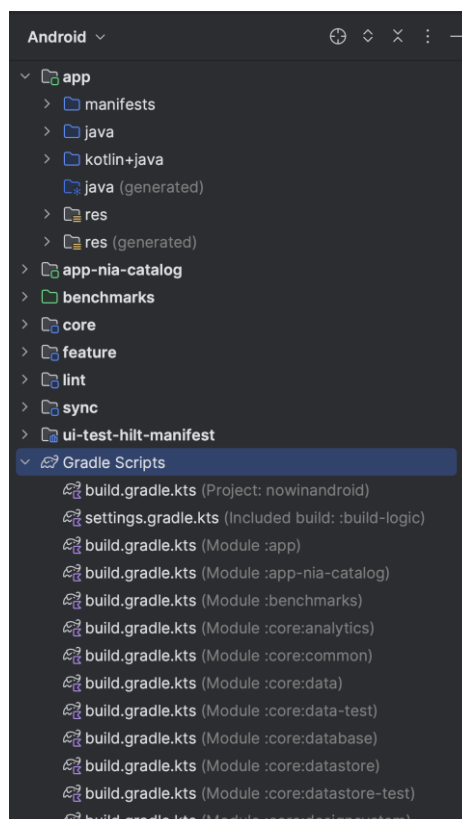


Ilustración 29: Estructura AndroidStudio

La organización de los archivos dentro de un proyecto de Android Studio, comenzando por la carpeta principal app, sigue una estructura bien definida. Dentro de esta carpeta, encontramos los siguientes directorios esenciales:

- **Manifests:** Este directorio alberga el archivo AndroidManifest.xml, que actúa como el **descriptivo fundamental** de la aplicación. Define los componentes de la app (actividades, servicios, receptores de intents, etc.) y sus características esenciales.
- **java:** Aquí reside el **código fuente** de la aplicación. Es importante notar que, aunque se utilice Kotlin como lenguaje de programación, el directorio se denomina java por convención predeterminada al crear el proyecto.
- **assets:** Este directorio está destinado a contener **recursos estáticos** que la aplicación utiliza tal cual, como modelos de detección de objetos adaptados para Android.
- **res (Resources):** Este es el directorio clave para todos los **recursos de diseño** de la aplicación, subdividido en:

- **drawable:** Contiene **archivos de imagen**, iconos y gráficos vectoriales personalizados utilizados en la interfaz de usuario.
- **font:** Almacena los **archivos de fuentes** (tipografías) para la visualización de texto.
- **layout:** Alberga los archivos **XML** que definen la **estructura y el diseño de las interfaces de usuario** (las pantallas de la aplicación), correspondiendo a las clases de la carpeta java.
- **raw:** Se utiliza para almacenar **archivos multimedia** en su formato original, como archivos de audio MP3.
- **values:** Contiene archivos **XML** que definen **valores estáticos** utilizados para dar estilo a la aplicación, como colors.xml (definición de colores), styles.xml (definición de estilos), y archivos de temas (themes.xml).

Finalmente, la carpeta **Gradle Scripts** contiene los archivos de configuración de la herramienta **Gradle**. Gradle es el sistema de construcción que **compila la aplicación**, tomando el código fuente (Kotlin o Java) y los archivos de diseño XML para convertirlos en archivos **DEX** (Dalvik Executable). Estos archivos DEX se empaquetan junto con otros recursos en un archivo comprimido con extensión **.APK** (Android Application Package). Este archivo APK es el paquete final necesario para instalar y ejecutar la aplicación en un dispositivo Android, conteniendo todos los elementos esenciales para su funcionamiento.

Principales componentes de una aplicación Android

Activity: Representa una única tarea enfocada que el usuario puede realizar. Es como una pantalla en una aplicación. Tienen un ciclo de vida que el desarrollador gestiona para asegurar que la aplicación se comporte correctamente. Además, se utiliza de contenedor para los Fragment para gestionar la interfaz de usuario.

Fragment: Porción de la interfaz de usuario de una Activity. Múltiples Fragment pueden existir en una sola Activity. Los Fragments promueven la modularidad y la reutilización. Tiene su propio ciclo de vida, influenciado por el ciclo de vida de la Activity anfitriona. Ver Ilustración 30 para comprender las etapas del ciclo de vida

Model: Representa la capa de datos de la aplicación donde se define la estructura y la lógica de los datos de la aplicación. Pueden ser clases que representas objetos de datos.

View: Responsable de renderizar la UI. Las vistas son componentes de la UI, como TextView, Button, ImageView, etc., y los layouts (archivos XML) que los organizan. Las vistas muestran los datos al usuario y permiten la interacción del usuario.

ViewModel: Un ViewModel almacena y gestiona los datos relacionados con la interfaz de un usuario de manera consciente del ciclo de vida. Esto significa que los datos sobreviven a los cambios de configuración, como las rotaciones de pantalla. Actúa como un puente entre la UI y la capa de datos, a menudo obteniendo datos de repositorios y exponiéndolos a la interfaz de usuario. Además, ayudan a mantener las Activities y los Fragments limpios al evitar que manejen directamente el acceso a los datos.

Intent: Objeto que se utiliza para comunicar entre componentes de Android, es decir, se utiliza principalmente para iniciar Activities o entregar datos entre componentes.

Repository: Clase que encapsula la lógica para acceder a los datos, actúa como un intermediario entre las diferentes fuentes de datos de un servicio web o de una base de datos local.

Adapter: Componente crucial que conecta la fuente de datos (p.ej. una lista de objetos) al RecyclerView (vista flexible para mostrar un gran conjunto de datos).

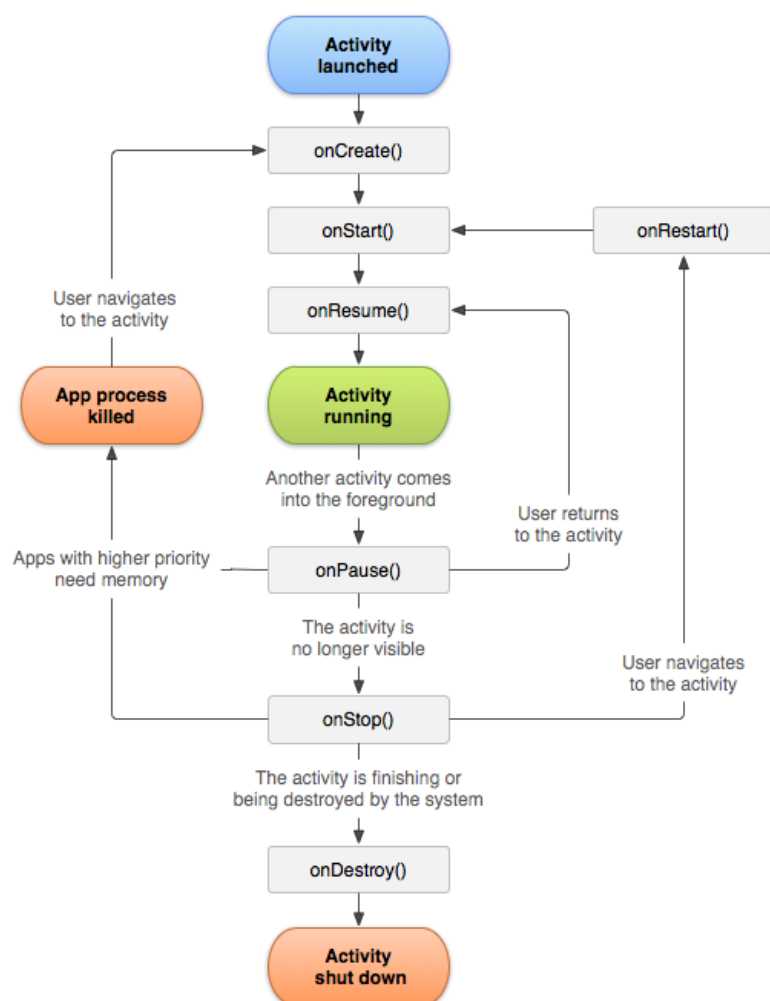


Ilustración 30: Diagrama ciclo de vida Activity

2.4. Servicios Backend-as-a-Service (BaaS): Firebase (22)

En el desarrollo de aplicaciones móviles modernas, los servicios **Backend-as-a-Service (BaaS)** ofrecen una solución integral que permite a los desarrolladores centrarse en la lógica del frontend sin necesidad de construir una infraestructura de backend desde cero. Firebase, la plataforma BaaS de Google, proporciona una amplia gama de servicios listos para usar, integrables de forma sencilla en aplicaciones Android.

En lugar de construir un backend desde cero con lenguajes como PHP, Python o Node.js, el desarrollador puede conectar directamente su aplicación al conjunto de

servicios proporcionado por plataformas como Firebase que proporciona las siguientes ventajas ver Tabla 27

Ventaja	Descripción
Desarrollo más rápido	Permite lanzar apps en menos tiempo al eliminar la necesidad de construir servidores personalizados.
Escalabilidad automática	Los servicios BaaS escalan automáticamente según la demanda de la app.
Seguridad integrada	Autenticación, autorización y protección de datos gestionados por el proveedor.
Base de datos en tiempo real (NoSQL)	Ideal para apps que requieren sincronización instantánea entre usuarios y dispositivos
Gestión de usuarios simplificada	Registro, inicio de sesión y recuperación de contraseñas fácilmente implementables.

Tabla 27: Ventajas Firebase

Firestore (23)

Proporciona un sistema de autenticación fácil de usar y seguro para varios métodos como correo electrónico/contraseña, número de teléfono e inicio de sesión social (Google, Facebook, Apple, etc.). Además de funciones como verificación de correo electrónico, restablecer contraseña, políticas de contraseñas, etc.

Cloud Firestore

Organiza los datos de en forma de colecciones de documentos, donde cada documento contiene información estructurada en pares clave-valor, permitiendo incluso subcolecciones para una mejor organización. Ofrece capacidades de consulta mucho más potentes y flexibles, permitiéndote filtrar, ordenar y combinar datos de manera eficiente. Diseñado para escalar automáticamente, Firestore maneja grandes volúmenes de datos y tráfico sin problemas. Si bien también ofrece sincronización en tiempo real, su principal fortaleza radica en su escalabilidad y la riqueza de sus consultas. Las reglas de seguridad son más expresivas y se integran con la autenticación de Firebase para un control de acceso más granular.

Firestore Realtime Database

Se trata de una estructura de árbol muy grande donde toda la información de la aplicación reside jerárquicamente. La herramienta Realtime Database se basa en este modelo JSON, ofreciendo una sincronización de datos increíblemente rápida en tiempo

real, ideal para experiencias colaborativas instantáneas. Sin embargo, las consultas son bastantes limitadas, dificultando la búsqueda compleja de información. Aunque para muchos escenarios escala bien, puede que manejar modelos de datos muy grandes y anidados puede volverse un desafío. Otro aspecto es su robusto soporte offline en dispositivos móviles asegurando que los usuarios puedan interactuar incluso sin conexión. La seguridad se define mediante reglas que controlan el acceso a las ramas de este árbol de datos.

¿Realtime Database o Cloud Firestore?

Para seleccionar entre ambos he optado por realizar un breve análisis entre ambos según la documentación de Firebase (24).

Las características de ambas se detallan en Tabla 28.

Característica	Firebase Realtime Database	Cloud Firestore
Modelo de Datos	Árbol JSON único	Colecciones de documentos con datos clave-valor
Consultas	Limitadas; principalmente por clave o un solo campo ordenado	Potentes y flexibles; múltiples cláusulas <code>where()</code> , ordenación
Escalabilidad	Puede requerir denormalización para grandes conjuntos de datos	Escalado automático y eficiente para grandes conjuntos de datos
Tiempo Real	Sincronización en tiempo real de baja latencia	Sincronización en tiempo real con buena latencia
Soporte Offline	Robusto para móvil (Android e iOS)	Soporte offline para web, iOS y Android
Seguridad	Reglas basadas en rutas en una estructura JSON	Reglas más expresivas integradas con Firebase Authentication
Precios	Principalmente ancho de banda y almacenamiento	Principalmente operaciones de lectura/escritura/eliminación
Casos de Uso Típicos	Chat en tiempo real, presencia, juegos multijugador sencillos	Aplicaciones sociales complejas, comercio electrónico, CMS

Tabla 28: Comparación Realtime Database y Cloud Firestore

De este modo, al comparar ambas herramientas, la opción por la que me he decantado es Cloud Firestore, dado que es más robusta y flexible que Realtime Database debido

a sus potentes consultas para filtrar recetas por ingredientes o gestionar el inventario, además de su escalabilidad para el crecimiento futuro. Aunque Realtime Database es rápida, las consultas complejas serían más difíciles de implementar y mantener.

Cloud Messaging (25)

La función principal de Firebase Cloud Messaging es proporcionar una manera confiable y eficiente de enviar mensajes entre el servidor (o la consola de Firebase) y las aplicaciones de los usuarios. Esto permite:

- **Enviar notificaciones:** Informar a los usuarios sobre eventos importantes, como nuevos mensajes de chat, promociones, notificaciones, actualizaciones de la aplicación, etc.
- **Entregar datos:** Enviar información a la aplicación para que la procese en segundo plano, como actualizar la configuración, sincronizar datos o activar descargas.

Crashlytics (26)

Crashlytics, es la herramienta de Firebase para el análisis de fallos, proporciona información en tiempo real sobre la estabilidad de las aplicaciones al recopilar automáticamente informes detallados que ayudan a los desarrolladores a identificar las causas de los problemas y agruparlos para priorizar las soluciones; además, ofrece registros y alertas personalizables para dar contexto y notificaciones rápidas, lo que finalmente contribuye a mejorar la experiencia del usuario y la calidad general de la aplicación.

Firebase Performance (27)

Firebase Performance Monitoring, es un servicio para obtener estadísticas sobre las implementaciones desarrolladas en la aplicación. Ayuda a comprender en tiempo real donde se puede mejorar el rendimiento de la app, de manera que sea sencillo a través de esa información solucionar problemas de rendimiento.

2.5 API Externa para Enriquecimiento de funcionalidades

Spoonacular es una API RESTful que proporciona mucho más de 365.000 recetas disponibles, con detalles nutricionales e información acerca de cada ingrediente utilizado en la receta, además de productos, menús y más. Se enfoca en aplicaciones web y móviles, posibilitando la consulta de datos a través de solicitudes HTTP (GET y POST) que proporcionan respuestas en formato JSON.

Entre sus características más relevantes sobresale:

- **Búsqueda de recetas:** posibilidad de filtrar recetas por ingredientes disponibles, tipo de dieta como vegetariana, vegana, sin gluten, etc., tipo de comida (p.ej. desayuno, almuerzo, cena, postre, etc.), distintas intolerancias, tiempo de preparación, nivel de dificultad o incluso por palabras clave.
- **Información nutricional:** proporciona datos detallados sobre las calorías, proteínas, carbohidratos, grasas y otros nutrientes esenciales de cada receta o ingrediente.
- **Análisis de ingredientes:** evalúa la calidad nutricional de cada ingrediente y detecta los alérgenos comunes.
- **Instrucciones paso a paso:** se incluye los pasos detallados para la elaboración de una receta, junto con imágenes e instrucciones.
- **Consulta por ID:** permite obtener la información completa de una receta con solo conocer su ID.

3. Diseño y Desarrollo del Modelo de Detección de Frutas y Verduras

En este apartado detallaré el proceso de diseño y desarrollo que se ha seguido para obtener el modelo entrenado, así como mencionar las distintas herramientas utilizadas y decisiones planteadas.

3.1 Estructura y herramientas

3.1.1 Dataset utilizado

Antes de comenzar con el proyecto, tuve que decidir qué base de datos debía utilizar y donde podía obtenerla. Para ello, navegue hacia la página web [Kaggle](https://www.kaggle.com/) donde se recoge muchos datos científicos y profesionales para Machine Learning ya que existe

una gran comunidad que resuelve distintos problemas orientados al aprendizaje automático y los comparte para aprender de otros.

La elección fue clara y concisa debido a que necesitaba un dataset donde se pudiera albergar distintas imágenes de frutas y verduras tanto en buen estado como en mal estado. Mi decisión fue “Fruit and Vegetable Disease (Healthy vs Rotten) (28). Este dataset consiste en 28 directorios, cada uno representado una combinación de en buen estado y en mal estado de imágenes de 14 tipos de frutas o verduras. Cabe resaltar el aspecto de su tamaño de 5.21GB con el que nos facilitará a la hora de que el modelo logre aprender patrones más variados y evitar el **overfitting** (cuando el modelo no rinde bien en la clasificación de escenarios nuevos). Cabe mencionar que este dataset está preparado con imágenes con técnicas de **data augmentation** que aumenta la robustez y generalización a la hora del entrenamiento de modelos. Esta técnica básicamente se refiere a las transformaciones de imágenes, por ejemplo, rotaciones, cambio de escala, técnicas de sal y pimienta, iluminación, etc.

Este dataset está organizado de la siguiente forma en la Ilustración 31.



Ilustración 31: Estructura Dataset

3.1.2 Preparación de la estructura del dataset

Antes de trabajar con el dataset para el entrenamiento del modelo, se debe organizar el conjunto de datos en una estructura por carpetas correcta para asegurar que el modelo pueda funcionar con datos nuevos (datos que no se han utilizado tras el entrenamiento) ya que si no creamos un conjunto de validación el modelo solo podrá memorizar patrones sin aprender a generalizar. Por tanto, se debe tener un directorio `train/` y `val/` y dentro de cada directorio un subdirectorio de cada clase con la que se quisiera trabajar, en mi caso se corresponde a los 28 directorios de cada fruta y verdura (p. ej, `Apple_Healthy`, `Apple_Rotten`, etc). Esta organización facilita la gestión de datos durante el entrenamiento y la validación del modelo. Por otro lado, es crucial la creación de un archivo de configuración `dataset.yaml` ver en Ilustración 32 que cumple la función de proporcionar información adicional al modelo YOLOv8 para que este pueda entender la estructura y el contenido de datos. Está dividido en los distintos nombres de clases a utilizar, el **path** (ruta base donde se encuentra el dataset), **nc** (número total de clases de objetos que el modelo debe aprender) y por ultimas las rutas específicas de las carpetas de **train** y de **val**.

```
names:
 0: Apple_Healthy
 1: Apple_Rotten
 2: Banana_Healthy
 3: Banana_Rotten
 4: Bellpepper_Healthy
 5: Bellpepper_Rotten
 6: Carrot_Healthy
 7: Carrot_Rotten
 8: Cucumber_Healthy
 9: Cucumber_Rotten
10: Grape_Healthy
11: Grape_Rotten
12: Guava_Healthy
13: Guava_Rotten
14: Jujube_Healthy
15: Jujube_Rotten
16: Mango_Healthy
17: Mango_Rotten
18: Orange_Healthy
19: Orange_Rotten
20: Pomegranate_Healthy
21: Pomegranate_Rotten
22: Potato_Healthy
23: Potato_Rotten
24: Strawberry_Healthy
25: Strawberry_Rotten
26: Tomato_Healthy
27: Tomato_Rotten
nc: 28
path: X:/Fruit And Vegetable Diseases Dataset/images
train: X:/Fruit And Vegetable Diseases Dataset/images/train/
val: X:/Fruit And Vegetable Diseases Dataset/images/val/
```

Ilustración 32: `dataset.yaml`

Para la preparación de la estructura, he preparado un script en Python `preparation_structure.py` utilizando las librerías **os** y **sutil** que nos ofrecerá las

distintas funcionalidades para manipular archivos e interactuar con el sistema operativo. El código es muy sencillo con el que crearemos los directorios, recorreremos todas las imágenes del dataset de forma aleatoria mediante una semilla con la función **random()**, dividiendo en un 80% de entrenamiento y un 20% de validación y creando las subcarpetas según el nombre de clase del objeto y así moviendo cada imagen a su correspondiente subcarpeta en Ilustración 33. Por último, se creará el archivo de configuración para YOLOv8 en la ruta base donde se encuentra el dataset ver Ilustración 34.

```
# Crear las carpetas de entrenamiento y validación si no existen
os.makedirs(train_dir, exist_ok=True)
os.makedirs(val_dir, exist_ok=True)

# Establecer una semilla aleatoria para reproducibilidad
random.seed(42)

# Recorrer todas las imágenes en 'images' y sus subdirectorios (sin incluir la carpeta 'images')
all_images = []
for root, dirs, files in os.walk(images_base_path):
    # Solo procesamos los archivos que están dentro de las subcarpetas de imágenes
    if root != images_base_path: # Evitar la carpeta raíz 'images'
        for file in files:
            if file.endswith(('.jpg', '.jpeg', '.png')): # Verificar extensiones de imagen
                all_images.append(os.path.join(root, file)) # Agregar la ruta completa de la imagen

# Dividir el conjunto de imágenes en entrenamiento (80%) y validación (20%)
train_images = random.sample(all_images, int(0.8 * len(all_images)))
val_images = [img for img in all_images if img not in train_images]

# Función para mover las imágenes a las carpetas correspondientes
def move_images(images, dest_dir):
    new = 0
    for img in images:
        # Crear subcarpetas en el directorio destino para imitar la estructura original
        relative_path = os.path.relpath(img, images_base_path) # Obtener la ruta relativa de la imagen
        dest_path = os.path.join(dest_dir, relative_path) # Ruta destino en la carpeta de entrenamiento o validación

        os.makedirs(os.path.dirname(dest_path), exist_ok=True) # Crear los directorios necesarios
        shutil.move(img, dest_path) # Mover la imagen a la carpeta correspondiente

# Mover las imágenes de entrenamiento y validación
move_images(train_images, train_dir)
move_images(val_images, val_dir)
```

Ilustración 33: Creación train/ y val/

```
# Crear archivo de configuración para YOLO
config_path = os.path.join(dataset_path, 'dataset.yaml')
with open(config_path, 'w') as f:
    f.write(f"path: {dataset_path}\n") # Ruta principal del dataset
    f.write(f"train: images/train/\n") # Carpeta de imágenes de entrenamiento
    f.write(f"val: images/val/\n") # Carpeta de imágenes de validación
    f.write("\n")
    f.write("names:\n")
    for i, name in enumerate([
        "Apple_Healthy", "Apple_Rotten", "Banana_Healthy", "Banana_Rotten",
        "Bellpepper_Healthy", "Bellpepper_Rotten", "Carrot_Healthy", "Carrot_Rotten",
        "Cucumber_Healthy", "Cucumber_Rotten", "Grape_Healthy", "Grape_Rotten",
        "Guava_Healthy", "Guava_Rotten", "Jujube_Healthy", "Jujube_Rotten",
        "Mango_Healthy", "Mango_Rotten", "Orange_Healthy", "Orange_Rotten",
        "Pomegranate_Healthy", "Pomegranate_Rotten", "Potato_Healthy", "Potato_Rotten",
        "Strawberry_Healthy", "Strawberry_Rotten", "Tomato_Healthy", "Tomato_Rotten"
    ]):
        f.write(f" {i}: {name}\n") # Escribir las clases en el archivo .yaml
```

Ilustración 34: Archivo configuración YOLOv8

Además, es de gran importancia anotar cada imagen con un cuadro delimitador, **bounding box**, para que indique donde se ubica exactamente el objeto a detectar dentro de la imagen. Las anotaciones se guardan en archivos de texto formato `.txt`, pero para ello antes se debe realizar un determinado proceso. Para el proceso de anotación de cada imagen se debe crear un nuevo directorio `labels/` donde se almacenará cada anotación de cada imagen realizado.

Sin embargo, la tarea de crear una anotación para cada objeto puede resultar muy tediosa y pesada debido a que se debe crear un archivo XML e introducir objeto por objeto su descripción: carpeta donde se encuentra, nombre del archivo, sus posiciones `x` y `y`, su alto y su ancho. En la siguiente imagen Ilustración 35 se puede ver el proceso manual de crear cada anotación sacada del blog de Juan Ignacio Bagnato (29).

```
<annotation>
  <folder>lego</folder>
  <filename>cropped12_231359.jpg</filename>
  <path>/detectaobjetos/images/lego3/cropped12_231359.jpg</path>
  <size>
    <width>4512</width>
    <height>4512</height>
    <depth>3</depth>
  </size>
  <object>
    <name>lego</name>
    <bndbox>
      <xmin>909</xmin>
      <ymin>879</ymin>
      <xmax>3200</xmax>
      <ymax>4512</ymax>
    </bndbox>
  </object>
</annotation>
```

Ilustración 35: Ejemplo de bounding box XML

Cierto es, que en nuestro caso es inviable en cuanto al tiempo de elaboración, ya que nuestros dataset alberga una cantidad de casi 30.000 imágenes.

Existen numerosas herramientas automáticas que facilitan el proceso de creación de bounding boxes como **Roboflow** o **MakeSense.ai**, aunque sean muy potentes, su gran desventaja es que se encuentran muy limitadas ya que resultan ser software pago. Por otro lado, también se encuentra. También se encuentra la herramienta manual **VGG Image Annotator (VIA)** con la que podremos crear nuestras propias anotaciones con total precisión y libertad, además su gran ventaja es que es un software gratuito para cualquier usuario. Sin embargo, otra vez se encuentra el problema de que abarcaría demasiado tiempo hacerlo imagen por imagen. Información contrastada desde la siguiente página web de Toolify (30)

La solución propuesta para este proyecto nace con la idea de lo aprendido y aplicado en la asignatura de tercer año **Tratamiento Digital de Imágenes** con la que nos beneficiaremos con la librería **OpenCV**. Para ello aplicaremos las diversas técnicas de segmentación aprendidas en el curso.

Para entender un poco la solución propuesta, explicaré las técnicas realizadas a la par que el funcionamiento del programa `segment_generator.py`

Las librerías utilizadas son **cv2** (OpenCV) para el procesamiento de imágenes y **numpy** para el procesamiento de operaciones numéricas, especialmente arrays.

Como primer paso se debe definir la estructura de **COCO** (Common Objects in Context) que utiliza el formato **JSON**. Básicamente, se trata un formato que está

diseñado para las tareas de segmentación de instancias, la detección de objetos y el etiquetado de imágenes. COCO proporciona una estructura limpia y legible para las anotaciones.

Para ello se definen las categorías con el nombre de las clases de los objetos y se crea un diccionario que creará un mapa clave con un ID único y el valor que será el nombre de cada categoría. En la estructura de COCO estableceremos una lista que contendrá los datos asociados a cada imagen, sus anotaciones y categoría. Es entonces que con la función `process_images_from_dir()` recorreremos cada subdirectorio (categoría) y asimismo, cada imagen dentro de cada categoría.

```
#Definir categorías
categories = [
    "Apple__Healthy", "Apple__Rotten", "Banana__Healthy", "Banana__Rotten",
    "Bellpepper__Healthy", "Bellpepper__Rotten", "Carrot__Healthy", "Carrot__Rotten",
    "Cucumber__Healthy", "Cucumber__Rotten", "Grape__Healthy", "Grape__Rotten",
    "Guava__Healthy", "Guava__Rotten", "Jujube__Healthy", "Jujube__Rotten",
    "Mango__Healthy", "Mango__Rotten", "Orange__Healthy", "Orange__Rotten",
    "Pomegranate__Healthy", "Pomegranate__Rotten", "Potato__Healthy", "Potato__Rotten",
    "Strawberry__Healthy", "Strawberry__Rotten", "Tomato__Healthy", "Tomato__Rotten"
]

# Crear mapeo de categorías
category_id_map = {name: idx + 1 for idx, name in enumerate(categories)}

# Estructura COCO
coco_data = {
    "images": [],
    "annotations": [],
    "categories": [{"id": i + 1, "name": name} for i, name in enumerate(categories)]
}

annotation_id = 1
img_idx = 1 # Iniciar con id de imagen como 1

# Función para procesar imágenes manteniendo la estructura de directorios
def process_images_from_dir(subdir, subset_name):
    new = {}
    global img_idx, annotation_id

    # Recorreremos las categorías dentro de la carpeta 'train' o 'val'
    for category in os.listdir(subdir):
        category_path = os.path.join(subdir, category)
        if not os.path.isdir(category_path):
            continue

        for img_name in os.listdir(category_path):
            img_path = os.path.join(category_path, img_name)
            img = cv2.imread(img_path)

            if img is None:
                continue
```

Ilustración 36: Estructura COCO segment_generator.py

Dentro del bucle, se realizará los pasos de segmentación, con los resultados ordenados de izquierda a derecha de cada proceso en la Ilustración 37 :

1. Se convierte la imagen a escalas de grises para reducir la complejidad de la imagen `cv2.cvtColor`

2. Se aplica un desenfoque gaussiano mediante el cálculo promedio de los valores de los píxeles mediante un kernel de tamaño 5x5, para suavizar la imagen y disminuir el ruido, con el efecto de evitar variaciones en la segmentación `cv2.GaussianBlur`
3. Se aplica umbralización adaptativa de Otsu convirtiendo las imágenes de una escala de grises a una imagen binaria. Esto se consigue estableciendo un umbral y convirtiendo los píxeles con mayor intensidad que el umbral en blanco y los demás en negro. Esto proporciona una imagen dividida en dos regiones `cv2.threshold`
4. Se realiza una operación de apertura morfológica para eliminar pequeños ruidos y suavizar los contornos en la imagen binaria `cv2.morphologyEx`
5. Finalmente, se detectan los contornos de los objetos segmentados `cv2.findContours`

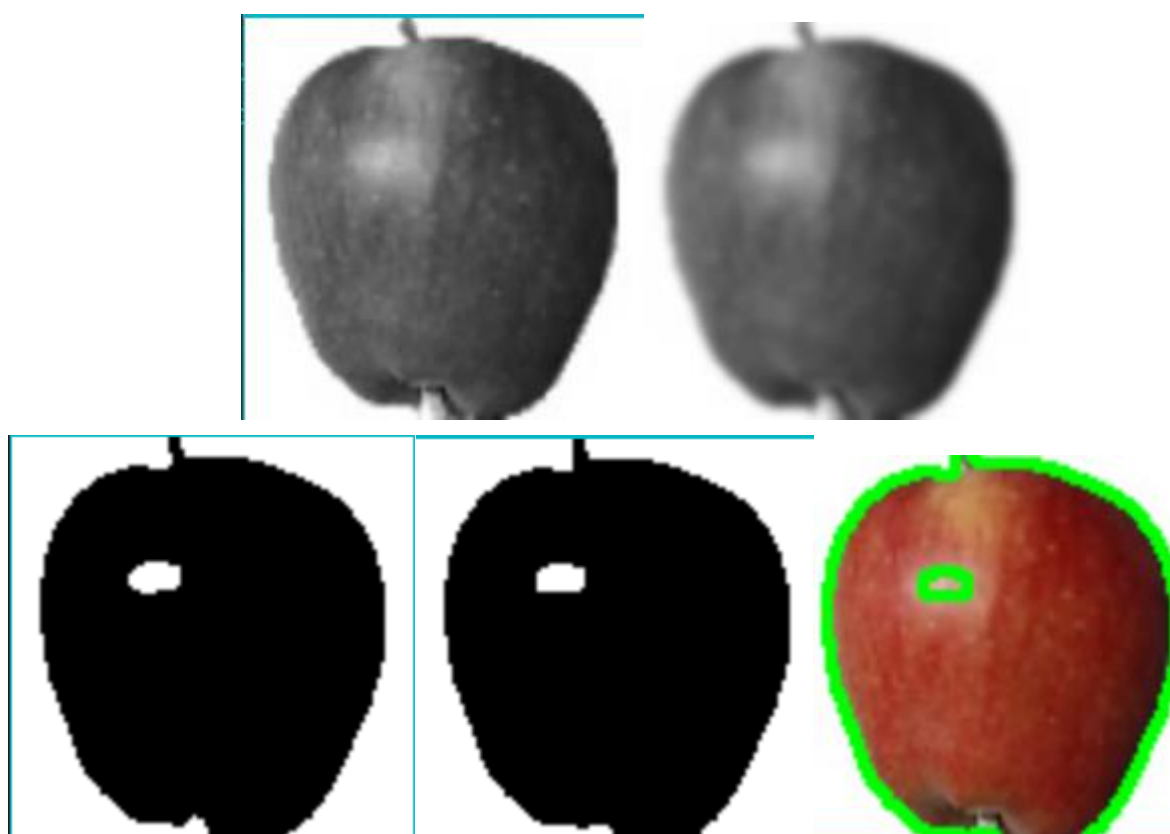


Ilustración 37: Proceso de segmentación

```
# Convertir a escala de grises y aplicar desenfoque
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
blurred = cv2.GaussianBlur(gray, ksize: (5, 5), sigmaX: 0)

# Aplicar segmentación adaptativa
_, thresh = cv2.threshold(blurred, thresh: 0, maxval: 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)

# Eliminar pequeños ruidos
kernel = np.ones(shape: (5, 5), np.uint8)
thresh = cv2.morphologyEx(thresh, cv2.MORPH_OPEN, kernel)

# Encontrar contornos
contours, _ = cv2.findContours(thresh, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
```

Ilustración 38: Funciones de segmentación OpenCV

Finalmente, se obtiene las rutas relativas de las imágenes procesadas y se guarda la imagen procesada con sus bounding boxes dibujados usando `cv2.imwrite()`, mientras que se agrega la información del ID de la imagen, la ruta relativa, la altura y la anchura. Además, se crean las anotaciones con el ID de la anotación, el ID de la imagen correspondiente, el ID de la categoría del objeto, el bounding box, la información de la segmentación, el área del contorno y, por último, si el objeto es parte de una multitud de objetos ver Ilustración 40. Por último, incrementamos el contador para continuar recorriendo imágenes y guardamos las anotaciones en formato COCO en un archivo JSON ver Ilustración 41. También se guarda cada imagen procesada para observar los resultados ver.

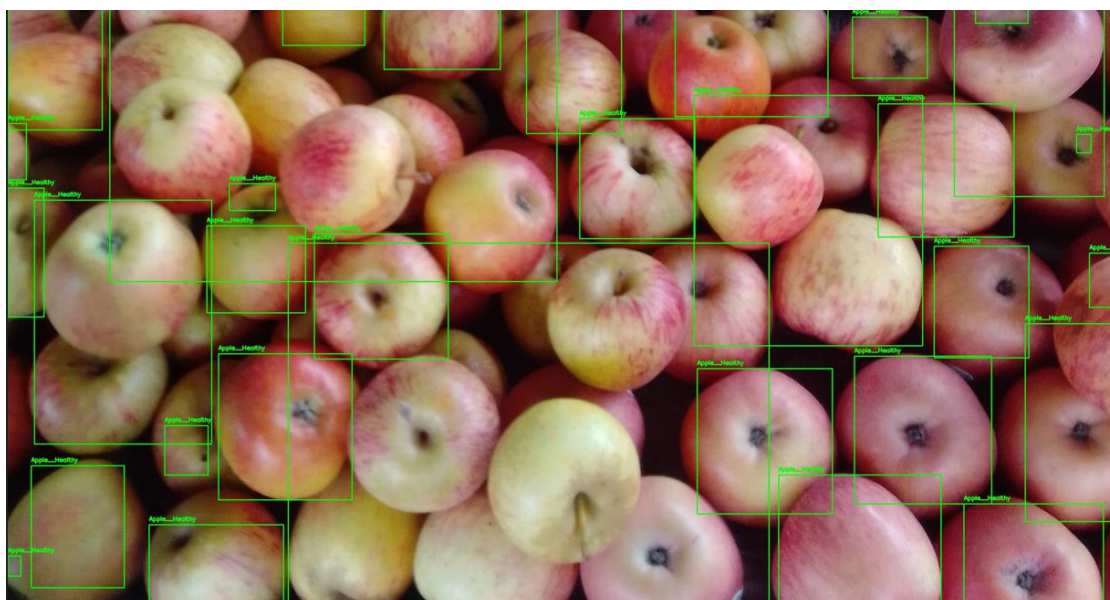


Ilustración 39: Imagen resultado de la segmentación


```

# Agregar imagen a la estructura COCO con ruta relativa
relative_img_path = os.path.relpath(processed_img_path, output_dir)
coco_data["images"].append({
    "id": img_idx,
    "file_name": relative_img_path, # Guardar ruta relativa
    "height": img.shape[0],
    "width": img.shape[1]
})

# Guardar anotaciones de cada contorno detectado
for contour in contours:
    if cv2.contourArea(contour) < 1000: # Filtrar ruidos pequeños
        continue

    # Obtener rectángulo delimitador
    x, y, w, h = cv2.boundingRect(contour)
    segmentation = contour.flatten().tolist()

    # Crear anotación para el contorno
    coco_data["annotations"].append({
        "id": annotation_id,
        "image_id": img_idx,
        "category_id": category_id_map[category], # Asignar ID de la categoría
        "bbox": [x, y, w, h],
        "segmentation": [segmentation],
        "area": cv2.contourArea(contour),
        "iscrowd": 0
    })
    annotation_id += 1

```

Ilustración 40: Código sobre anotaciones COCO

```

# Dibujar bounding box en la imagen
cv2.rectangle(img, (x, y), (x + w, y + h), (0, 255, 0), 2)
cv2.putText(img, category, (x, y - 10), cv2.FONT_HERSHEY_SIMPLEX, fontScale=0.5, color=(0, 255, 0), thickness=2)

# Guardar la imagen procesada con anotaciones en su carpeta correspondiente
cv2.imwrite(processed_img_path, img)

# Incrementar el índice de la imagen
img_idx += 1

# Procesar imágenes de 'train' y 'val'
train_dir = os.path.join(dataset_dir, "train")
val_dir = os.path.join(dataset_dir, "val")

process_images_from_dir(train_dir, subset_name="train")
process_images_from_dir(val_dir, subset_name="val")

# Guardar las anotaciones COCO en formato JSON
json_output_path = "X:/Fruit And Vegetable Diseases Datasetcp/annotations/dataset_coco.json"
with open(json_output_path, "w") as f:
    json.dump(coco_data, f, indent=4)

print(f" Anotaciones guardadas en {json_output_path}")

```

Ilustración 41: Código sobre guardar archivo COCO

Terminado esto, todavía no se ha concluido los pasos previos para entrenar el modelo.

Debido a que YOLOv8 solo puede trabajar con formatos de texto planos, es necesario entonces crear un archivo de texto por cada imagen ya que el JSON se trata de un formato demasiado complejo por el que el modelo no puede trabajar eficientemente.

El formato que espera YOLO es una fila por objeto en `class x_center y_center width height`, cada archivo de texto contiene las coordenadas normalizadas de las regiones de interés, así como la clase de cada objeto detectado, debiendo empezar la primera clase con un índice cero. En las siguientes imágenes se observa el formato YOLO. (31)

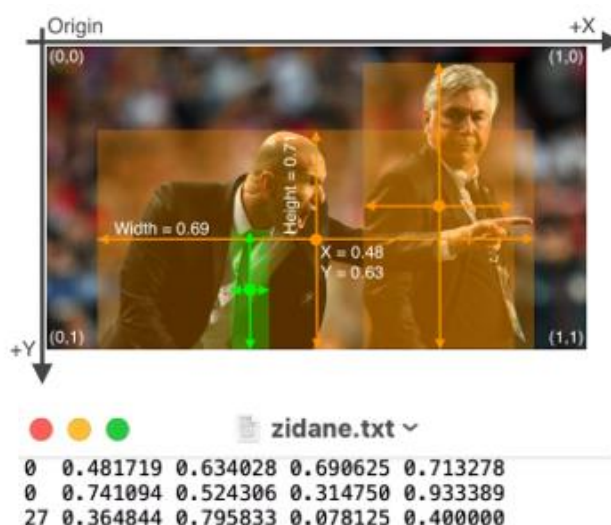


Ilustración 42: Formato YOLO imagen

Aún tras varios intentos, no he podido utilizar la función `convert_coco` de la librería de ultralytics presentado el mismo error que varios usuarios de la comunidad de Ultralytics (32). Por tanto, navegando encontré que una publicación de StackOverFlow que resolvía este problema sin el uso de la librería. (33)

Por ello, he creado un script que convierte el formato COCO a YOLO `coco_to_yolo.py`. Utilizaremos la librería **ijson**, ya que leeremos el contenido del archivo `dataset_coco.json`, esta nos ofrece leer el archivo sin cargarlo entero en memoria. Primero extraemos de cada entrada una categoría para guardar el nombre con el objetivo de saber qué etiqueta textual corresponde a cada ID. Previamente a ello, crearemos las estructuras necesarias para almacenar el nombre de categoría, la ruta de la imagen y ancho y alto de cada imagen ver Ilustración 43. Tras leer cada imagen, procesare cada anotación, empezando por la categoría id 0 y determino a si esta imagen pertenece al directorio de entrenamiento o al de validación. Una vez el

directorio de la categoría este creado y el nombre de cada archivo de texto, obtendré las dimensiones de la imagen desde el diccionario y calculare las coordenadas normalizadas para YOLO ver Ilustración 44. Como resultado, se generará para cada clase un directorio categoría y un archivo de texto plano para cada imagen de la categoría indicando el bounding box listo para entrenar el modelo YOLOv8.

```
# Diccionarios para almacenar información
category_map = {} # COCO ID → Nombre de categoría
image_dict = {} # Image ID → Ruta de la imagen
image_data = {} # Image ID → {width, height}

# Leer el JSON de COCO de manera eficiente
with open(coco_json_path, "r") as f:
    # Extraer categorías
    for category in ijson.items(f, "categories.item"):
        category_map[category["id"]] = category["name"]

    # Volver a abrir el archivo para imágenes (reset)
    f.seek(0)

    # Extraer imágenes y almacenarlas en un diccionario
    for image in ijson.items(f, "images.item"):
        image_dict[image["id"]] = image["file_name"].replace("\\", "/")
        image_data[image["id"]] = {"width": image["width"], "height": image["height"]}

    # Volver a abrir el archivo para anotaciones
    f.seek(0)

    # Procesar anotaciones
    for annotation in ijson.items(f, "annotations.item"):
        image_id = annotation["image_id"]
        category_id = annotation["category_id"] - 1 # YOLO usa índices desde 0
        bbox = annotation["bbox"]

        # Obtener ruta de la imagen correspondiente
        if image_id not in image_dict:
            print(f" Error: No se encontró la imagen con ID {image_id}. Se omite la anotación.")
            continue

        image_path = image_dict[image_id]
```

Ilustración 43: Lectura archivo COCO

```
# Generar nombre del archivo de etiqueta
label_filename = os.path.splitext(os.path.basename(image_path))[0] + ".txt"
label_path = os.path.join(category_label_dir, label_filename)

# Obtener dimensiones de la imagen desde el diccionario
img_width = image_data[image_id]["width"]
img_height = image_data[image_id]["height"]

# Calcular coordenadas normalizadas para YOLO
x_center = (bbox[0] + bbox[2] / 2) / img_width
y_center = (bbox[1] + bbox[3] / 2) / img_height
width = (bbox[2] - bbox[0]) / img_width
height = (bbox[3] - bbox[1]) / img_height

yolo_annotation = f"{category_id} {x_center:.6f} {y_center:.6f} {width:.6f} {height:.6f}\n"

# Guardar anotaciones en el archivo correspondiente
with open(label_path, "a") as label_file:
    label_file.write(yolo_annotation)
```

Ilustración 44: Calcular coordenadas normalizadas

Como resultado de la preparación de la estructura del modelo, se debe tener como resultado el siguiente árbol:

```
Fruit And Vegetable Diseases Dataset /  
├── annotations/  
│   ├── dataset_coco.json  
├── processed_images_segmentadas/  
├── dataset.yaml  
├── images/  
│   ├── train/  
│   │   ├── Apple__Healthy/  
│   │   │   ├── freshApple (1).png  
│   │   └── val/  
└── labels/  
    ├── train/  
    │   ├── Apple__Healthy/  
    │   │   ├── freshApple (1).txt  
    └── val/
```

3.2 Entrenamiento del modelo YOLOv8

Para entrenar, validar, inferir y exportar modelos YOLOv8 es requerido importar la librería oficial de código abierto **ultralytics**. Esta librería nos facilitara mucho las tareas para el entrenamiento. Sin embargo, por defecto esta librería hace uso de la **CPU**, esto es un inconveniente ya que el entrenamiento podría ser muy lento. Es por ello que haré uso de la plataforma de computación paralela de **NVIDIA** para poder acelerar el proceso utilizando la **GPU**. Es obvio que este software es solo compatible con tarjetas gráficas NVIDIA.

CUDA ToolKit se descarga desde la página oficial de NVIDIA Developer (34). Una vez descargado, se debe instalar según las preferencias de instalación deseadas Ilustración 45.



Ilustración 45: Instalador NVIDIA CUDA

Por último, para integrar el soporte CUDA, en nuestro proyecto, ejecutaré este comando en el entorno virtual sobre el que trabajo (35):

```
pip3 install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu126
```

Gracias a que Pytorch tiene un soporte nativo para CUDA y trabaja como framework principal de YOLO, es posible entrenar con esta herramienta los modelos YOLO.

En sistemas Windows cuando se utilizan varias bibliotecas que dependen del OpenMP, como Pytorch y Numpy, esto puede generar conflictos. Para evitar esto, he incluido la línea `os.environ["KMP_DUPLICATE_LIB_OK"] = "TRUE"`. Esta solución la he aplicado gracias a una publicación de StackOverFlow (36).

A continuación, en el script `train_yolo.py` verifico si la GPU está disponible para trabajar y con el parámetro `device` específico el dispositivo para el entrenamiento.

En la documentación de Ultralytics se especifica la descripción de cada parámetro para la función de entrenamiento, estos parámetros son realmente interesantes ya que son cruciales para la optimización del entrenamiento. Como se puede observar en el script Ilustración 46, empiezo cargando el modelo YOLOv8 nano preentrenado. Los parámetros seleccionados para el entrenamiento son los siguientes:

- Data: Ruta del archivo de configuración yaml

- Epochs: Número total de épocas (50) que representa cada iteración por el conjunto de datos, afectando el valor al rendimiento y duración de entrenamiento del modelo.
- Imgsz: Tamaño de imagen 416px, afectando el valor a la precisión del modelo y complejidad computacional.
- Batch: Número de imágenes por lote, un valor pequeño podría notar más detalles de cada imagen.
- Workers: Numero de subprocesos de procesamiento paralelo, 0 para evitar errores en Windows.
- Verbose: En falso para no mostrar mensajes intermedios del proceso.

```
os.environ["KMP_DUPLICATE_LIB_OK"]="TRUE"
if __name__ == '__main__':
    # Verificar si CUDA está disponible
    device = 'cuda' if torch.cuda.is_available() else 'cpu'
    print(f"Entrenando en el dispositivo: {device}")

    # Definir el archivo de configuración del dataset
    yaml_path = "X:/Fruit And Vegetable Diseases Dataset/dataset.yaml"

    # Cargar el modelo YOLOv8 preentrenado
    model = YOLO("yolov8n.pt")

    # Entrenar el modelo
    model.train(data=yaml_path, epochs=50, imgs=416, batch=8, workers=0, verbose=False, device=device)
```

Ilustración 46: Entrenamiento modelo script

Por último, para estudiar los resultados obtenidos del entrenamiento, he recreado las siguientes gráficas recogiendo las métricas del entrenamiento ver Ilustración 47:

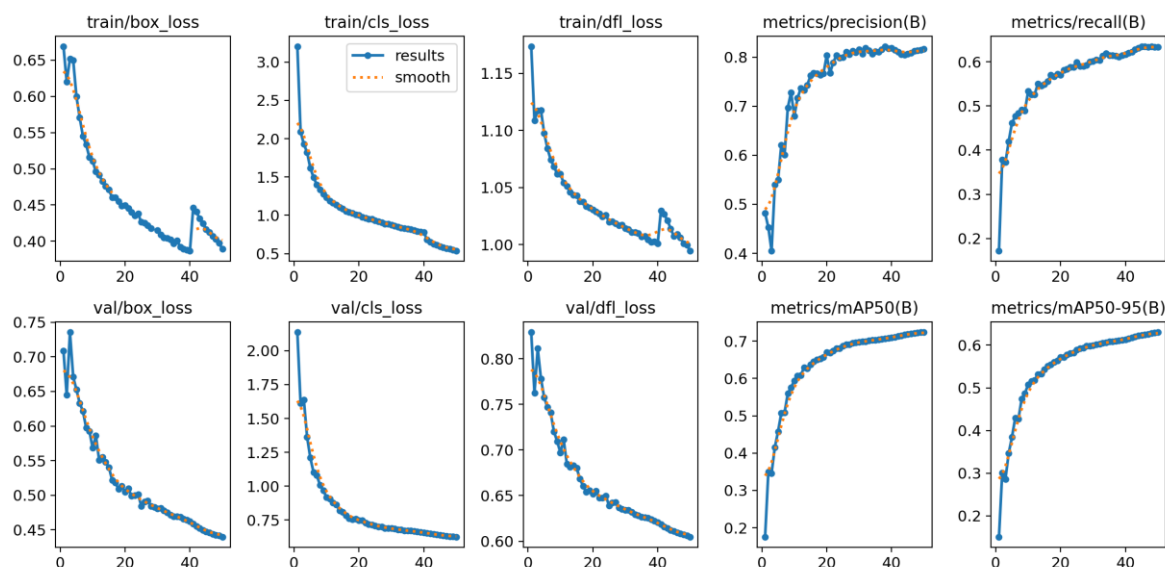


Ilustración 47: Gráficas obtenidas del entrenamiento

Nota: El eje horizontal x representa el número de épocas mientras que el eje vertical y representa los valores obtenidos de las métricas.

3.2.1 Gráfico de Perdida

La pérdida se refiere a como el modelo se equivoca al realizar predicciones durante el entrenamiento y validación. Si el valor disminuye significa que el porcentaje de cometer errores es menor.

Pérdida de caja

Tanto como entrenamiento como validación han descendido de forma monótona empezando desde el 0.7 y acercándose al 0.44, indicando mejora en la localización de objetos.

Interpretación: Como ambas gráficas siguen una tendencia similar y su resultado sea prácticamente igual, quiere decir que esta el ordenador aprendiendo a dibujar cuadros de forma general para entender hasta imágenes nuevas, esto indica una buena **generalización**.

Pérdida de clasificación

El rápido descenso inicial para ambos escenarios refleja un aprendizaje eficaz, donde para el entrenamiento se estabiliza aproximadamente en el valor 0.55

mientras que en la validación en el 0.75, alcanzando el modelo su capacidad máxima para diferenciar entre categorías.

Pérdida de distribución focal

Según las funciones, se puede observar que el modelo está refinando la ubicación de los objetos tanto en su época de entrenamiento como con datos nuevos, mejorando la precisión espacial.

3.2.2 Grafico de Detección

Precisión

La evolución del modelo mejora de 0.5 a 0.8 la precisión en sus detecciones, reduciendo la probabilidad de falsos positivos y aumentando la confianza de las predicciones.

Recall

En esta función se evalúa la proporción de objetos correctamente detectados sobre el total de objetos reales. Su aumento culmina con un valor de 0.63.

Al principio se detectaba menos objetos, pero el modelo ha continuado mejorando su capacidad para no pasar por alto objetos reales.

mAP@0.5 y mAP@50-95

En esta función se evalúa mediante un umbral de solapamiento que establece un porcentaje de margen de error en la tarea de evaluar si el cuadro predicho coincidir con el real. Para ambos niveles, se ha partido de un valor de 0.1 y se ha logrado una precisión superior al 0.65, esto indica un buen desempeño en la predicción.

3.2.3 Matriz de confusión

La matriz de confusión CxC compara un numero de clases C, para cada par de clases (i,j), el eje horizontal representa las clases reales y el eje vertical representa las predicciones del modelo. Las celdas ubicadas dentro de la diagonal principal describen las veces que el modelo predijo correctamente (verdaderos positivos, TP), mientras que las que se encuentran fuera de esta significan los errores donde el modelo predijo una clase, pero la etiqueta real era otra (falso positivo, FP) o donde el modelo no

predijo una clase, pero la etiqueta real sí lo era (falso negativo, FN). Además, se denomina verdadero negativo (TN) a los casos en que el modelo ha predicho correctamente una clase que no aparecía. (37)

		Ground truth		
		+	-	
Predicted	+	True positive (TP)	False positive (FP)	Precision = $TP / (TP + FP)$
	-	False negative (FN)	True negative (TN)	
		Recall = $TP / (TP + FN)$		Accuracy = $(TP + TN) / (TP + FP + TN + FN)$

Ilustración 48: Matriz de confusión (Fuente: Rune Hylsberg Jacobsen)

Gracias a la matriz de confusión se puede representar gráficamente los aciertos y los errores cometidos por un clasificador para cada una de las categorías trabajadas ver Ilustración 49.

En nuestra matriz de confusión, se analiza lo siguiente:

- Cierta confusión entre clases similares como el ejemplo de la columna “Bellpepper__Healthy” donde solo 298 son verdaderos positivos, pero hay 8 clases clasificados como falsos positivos.
- Clases con menos muestras como Grape__Healthy, Jujube__Rotten poseen mucho menos de 40 verdaderos positivos, esto se debe a que la cantidad de datos para estas clases era menor.
- La clase “background” indica las predicciones donde no había ningún objeto de las clases, por ejemplo, en Apple__Healthy se ha obtenido 225 detecciones falsas sobre objetos irrelevantes.
- La diagonal principal destaca las clases de las frutas como manzana, plátano, mango y naranja. Estos valores han indicado que el modelo domina bien la detección de estas clases.

En un balance global la matriz de confusión muestra un buen rendimiento general en la mayoría de las frutas y verduras. Sin embargo, algunas de ellas en versiones podridas muestran un pequeño porcentaje de confusión.

La matriz de confusión normalizada muestra el porcentaje correcto de etiquetas reales predichas de forma correcta, obteniendo un valor del casi 80% para la mayoría de las frutas/verduras ver Ilustración 50.

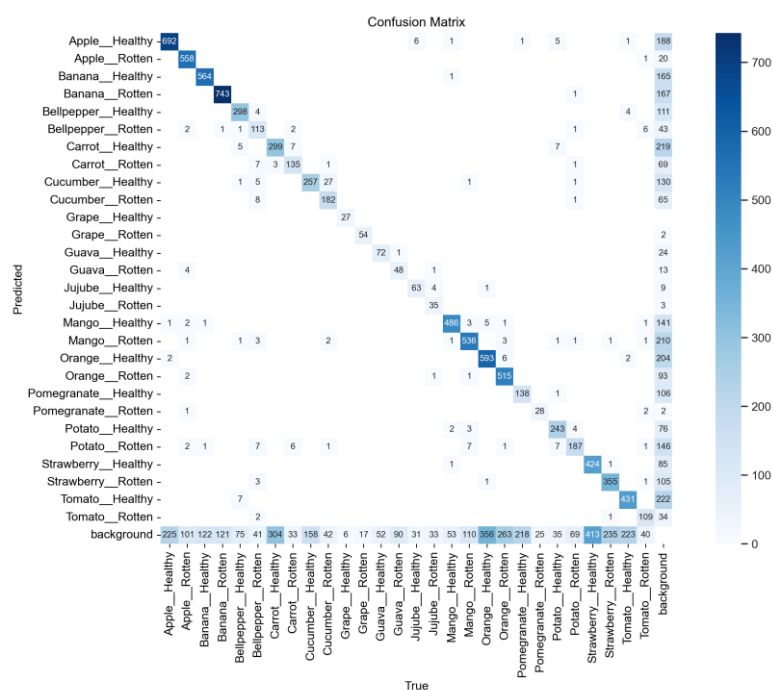


Ilustración 49: Matriz de confusión frutas/verduras

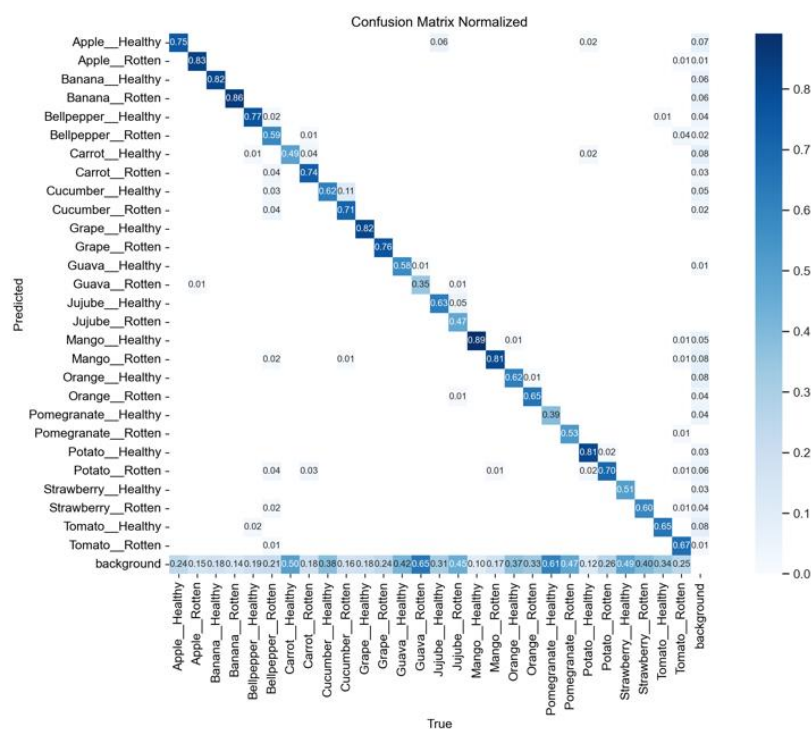


Ilustración 50: Matriz de confusión normalizada

Ejemplo del resultado de entrenamiento del modelo, extracto de un lote del directorio de validación ver Ilustración 51.

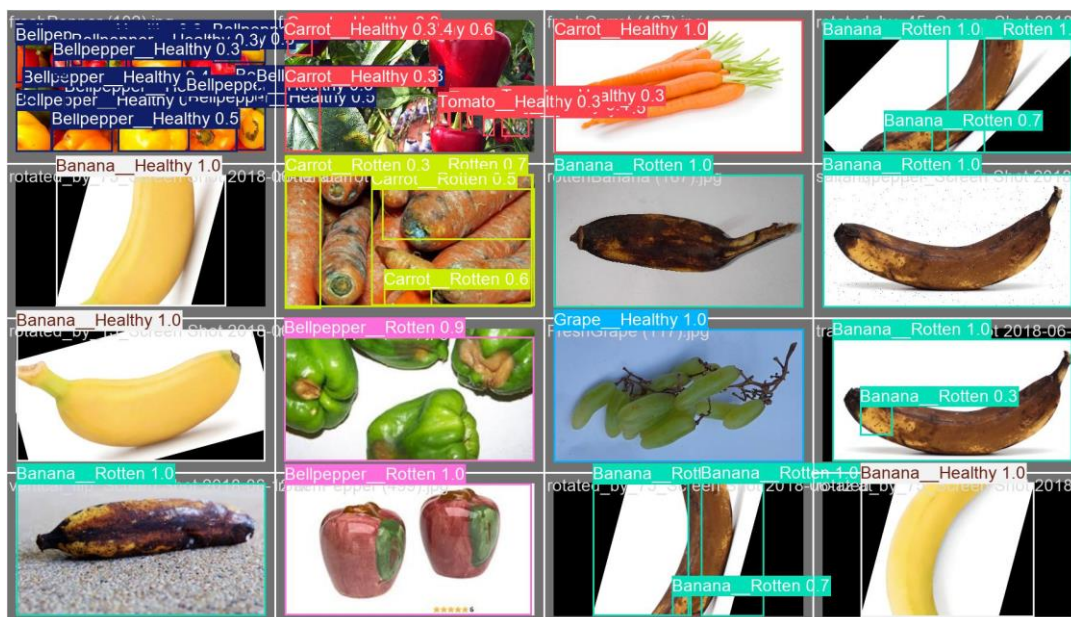


Ilustración 51: Lote de predicción val/

3.3 Adaptación del modelo para Android

En esta fase escogí por exportar el modelo a ONNX en lugar del comúnmente utilizado Torchscript, ya que ONNX ofrece una mayor interoperabilidad con los runtimes móviles. Además según Ultralytics (38), los modelos ONNX tienen mejores tiempos de ejecución en CPUs y GPUs que otros formatos.

Por otro lado, esta selección se debe a un error persistente de incompatibilidad de dimensiones (`RuntimeError: The size of tensor a (3549) must match the size of tensor b (8400) at non-singleton dimension 2`). Sin embargo, este problema no se presentó a la hora de trabajar con el formato ONNX.

Para exportar con ONNX es muy sencillo, tan solo se requiere cargar el modelo, en este caso el archivo `runs/train/weights/best.pt` (este modelo contiene los pesos de la época en la que alcanzo la mejor métrica de validación), y por último exportarlo especificando la versión (opset) y simplificando el grafico para mejorar el rendimiento (simplify). Ver script Ilustración 52 donde se exporta a ONNX.

```
# Cargar el modelo entrenado
model = YOLO("X:/PycharmProjects/Fruit-and-Vegetable-Classfier/runs/detect/train/weights/best.pt")

# Exportar a ONNX
model.export(format="onnx", opset=12, simplify=True)
```

Ilustración 52: export_to_onnx.py

4. Diseño y Desarrollo de la Aplicación "SmartFreezer"

En este capítulo se explica el proceso de diseño en implementación de la aplicación móvil. Se detalla la arquitectura software utilizada, la configuración del IDE, las distintas funcionalidades que le dan vida a la aplicación, el diseño de la UI y UX, así como las tecnologías y librerías que han sido importantes para la construcción de la app.

4.1 Arquitectura de la Aplicación

El patrón de arquitectura seguido para la construcción de la aplicación ha sido Model-View-ViewModel (MVVM). Este facilita la separación de responsabilidades y muestra un código más limpio y organizado. Además de utilizar el patrón MVVM, se han incorporado otros principios de diseño como el Patrón Repositorio, Inyección de Dependencias, el Patrón Observador y el uso de DataSources.

El patrón arquitectónico MVVM divide la aplicación en tres componentes principales:

1. Model

Representa la capa de datos y la lógica de negocio de la aplicación, y no tiene conocimiento directo del View o del ViewModel. Esta capa incluye:

- Clases de Datos: Definen la estructura de los datos manejados, por ejemplo, para las recetas he definido **Result.kt** para describir que datos necesito.
- Lógica de Negocio: Reglas para manipular o procesar estos datos.
- Fuentes de Datos: Encapsulan el acceso a los datos, ya sean remotos o locales.
 - **RemoteDataSource.kt** es responsable de obtener los datos de la API de Spoonacular. utiliza **FoodRecipesApi.kt**.

- Acceso a Firebase Firestore: Lógica para operaciones CRUD en datos del inventario, recetas, perfil, etc. Esta lógica está distribuida entre ViewModel y Fragments.
- Acceso a Firebase Authentication: Lógica para gestionar la autenticación de usuarios.
- Patrón Repositorio: Tan solo existe **RecipeRepository.kt** que actúa como intermediario entre **RecipesViewModel.kt** y **RemoteDataSource.kt**. Abstrae el origen de los datos y proporciona una Api limpia para que los ViewModels accedan a la información.

2. View

Es la capa que representa la interfaz de usuario. Su responsabilidad es mostrar los datos al usuario y capturar sus interacciones. Está compuesta por:

- Activities: HomeActivity es crucial, ya que aloja el NavHostFragment que se trata de la barra de navegación inferior de la aplicación, con el que coordina la visualización de los diferentes Fragments. Los activities actúan como contenedores principales para los Fragments.
- Fragments: Representan las pantallas siendo los principales observadores de los datos expuestos por los ViewModels, en este caso **RecipesFragment.kt** con el **RecipesViewModel.kt**.
- Layouts XML: Definen la estructura visual de las Activities y Fragment. En mi aplicación he incorporado **ViewBinding** para un acceso seguro y eficiente de las vistas (**FragmentRecipesBinding**). También se utiliza **DataBinding** en layouts como **recipes_row_layout.xml** para vincular directamente los datos de un objeto **Result.kt** a las vistas.
- Adapters como **RecipesAdapter.kt** para mostrar cada receta de manera eficiente en un RecyclerView.
- Navigation Component gestiona la navegación entre Fragments, definida en **nav_graph.xml**. **HomeActivity.kt** intercuta con el NavController para cambiar entre los destinos principales como Recetas, Inventario, Lista de compra, etc.

3. ViewModel

Actúa como un intermediario entre la View y el Model. Prepara y gestiona los datos para la View, exponiéndolos a través de objetos observables como **LiveData**. No tiene

conocimiento directo de la View. Por ejemplo, **RecipesViewModel.kt** cumple con lo siguiente:

- Solicita los datos de **RecipeRepository.kt**
- Utiliza LiveData para exponer los datos. **RecipesFragment** observa estos LiveData y actualiza la UI cuando los datos cambian cumpliendo el patrón Observador. La clase sellada **NetworkResult.kt** se usa con **LiveData** para comunicar el estado de las operaciones red (Success, Loading, Error) a la View.
- Uso **ViewModelScope** para lanzar corrutina que se cancelan automáticamente cuando el ViewModel se borra, garantizando que los recursos no se consumen innecesariamente.

4. Patrones y Principios de Diseño aplicados

- **Patrón Repositorio**

Con el uso de **RecipeRepository** podremos centralizar el acceso a los datos de recetas, actuando como un a única fuente de verdad para el **RecipesViewModel** respecto a los datos de la API de Spoonacular. Asimismo, abstrae la lógica de obtención de datos del **RemoteDataSource**. Este patrón garantiza una mejor separación de responsabilidad y facilita la sustitución de fuentes de datos o incluso la adición de caché.

- **Inyección de Dependencias con la herramienta Hilt**

En muchos módulos de la aplicación como Activities, Fragments, ViewModels..., utilizamos este patrón para gestionar las dependencias, esto reduce la cantidad de creación manual de objetos

En la aplicación **NetworkModule.kt** provee instancias de **OkHttpClient**, **GsonCoberterFactory**, **Retrofit** y **FoodRecipesApi**. Hilt inyecta **FoodRecipesApi** en **RemoteDataSource**, **RemoteDataSource** en **RecipeRepository**, y **RecipeRepository** junto con **FoodRecipesApi** en **RecipesViewModel**.

- **Patrón Observador con LiveData**

RecipesViewModel expone datos a través de LiveData (**recipesResponse: LiveData<NetworkResult<List<Result>>>**)-

Los Fragments (Observers) se suscriben a estos LiveData. Cuando los datos en el LiveData cambian (ej. se completa una llamada de red), los observadores son notificados automáticamente y pueden actualizar la UI en consecuencia. Esto crea un flujo de datos reactivo y desacoplado.

- **Singleton a través de Hilt y Object Declarations**

Por ejemplo, en NetworkModule.kt con @Singleton, asegura que solo exista una instancia de los objetos que provee en toda la aplicación.

Las declaraciones object en Kotlin también crean instancias únicas. En este caso **FCMTopicManager**, **LocaleHelper** y **Constants** (donde se declara las constantes de configuración y consultas de la API).

4.2 Diseño y herramientas utilizadas de Firebase para la app

Antes comenzar a explicar el funcionamiento de la aplicación, quiero entrar en detalle con los aspectos técnicos que he abordado para la gestión de datos con la herramienta Firebase.

El diagrama de base de datos utilizado es el siguiente Ilustración 53:

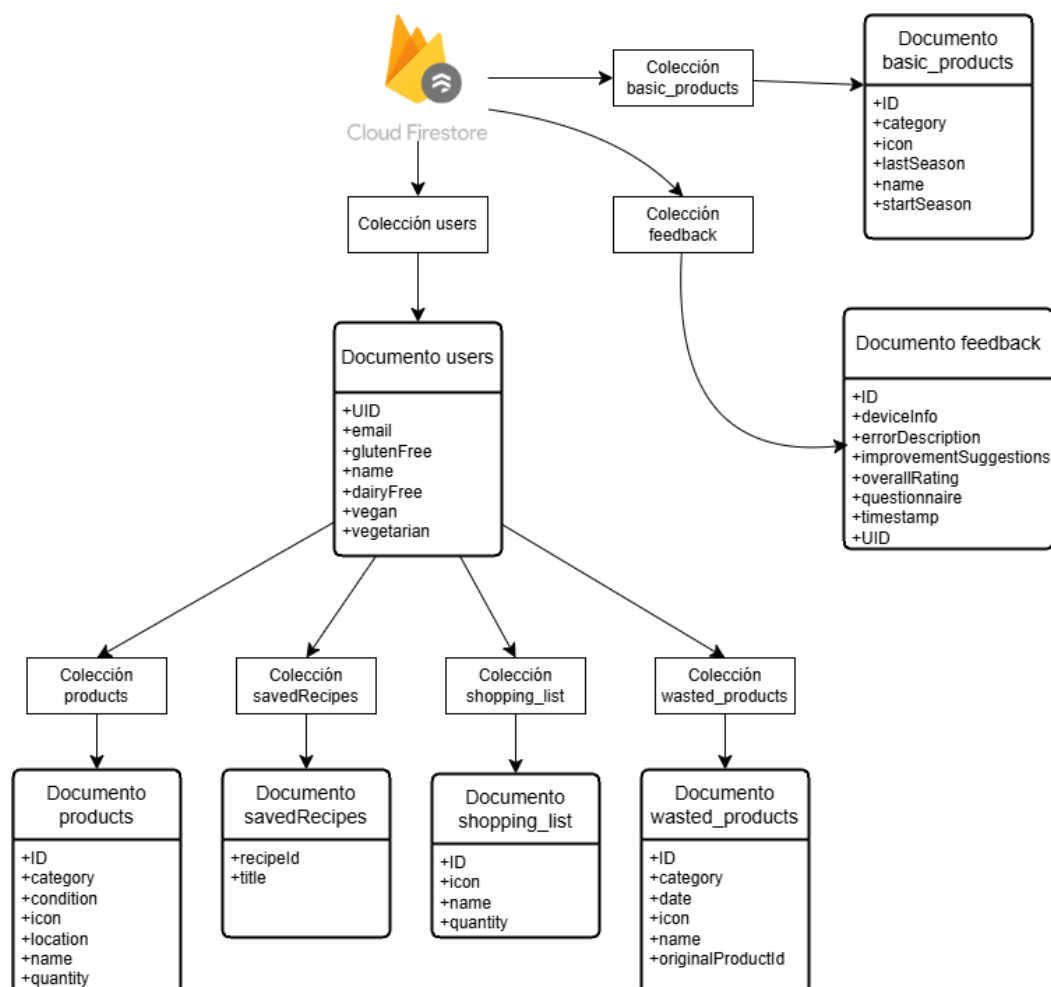


Ilustración 53: Diagrama BBDD de la app

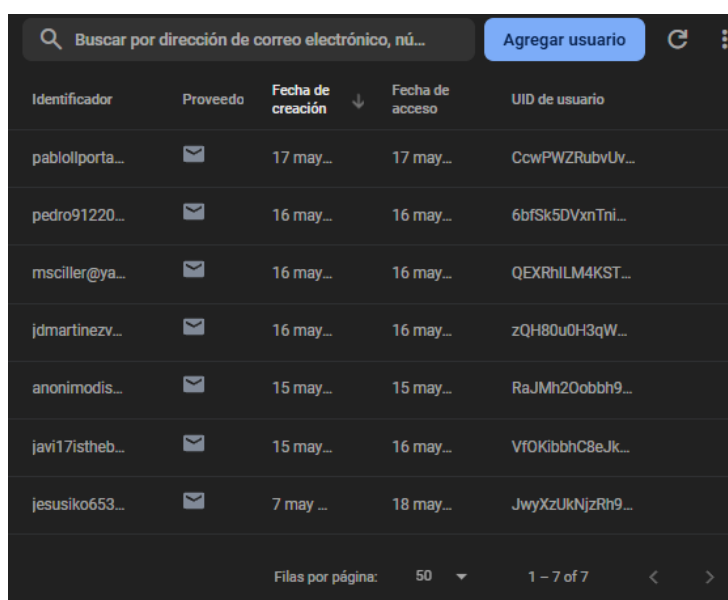
He creado tres colecciones principales:

- **Colección users:** por cada usuario, la función de Firebase Auth genera un identificador único de usuario. En cada usuario se guarda su email, sus preferencias dietéticas en booleanos y su nombre. Además, también se crea las siguiente subcolecciones
 - **Products:** Se trata de los productos que tiene el usuario en su inventario, se guarda el id, la categoría (fruta o verdura), el estado (fresco o podrido), la localización (nevera, despensa, congelador), el nombre, la cantidad y icon (nombre del archivo imagen drawable).
 - **SavedRecipes:** Esta representa a las recetas guardadas del usuario, guardando el id de la receta y el título.
 - **Shopping_list:** Significa los productos que tiene el usuario agregado a la lista de la compra.

- Wasted_products: Todos los desperdicios del usuario donde se genera un id automático, pero se guarda el id del producto que ha sido desperdiciado ya que se debe evitar duplicados y la fecha registrada.
- La colección basic_products representa la información estática de cada producto, es decir, su fecha de temporada, su nombre, categoría y su icono.
- La colección feedback tiene la finalidad de conocer la valoración de cada usuario, se guarda la información de que versión está utilizando de la app y su modelo de teléfono. En esta reseña se guarda los valores del 1 al 5 de cada aspecto como aspectos de usabilidad y experiencia de usuario. Además, se le pregunta al usuario que errores o mejoras añadiría a la aplicación.

Firestore Authentication

En Firebase Console se encuentra la función de Authentication donde podemos gestionar a los usuarios autenticados en la aplicación ver Ilustración 54. Esta herramienta nos facilita el proceso de verificación de dirección de correo electrónico, restablecer contraseña, etc., debido a las plantillas ya establecidas para enviar un mensaje vía correo electrónico al usuario ver Ilustración 55. La integración de Firebase Authentication en la aplicación simplifica el proceso de gestión de usuarios, al aprovechar sus funciones preexistentes para el registro, inicio de sesión, recuperación de contraseñas, etc.



Identificador	Proveído	Fecha de creación	Fecha de acceso	UID de usuario
pablollporta...	✉	17 may...	17 may...	CcwPWZRubvUv...
pedro91220...	✉	16 may...	16 may...	6bfSk5DVxnTni...
mrciller@ya...	✉	16 may...	16 may...	QEXRhILM4KST...
jdmartinezv...	✉	16 may...	16 may...	zQH80u0H3qW...
anonimodis...	✉	15 may...	15 may...	RaJMh2Oobbh9...
javi17istheb...	✉	15 may...	16 may...	VfOKibbhC8eJk...
jesusiko653...	✉	7 may ...	18 may...	JwyXzUkNjzRh9...

Files por página: 50 1 - 7 of 7

Ilustración 54: Gestión de usuarios Firebase

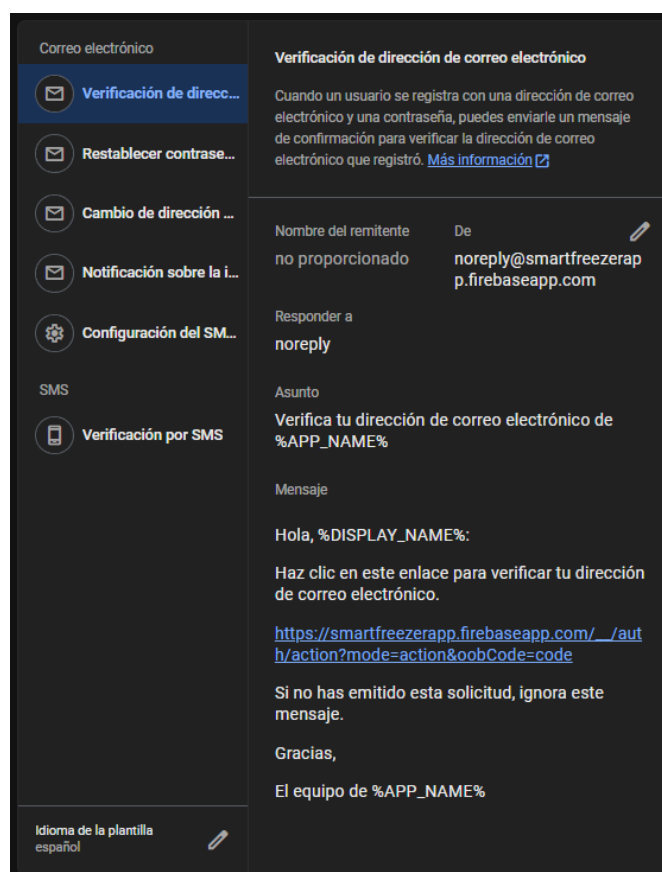


Ilustración 55: Plantillas Firebase

Firestore Cloud Messaging

Esta opción de Firebase tiene como utilidad crear notificaciones personalizadas al usuario, aprovecharemos esta herramienta para establecer notificaciones en la que se recomienda al usuario una receta según sus preferencias dietéticas ver Ilustración 56. Es sencillo, la configuración de notificaciones con esta utilidad, por ejemplo, a través del ajuste de datos personalizados podemos declarar un tipo de dieta y el identificador de una receta con la que podremos crear, en el caso de la Ilustración 57, una receta sin lactosa. Cloud Messaging resulta muy potente ya que podremos ajustar la programación de la notificación y la selección de usuarios específicos.

1 Notificación

Título de la notificación ⓘ
Ingresa un título opcional

Texto de la notificación
Tenemos nuevas sugerencias sin lactosa para ti

Imagen de notificación (opcional) ⓘ
Ejemplo: <https://tuapp.com/imagen.png>

Nombre de la notificación (opcional) ⓘ
Tenemos nuevas sugerencias sin lactosa para ti

Enviar mensaje de prueba

Vista previa del dispositivo

En esta vista previa, se ofrece una idea general de cómo se mostrará tu mensaje en un dispositivo móvil. La apariencia real del mensaje varía en función del dispositivo. Para obtener resultados precisos, prueba con un dispositivo real.

Estado inicial Vista expandida

Android

Apple

Siguiete

Ilustración 56: Personalización notificación

5 Opciones adicionales (opcional)

Todos los campos son opcionales

Canal de notificación de Android ⓘ

Datos personalizados ⓘ

FILTER_DIET dairyFree

RECIPE_ID 716406

Clave Valor

Sonido
Habilitado

Ilustración 57: Ajustes notificación con valores adicionales

Por último, para que el usuario pueda recibir las notificaciones, se debe configurar lo siguiente dentro de la aplicación:

MyFirebaseMessagingService es una clase que he creado que hereda de la clase **FirebaseMessagingService** de Firebase con la que permite al usuario recibir notificaciones.

Funciona de la siguiente forma: cuando el usuario recibe una notificación desde Firebase, `onMessageReceived()` ver Ilustración 58 extra los datos de la notificación y decide si se debe mostrar una notificación local. Mientras que

`sendNotification()` ver Ilustración 59 comprueba en **SharedPrefManager** que el usuario tiene activadas las notificaciones. Como resultado, al cumplir las condiciones anteriores, se construye y se muestra una notificación local en la app.

Por otro, lado he creado otra clase que administra la suscripción y desuscripción a topics, según las preferencias dietéticas del usuario, es decir, primero llama a `unsubscribeFromAllDietTopics()` ver Ilustración 60 para evitar suscripciones duplicadas, y luego suscribe al usuario solo a los topics de dieta que ha marcado como **true** en su colección de Firestore. Además, existe la función de desuscribirse de todos los topics relaciones con dietas para mantener las suscripciones limpias y actualizadas `unsubscribeFromAllDietTopics()` ver Ilustración 61.

```
override fun onMessageReceived(remoteMessage: RemoteMessage) {
    Log.d(TAG, "From: ${remoteMessage.from}")

    // Variables para los datos de la notificación
    var notificationTitle = remoteMessage.notification?.title ?: "SmartFreezer"
    var notificationBody = remoteMessage.notification?.body ?: "Tenemos nuevas sugerencias para ti."

    // Extraer datos personalizados
    val dataPayload = remoteMessage.data
    val filterDiet = dataPayload["FILTER_DIET"]
    val recipeId = dataPayload["RECIPE_ID"]

    Log.d(TAG, "Data Payload: $dataPayload")

    // Mostrar notificación
    sendNotification(notificationTitle, notificationBody, filterDiet, recipeId)
}

override fun onNewToken(token: String) {
    Log.d(TAG, "Refreshed token: $token")
}
```

Ilustración 58: Función `onMessageReceived`

```

private fun sendNotification(
    title: String,
    messageBody: String,
    filterDiet: String?,
    recipeId: String?
) {
    // Verificar si el usuario tiene activadas las notificaciones
    val sharedPrefManager = SharedPrefManager(applicationContext)
    if (!sharedPrefManager.getNotificationStatus()) {
        Log.d(TAG, "Notificaciones desactivadas por el usuario.")
        return
    }

    // Verificar si el usuario aún tiene esta preferencia
    val userId = FirebaseAuth.getInstance().currentUser?.uid ?: return

    FirebaseFirestore.getInstance().collection("users")
        .document(userId)
        .get()
        .addOnSuccessListener { document ->
            val user = document.toObject(User::class.java) ?: return@addOnSuccessListener

            when (filterDiet) {
                "vegan" -> if (!user.vegan) return@addOnSuccessListener
                "vegetarian" -> if (!user.vegetarian) return@addOnSuccessListener
                "glutenFree" -> if (!user.glutenFree) return@addOnSuccessListener
                "dairyFree" -> if (!user.dairyFree) return@addOnSuccessListener
            }

            showActualNotification(title, messageBody, filterDiet, recipeId)
        }
}

```

Ilustración 59: Función sendNotification

```

// Suscribirse al usuario según sus preferencias
fun subscribeToDietTopics(user: User) {
    unsubscribeFromAllDietTopics() // Primero nos aseguramos de desuscribir de todo

    if (user.vegan) {
        FirebaseMessaging.getInstance().subscribeToTopic("vegan")
            .addOnCompleteListener { task ->
                Timber.d("Subscribed to vegan topic: ${task.isSuccessful}")
            }
    }
    if (user.vegetarian) {
        FirebaseMessaging.getInstance().subscribeToTopic("vegetarian")
            .addOnCompleteListener { task ->
                Timber.d("Subscribed to vegetarian topic: ${task.isSuccessful}")
            }
    }
    if (user.glutenFree) {
        FirebaseMessaging.getInstance().subscribeToTopic("glutenFree")
            .addOnCompleteListener { task ->
                Timber.d("Subscribed to glutenFree topic: ${task.isSuccessful}")
            }
    }
    if (user.dairyFree) {
        FirebaseMessaging.getInstance().subscribeToTopic("dairyFree")
            .addOnCompleteListener { task ->
                Timber.d("Subscribed to dairyFree topic: ${task.isSuccessful}")
            }
    }
}

```

Ilustración 60: Función subscribeToDietTopics

```
// Desuscribe de todos los topics dietéticos
fun unsubscribeFromAllDietTopics() {
    val topics = listOf("vegan", "vegetarian", "glutenFree", "dairyFree")
    topics.forEach { topic ->
        FirebaseMessaging.getInstance().unsubscribeFromTopic(topic)
            .addOnCompleteListener { task ->
                Timber.d("Unsubscribed from $topic: ${task.isSuccessful}")
            }
    }
}
```

Ilustración 61: Función unsubscribeFromAllDietTopics

El flujo de notificaciones funciona de forma tan sencillo, que solo sigue estos pasos:

1. Usuario activa dieta tipo “vegetarian”.
2. FCMTopicManager lo suscribe al topic “vegetarian”.
3. Desde el servidor, se envía una notificación al topic.
4. MyFirebaseMessagingService la recibe, valida los datos y las preferencias
5. Se muestra una notificación que al, pulsarla, abre la app a una receta sugerida.

4.3 Funcionamiento de la aplicación

En este apartado se explica el funcionamiento principal del código, organizado según la arquitectura MVVM: Vista (View), Modelo (Model) y ViewModel. Se detallan las responsabilidades y funciones clave de cada componente dentro de la aplicación.

4.3.1 Model

Para representar la capa de datos y la lógica de presentación, se incluye en la app:

Las **clases de datos** para los resultados de las recetas `Result.kt`, para los productos del inventario `UserProduct.kt`, para la información del usuario `User.kt`. Estas clases nos ayudaran para transportar los datos de los servicios de la API o Firebase. ver ejemplo Ilustración 63. Para obtener los datos de la API, he creado la estructura a través del resultado de la siguiente consulta

<https://api.spoonacular.com/recipes/complexSearch?ids=715538&apiKey=numberApiKey&diet=&type=&addRecipeInformation=true&fillIngredients=true>

Esto nos devuelve un JSON con el que a través del plugin `JsonToKotlinClass` podremos transformarlo a clases de datos, ver Ilustración 62.

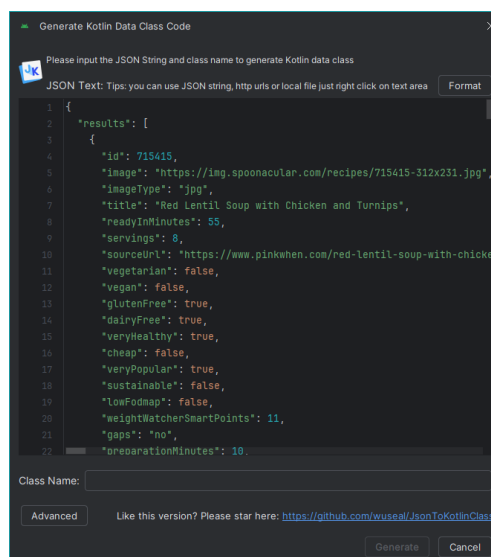


Ilustración 62: JSON to Kotlin Class

```
package com.example.smartfreezer.models

import com.google.gson.annotations.SerializedName

data class Result(
    @SerializedName("dishTypes")
    val dishTypes: List<String>,
    @SerializedName("id")
    val id: Int,
    @SerializedName("image")
    val image: String,
    @SerializedName("readyInMinutes")
    val readyInMinutes: Int,
    @SerializedName("spoonacularScore")
    val spoonacularScore: Double,
    @SerializedName("title")
    val title: String,
)
```

Ilustración 63: Result.kt

Los **repositorios**, en este caso `RecipeRepository.kt`, que actúa como intermediario entre `RecipesViewModel` y la API de Spoonacular. Su función es centralizar el acceso a fuentes de datos. Ver Ilustración 64.

```
package com.example.smartfreezer.repository

import com.example.smartfreezer.RemoteDataSource
import dagger.hilt.android.scopes.ActivityRetainedScoped
import javax.inject.Inject

@ActivityRetainedScoped
class RecipeRepository @Inject constructor(
    remoteDataSource: RemoteDataSource,
) {
    val remote = remoteDataSource
}
```

Ilustración 64: RecipeRepository.kt

Las **fuentes de datos**, la clase `RemoteDataSource.kt` (ver Ilustración 65), obtiene e interpreta la información mediante llamadas a la API remota usando Retrofit. Las llamadas se encuentran definidas en la interfaz `FoodRecipesApi.kt` (ver Ilustración 67), donde **`getRecipes()`** sirve para buscar y obtener una gran cantidad de recetas, y **`getRecipeDetails()`** que a través del ID de una receta obtengo toda su información. Las llamadas están configuradas en `NetworkModule.kt` (ver Ilustración 66), que utiliza el patrón Singleton y de inyección de dependencias Hilt (`@Provides`) para asegurar que todos los componentes de red se inicialicen una sola vez y estén disponibles de forma eficiente en cualquier lugar que se necesite en la aplicación.

```
import com.example.smartfreezer.api.FoodRecipesApi
import com.example.smartfreezer.models.FoodRecipe
import javax.inject.Inject
import retrofit2.Response

class RemoteDataSource @Inject constructor(
    private val foodRecipesApi: FoodRecipesApi
) {
    suspend fun getRecipes(queries: Map<String, String>): Response<FoodRecipe> {
        return foodRecipesApi.getRecipes(queries)
    }
}
```

Ilustración 65: RemoteDataSource.kt

```
@Singleton
@Provides
fun provideApiService(retrofit: Retrofit) : FoodRecipesApi {
    return retrofit.create(FoodRecipesApi::class.java)
}
```

Ilustración 66: NetworkModule.kt


```
package com.example.smartfreezer.api

import com.example.smartfreezer.models.FoodRecipe
import com.example.smartfreezer.models.RecipeDetails
import retrofit2.Response
import retrofit2.http.GET
import retrofit2.http.Path
import retrofit2.http.Query
import retrofit2.http.QueryMap

interface FoodRecipesApi {

    @GET("/recipes/complexSearch")
    suspend fun getRecipes(
        @QueryMap queries: Map<String, String>
    ): Response<FoodRecipe>

    @GET("/recipes/{id}/information")
    suspend fun getRecipeDetails(
        @Path("id") recipeId: Int,
        @Query("apiKey") apiKey: String,
        @Query("includeNutrition") includeNutrition: Boolean
    ): Response<RecipeDetails>
}
```

Ilustración 67: FoodRecipeApi.kt

Nota: Retrofit es una librería de código abierto que se encarga de las solicitudes HTTP como GET, POST, etc. Esto facilita la interacción con los servicios web ya que evita la construcción de código para manejar estas interacciones. Por otro lado, Dagger Hilt es una biblioteca de inyección de dependencias para Android que reduce la cantidad de código repetitivo.

4.3.2 View

Las Views que he implementado son las siguientes:

Los **activities** principales utilizados en la app son los siguientes:

- **MainActivity**: Sirve como pantalla de entrada de la app. Es la primera pantalla que muestra la aplicación, y su única función es mostrar un botón que lleve al usuario a la pantalla de login.
- **LoginActivity**: Pantalla de inicio de sesión de la app, controla toda lógica de autenticación del usuario con Firebase a través de la función `signInWithEmailAndPassword()` Ilustración 68; si el login es exitoso nos redirige a **HomeActivity**, también existe una redirección si el usuario desea registrar.
 - Usar Firebase Auth para verificar email y contraseña

- Guardar preferencia “mantener sesión iniciada” en `SharedPreferences`.
- Lógica de sesión automática donde si el usuario ya inicio sesión previamente y su correo esta verificado, lo manda directamente a **HomeActivity** sin pedirle datos otra vez
- Opción de reestablecer contraseña mediante un dialogo donde el usuario introduce su correo para recibir el email a través de la función `showForgotPasswordDialog` (Ilustración 69) gracias a la opción `sendPasswordResetEmail` de `Firebase Auth`.
- Manejo de mensajes personalizados de error.
- Al iniciar sesión, recupera los datos del usuario desde `Firestore` y **lo suscribe a temas FCM (Firebase Cloud Messaging)** relacionados con su dieta.

```
auth.signInWithEmailAndPassword(email, password)
    .addOnCompleteListener(this) { task ->
        if (task.isSuccessful) {
            val user = auth.currentUser
            if (user != null && user.isEmailVerified) {
                // Guardar preferencia de mantener sesión
                sharedPreferences.edit().apply {
                    putBoolean("keep_logged_in", keepLoggedInCheckBox.isChecked)
                }.apply()

                startActivity(Intent(this, HomeActivity::class.java))
                finish()
            } else {
                auth.signOut()
                showErrorToast("Debes verificar tu correo antes de iniciar sesión")
            }
        } else {
            val error = when (task.exception) {
                is FirebaseAuthInvalidUserException -> "Error: {Usuario no registrado}"
                is FirebaseAuthInvalidCredentialsException -> "Error: {Credenciales incorrectas}"
                else -> "Error: {task.exception?.message}"
            }
            showErrorToast(error)
        }
    }
}
```

Ilustración 68: Lógica de inicio de sesión

```
private fun showForgotPasswordDialog() {
    val builder = AlertDialog.Builder(this)
    builder.setTitle("Restablecer contraseña")
    builder.setMessage("Ingresa tu correo electrónico para recibir un enlace de...")

    val input = TextInputEditText(this).apply {
        hint = "Correo electrónico"
        setPadding(32, 16, 32, 16)
    }

    builder.setView(input)
    builder.setPositiveButton("Enviar") { dialog, _ ->
        val email = input.text.toString().trim()
        if (email.isNotEmpty()) {
            auth.sendPasswordResetEmail(email)
            .addOnCompleteListener { task ->
                if (task.isSuccessful) {
                    Toast.makeText(this, "Correo de restablecimiento enviado", Toast.LENGTH_LONG).show()
                } else {
                    showErrorToast("Error al enviar correo: {task.exception?.message}")
                }
            }
        }
    }
    builder.setNegativeButton("Cancelar", null)
    builder.show()
}
```

Ilustración 69: Lógica para restablecer contraseña

- **RegisterActivity:** Pantalla de registro de la app, permite crear una cuenta con Firebase usando `createUserWithEmailAndPassword()` ver Ilustración 70; si el registro es exitoso el usuario recibe un correo de verificación, tras aceptarlo, se guarda su información en Firestore.
 - Usar Firebase Auth para crear usuarios.
 - Envía un correo de verificación con `sendEmailVerification()` ver Ilustración 71.
 - Checkbox para aceptar términos, con acceso a **TermsActivity**.
 - Opción de volver a LoginActivity.
 - Manejo de mensajes personalizados de error.

```

if (!acceptTermsCheckBox.isChecked) {
    showErrorToast(getString(R.string.register_terms_not_accepted))
    isValid = false
}

if (isValid) {
    auth.createUserWithEmailAndPassword(email, password)
        .addOnCompleteListener(this) { task ->
            if (task.isSuccessful) {
                val user = auth.currentUser
                user?.sendEmailVerification()
                    ?.addOnCompleteListener { verifyTask ->
                        if (verifyTask.isSuccessful) {
                            verificationEmailSent = true
                            showVerificationDialog(email, user.uid, name)
                        } else {
                            user.delete()
                            showErrorToast(getString(R.string.register_verification_error, verifyTask.exception?.message))
                        }
                    }
            } else {
                when (task.exception) {
                    is FirebaseAuthUserCollisionException -> {
                        showErrorToast(getString(R.string.error_email_already_exists))
                    }
                    else -> {
                        showErrorToast(getString(R.string.register_error, task.exception?.message))
                    }
                }
            }
        }
}

```

Ilustración 70: createUserWithEmailAndPassword

```

private fun showVerificationDialog(email: String, userId: String, name: String) {
    val builder = AlertDialog.Builder(this)
    builder.setTitle(getString(R.string.verify_email_title))
    builder.setMessage(getString(R.string.verify_email_message, email))

    builder.setPositiveButton(getString(R.string.verify_email_resend)) { dialog, _ ->
        auth.currentUser?.sendEmailVerification()
            ?.addOnCompleteListener { task ->
                if (task.isSuccessful) {
                    Toast.makeText(this, getString(R.string.verify_email_resend), Toast.LENGTH_SHORT).show()
                } else {
                    showErrorToast(getString(R.string.register_verification_error, task.exception?.message))
                }
            }
    }

    builder.setNegativeButton(getString(R.string.verify_email_understood)) { dialog, _ ->
        // Solo guardar en Firestore después de verificación
        val userRef = db.collection("users").document(userId)
        userRef.set(hashMapOf(
            "name" to name,
            "email" to email,
            "verified" to true
        ))

        auth.signOut()
        startActivity(Intent(this, LoginActivity::class.java).apply {
            putExtra("emailPendingVerification", true)
        })
        finish()
    }

    builder.setOnCancelListener {
        if (!verificationEmailSent) {
            auth.currentUser?.delete()
        }
    }
}

```

Ilustración 71: sendEmailVerification

- **HomeActivity:** Pantalla principal de la app, contiene la navegación entre fragmentos. Es la primera pantalla tras el login exitoso.
 - Verifica si el usuario ha iniciado sesión, además detecta enlaces de verificación de correo electrónico y muestra un mensaje si se confirma.
 - Gestiona la navegación con NavController y un SmoothBottomBar ver lógica en Ilustración 72. Para crear la navegación se debe crear el archivo en res/navigation/nav_graph.xml, en este se define el grafo de navegación de una app, especificando los fragmentos disponibles y como se puede navegar entre ellos, además se le puede especificar argumentos que se usan para enviar datos de un fragmento a otro durante la navegación. Ver grafo visual en Ilustración 73.
 - Controla también la navegación entre pestañas secundarias de fragmentos.
 - Si se recibe una notificación (con `RECIPE_ID` o `FILTER_DIET`), redirige automáticamente a **RecipeDetailsFragment.kt**.

```
// Setup NavController
val navHostFragment = supportFragmentManager
    .findFragmentById(R.id.nav_host_fragment) as NavHostFragment
navController = navHostFragment.navController

// Muestra InventoryFragment por defecto
navController.navigate(R.id.inventoryFragment)

// Mapeo de items por índice
binding.bottomBar.setOnItemSelectedListener { index ->
    when (index) {
        0 -> navController.navigate(R.id.listFragment)
        1 -> navController.navigate(R.id.scannerFragment)
        2 -> navController.navigate(R.id.inventoryFragment)
        3 -> navController.navigate(R.id.recipesFragment)
        4 -> navController.navigate(R.id.alertsFragment)
    }
    true
}

// Mantén la barra sincronizada con el fragmento actual
navController.addOnDestinationChangedListener { _, destination, _ ->
    val index = when (destination.id) {
        R.id.listFragment -> 0
        R.id.scannerFragment -> 1
        R.id.inventoryFragment -> 2
        R.id.recipesFragment -> 3
        R.id.alertsFragment -> 4
        R.id.addShoppingListFragment -> -1 // No cambian la selección
        else -> 2 // Default: inventario
    }
    if (index != -1) { // Solo actualiza si no es -1
        binding.bottomBar.itemActiveIndex = index
    }
}
```

Ilustración 72: NavController



Ilustración 73: nav_graph.xml

- **ProfileActivity:** Pantalla que permite al usuario modificar su perfil en la app. Trabaja con Firebase Auth y Firestore para obtener y actualizar los datos del usuario. Cabe mencionar, que en esta actividad es donde se establece que notificación de receta recomendada recibirá el usuario a través de las preferencias escogidas.

La información del usuario se carga desde la colección "users" accediendo al documento correspondiente al usuario actual, el cual es obtenido mediante el método `getCurrentUser()` de la actividad que instanció el proceso. Posteriormente, la vista se actualiza en el método `loadUserData()` ver Ilustración 74 .

- Utiliza diálogos para editar nombre, correo, contraseña, preferencias dietéticas, y también, para cerrar sesión y eliminar cuenta.
- Siempre se valida la contraseña antes de hacer cambios sensibles para notificar al usuario de su acción.

```
private fun loadUserData() {  
    // Obtener datos de Firestore  
    db.collection("users").document(userId)  
        .get()  
        .addOnSuccessListener { document ->  
            if (document.exists()) {  
                // Mostrar nombre  
                val name = document.getString("name") ?: "Usuario"  
                binding.userNameTextView.text = name  
                binding.tvGreetingProfile.text = "Hola, {name}"  
  
                // Mostrar preferencias  
                val vegan = document.getBoolean("vegan") ?: false  
                val vegetarian = document.getBoolean("vegetarian") ?: false  
                val glutenFree = document.getBoolean("glutenFree") ?: false  
                val dairyFree = document.getBoolean("dairyFree") ?: false  
  
                val dietText = buildString {  
                    if (vegan) append("Vegano")  
                    else if (vegetarian) append("Vegetariano")  
                    else append("Omnívoro")  
                }  
  
                val intolerancesText = buildString {  
                    val intolerances = mutableListOf<String>()  
                    if (glutenFree) intolerances.add("Gluten")  
                    if (dairyFree) intolerances.add("Lactosa")  
  
                    if (intolerances.isEmpty()) {  
                        append("Ninguna")  
                    } else {  
                        append(intolerances.joinToString(", "))  
                    }  
                }  
            }  
        }  
}
```

Ilustración 74: Código ProfileActivity método loadUserData()

- **SettingsActivity:** Pantalla donde aparece la configuración de la app. Esta actividad permite al usuario lo siguiente:
 - Activar/Desactivar notificaciones en la función `setupNotificationSwitch()` ver Ilustración 75. Si el usuario activa la opción se suscribe al topic "all" de Firebase Messaging mientras que si lo desactiva se desuscribe y no recibirá notificaciones.
 - Cambiar el tema claro/oscuro. La aplicación lee el estado actual con `resources.configuration.uiMode` y cambiara el tema con `AppCompatDelegate.setDefaultNightMode(...)` ver Ilustración 76. Para que mi aplicación en Android cambie automáticamente entre **modo claro y modo oscuro**, evité usar colores directos en los archivos XML

de diseño. En lugar de eso, definí nombres genéricos de colores que Android puede adaptar según el tema del sistema.

Primero, creé algunos atributos personalizados como `<attr name="colorTextPrimary" format="reference|color"/>`, y los coloqué en el archivo `attrs.xml`. Estos atributos funcionan como etiquetas que describen el propósito del color, no el color en sí.

Después, configuré dos versiones del archivo `colors.xml`:

- Una está en `res/values/`, donde asigné los colores que se usan en el **modo claro**.
- La otra está en `res/values-night/`, con los colores correspondientes para el **modo oscuro**.

Por último, en los archivos de temas (`themes.xml` y `themes-night.xml`), enlacé esos atributos con los colores definidos en `colors.xml`.

De esta forma, Android se encarga de cambiar los colores automáticamente cuando el usuario activa el modo oscuro en su dispositivo.

- Cambiar el idioma mediante un dialogo con las opciones español o inglés ver Ilustración 77. Si se selecciona otro idioma, se actualiza el Locale (configuración regional) y reinicia la app con un Intent a `HomeActivity`. Para que Android cambie el idioma automáticamente, cada texto debe guardarse con un nombre genérico en `strings.xml` (español) y `strings-en.xml` (inglés).
- Ver información sobre “SmartFreezer”, términos o soporte
- Guardar las preferencias locales escogidas por el usuario (idioma, modo oscuro y notificaciones) a través de la clase **SharedPrefManager.kt**


```
private fun setupNotificationSwitch() {
    binding.notificationsSwitch.isChecked = sharedPrefManager.getNotificationStatus()

    binding.notificationsSwitch.setOnCheckedChangeListener { _, isChecked ->
        sharedPrefManager.setNotificationStatus(isChecked)

        if (isChecked) {
            FirebaseMessaging.getInstance().subscribeToTopic("all")
            .addOnCompleteListener { task ->
                if (task.isSuccessful) {
                    Toast.makeText(this,
                        "Notificaciones activadas", Toast.LENGTH_SHORT).show()
                }
            }
        } else {
            FirebaseMessaging.getInstance().unsubscribeFromTopic("all")
            .addOnCompleteListener { task ->
                if (task.isSuccessful) {
                    Toast.makeText(this,
                        getString(R.string.notificaciones_desactivadas), Toast.LENGTH_SHORT).show()
                }
            }
        }
    }
}
}
```

Ilustración 75: Código SettingsActivity.kt método setupNotificationSwitch()

```
private fun setupDarkModeSwitch() {
    // Configurar el switch según el modo actual
    val currentNightMode = resources.configuration.uiMode and Configuration.UI_MODE_NIGHT_MASK
    binding.darkModeSwitch.isChecked = currentNightMode == Configuration.UI_MODE_NIGHT_YES

    binding.darkModeSwitch.setOnCheckedChangeListener { _, isChecked ->
        if (isChecked) {
            AppCompatDelegate.setDefaultNightMode(AppCompatDelegate.MODE_NIGHT_YES)
        } else {
            AppCompatDelegate.setDefaultNightMode(AppCompatDelegate.MODE_NIGHT_NO)
        }
        sharedPrefManager.setDarkModeEnabled(isChecked)
    }
}
}
```

Ilustración 76: Código SettingsActivity.kt método setupDarkModeSwitch()

```
private fun changeAppLanguage(languageCode: String) {
    sharedPrefManager.setAppLanguage(languageCode)
    LocaleHelper.setLocale(this, languageCode)

    // Forzar recreación de toda la aplicación
    val intent = Intent(this, HomeActivity::class.java).apply {
        flags = Intent.FLAG_ACTIVITY_CLEAR_TASK or Intent.FLAG_ACTIVITY_NEW_TASK
        putExtra("navigateTo", "inventory")
    }
    startActivity(intent)
    overridePendingTransition(android.R.anim.fade_in, android.R.anim.fade_out)
}

data class LanguageOption(val name: String, val code: String)
```

Ilustración 77: Código SettingsActivity.kt método changeAppLanguage()

- **BaseActivity:** Clase que heredan las actividades que sirve para aplicar automáticamente el idioma elegido por el usuario usando LocaleHelper y evitar que tener que repetir código de cambio de idioma para cada actividad. Tiene como función cargar el idioma antes de que se cree la interfaz. Ver el código Ilustración 78.

```
open class BaseActivity : AppCompatActivity() {  
    override fun attachBaseContext(newBase: Context) {  
        val langCode = SharedPrefManager(newBase).getAppLanguage()  
        super.attachBaseContext(  
            if (langCode.isNotEmpty()) LocaleHelper.setLocale(newBase, langCode)  
            else newBase  
        )  
    }  
}
```

Ilustración 78: BaseActivity.kt

- **TermsActivity, AboutActivity y FeedbackActivity:** Permiten al usuario ver información legal y de terceros, conocer más sobre la app y enviar valoraciones.

Tras explicar todas las activities, explicare en detalle los fragments más interesantes:

- **InventoryFragment** donde el usuario podrá gestionar su inventario.
La función principal de esta pantalla es obtener los productos del usuario desde Firestore, los convierte en objetos UserProduct y los guarda en fullItemList con el método `loadItemsFromFirestore()` ver Ilustración 79. Luego aplica los filtros para actualizar la vista mediante la función `applyFilters()` ver Ilustración 80, el usuario puede filtrar mediante menús desplegables por ubicación, categoría, condición o buscar mediante una barra de navegación. Otros aspectos que tiene permitido el usuario, es navegar a productos desperdiciados (`WastedProductsFragment.kt`), ir a su perfil o ajustes, agregar o eliminar productos.

```
private fun loadItemsFromFirestore() {
    val user = FirebaseAuth.getInstance().currentUser ?: return
    firestore.collection("users")
        .document(user.uid)
        .collection("products")
        .get()
        .addOnSuccessListener { snapshot ->
            fullItemList = snapshot.mapNotNull { doc ->
                val name = doc.getString("name") ?: return@mapNotNull null
                val icon = doc.getString("icon") ?: return@mapNotNull null
                val condition = doc.getString("condition") ?: ""
                val iconRes = resources.getIdentifier(icon, "drawable", requireContext().packageName)

                UserProduct(
                    idProduct = doc.id,
                    name = name,
                    icon = icon,
                    category = doc.getString("category") ?: "",
                    condition = condition,
                    location = doc.getString("location") ?: "",
                    idUser = user.email ?: "",
                    quantity = 1
                ).apply { iconDrawableRes = iconRes }
            }
            applyFilters()
        }
        .addOnFailureListener {
            Toast.makeText(requireContext(),
                "Error al cargar inventario", Toast.LENGTH_SHORT).show()
        }
}
```

Ilustración 79: Método loadItemsFromFirestore InventoryFragment.kt

```
private fun applyFilters() {
    val location = spinnerLocation.text.toString()
    val query = searchBar.text.toString().lowercase()

    val filtered = fullItemList.filter { item ->
        (location == "Todos" ||
            (location == "Nevera" && item.location == "nevera") ||
            (location == "Congelador" && item.location == "congelador") ||
            (location == "Despensa" && item.location == "despensa")) &&

            (currentCategoryFilter == "Todos" ||
                (currentCategoryFilter == "Fruta" && item.category == "fruta") ||
                (currentCategoryFilter == "Verdura" && item.category == "verdura")) &&

            (currentConditionFilter == "Todos" ||
                (currentConditionFilter == "Fresco" && item.condition == "fresco") ||
                (currentConditionFilter == "Podrido" && item.condition == "podrido")) &&

            (query.isBlank() || item.name.lowercase().contains(query))
    }

    adapter.updateData(filtered)
    tvProductCount.text = "Se encontraron ${filtered.size} producto(s)."
}
```

Ilustración 80: Método applyFilters de InventoryFragment.kt

- **RecipesFragment** está pantalla muestra las distintas recetas disponibles que tiene el usuario a su disposición.

Las recetas son mostradas a través de un RecyclerView utilizando **RecipesAdapter**. Cada ítem de la lista muestra la imagen, título, y otros detalles breves de la receta. Estas recetas pueden ser filtradas mediante un BottomSheetDialog, los filtros disponibles son tipo de plato, tipo de dieta e ingredientes disponibles del usuario (obtenido desde Firestore y mostrados mediante un RecyclerView). Existe la opción de cargar más recetas y para mejorar la experiencia de usuario se ha agregado una animación de espera

(Shimmer). Otra funcionalidad, es que el usuario puede guardar o quitar recetas de sus favoritos a través de Firestore. Si el usuario selecciona una receta, navega a los detalles de la receta **RecipeDetailsFragment** utilizando el **NavigationComponent** y pasando el **recipeId**. Por último, el usuario puede navegar hacia su lista de recetas favoritas. La función encargada de pedir las recetas a la API a través del **ViewModel** es **requestApiData()**, primero se encarga de que se escuche la respuesta de la API tan solo una vez. Si todo salió de forma correcta, se muestra las recetas en la lista y esconde el **Shimmer**.

```
private fun requestApiData(isLoadMore: Boolean = false) {
    if (!isObserverSet) {
        recipesViewModel.recipesResponse.observe(viewLifecycleOwner) { result ->
            when (result) {
                is NetworkResult.Success -> {
                    binding.shimmerLayout.stopShimmer()
                    binding.shimmerLayout.visibility = View.GONE
                    binding.recipesRecyclerView.visibility = View.VISIBLE
                    binding.swipeRefreshLayout.isRefreshing = false

                    result.data?.let { newRecipes ->
                        if (!isLoadMore) {
                            recipesList.clear()
                        }
                        recipesList.addAll(newRecipes)
                        recipesAdapter.setData(recipesList)
                    }
                }

                is NetworkResult.Error -> {
                    binding.shimmerLayout.stopShimmer()
                    binding.shimmerLayout.visibility = View.GONE
                    binding.recipesRecyclerView.visibility = View.GONE
                    binding.swipeRefreshLayout.isRefreshing = false
                }

                is NetworkResult.Loading -> {
                    binding.shimmerLayout.startShimmer()
                    binding.shimmerLayout.visibility = View.VISIBLE
                    binding.recipesRecyclerView.visibility = View.GONE
                }
            }
        }
        isObserverSet = true
    }

    recipesViewModel.getRecipes(recipesViewModel.applyQueries())
}
```

Ilustración 81: Método requestApiData de RecipesFragment.kt

- **ScannerFragment** es la pantalla que se encarga de la interacción con el modelo de detección de frutas y verduras a través de la cámara del dispositivo. Su responsabilidad es cargar el modelo ONNX, desde los assets de la aplicación. Si los permisos de la cámara están concedidos, se lanza un intent para abrir la cámara del teléfono.

La función principal encargada de escanear la imagen es `processImage()` en las siguientes ilustraciones se puede ver la función Ilustración 82 e Ilustración 83.

Explicaré paso a paso la función:

- Prepara la imagen para el modelo, redimensionándola a 416x416 píxeles, que es el tamaño que el modelo espera. Luego crea un buffer para introducir los píxeles convertidos en números flotantes, normalizados para que el modelo los entienda. Por último, convierte la imagen redimensionada en este buffer usando una función que normaliza y transforma la imagen.
- A continuación, se crea el tensor para el modelo que es la estructura que el modelo utiliza para procesar los datos.
- Luego se ejecuta el modelo pasando el tensor con la imagen al modelo y este devuelve un resultado de muchas predicciones con coordenadas del objeto probabilidad para cada clase.
- Más adelante se filtra y se recoge las detecciones confiables recorriéndolas todas y encontrando la clase con mayor probabilidad que el umbral de confianza 0.5.
- Para finalizar, se muestra los resultados, si no ha habido detecciones se muestra la imagen original sin cambios, mientras que, si se encontraron detecciones, se muestra un dialogo donde el usuario podrá o añadir el producto al inventario con las características obtenidas u omitir.

```

private fun processImage(bitmap: Bitmap) {
    try {
        val resized = Bitmap.createScaledBitmap(bitmap, 416, 416, true)
        val bufferSize = 3 * 416 * 416
        val inputBuffer = FloatBuffer.allocate(bufferSize)
        TensorImageUtils.bitmapToFloatBuffer(resized, 0, 0, 416, 416, FloatArrayOf(0f,0f,0f), FloatArrayOf(1f,1f,1f), inputBuffer, 0)
        inputBuffer.rewind()
        val inputTensor = OnnxTensor.createTensor(OnnxEnv, inputBuffer, LongArrayOf(1, 3, 416, 416))

        val confidenceThreshold = 0.5f // Umbral de confianza

        ortSession?.run(mapOf("images" to inputTensor))?.use { results ->
            val output = results[0].value as Array<Array<FloatArray>>
            val rawOutput = output[0]
            val numClasses = 28
            val numPredictions = 3549
            val detections = mutableListOf<Detection>()
            // (Tu bucle de detección ...)
            for (i in 0 until < numPredictions) {
                val x = rawOutput[0][i]; val y = rawOutput[1][i]; val w = rawOutput[2][i]; val h = rawOutput[3][i]
                var maxClass = -1; var maxScore = -Float.MAX_VALUE
                for (c in 0 until < numClasses) {
                    val classIndexInRawOutput = 4 + c
                    if (classIndexInRawOutput >= rawOutput.size) continue
                    val classScore = rawOutput[classIndexInRawOutput][i]
                    if (classScore > maxScore) { maxScore = classScore; maxClass = c }
                }
                if (maxScore < confidenceThreshold) continue
                if (maxClass !in labels.indices) continue
                val label = labels[maxClass]
                detections.add(Detection(label, maxScore, RectF((x - w/2f) * bitmap.width, (y - h/2f) * bitmap.height, (x + w/2f) * bitmap.width, (y + h/2f) * bitmap.height)))
            }
        }
    }
}

```

Ilustración 82: Método processImage parte 1 de ScannerFragment

```

if (detections.isNotEmpty()) {
    val bitmapWithDetections = showDetectionsOnImage(bitmap, detections) // Devuelve el bitmap con las cajas
    updateScannedImageDisplay(bitmapWithDetections) // Muestra la imagen con las detecciones
    updateTextStatus("Se encontraron ${detections.size} producto(s).", false)
    showMultipleDialogs(detections.sortedByDescending { it.confidence }, index = 0, processedClasses = mutableSetOf())
} else {
    updateScannedImageDisplay(bitmap) // Muestra la imagen original sin detecciones
    updateTextStatus(getString(R.string.no_se_detectaron_productos_reintentelo_de_nuevo), false)
}

} ?: run {
    updateTextStatus(getString(R.string.fallo_en_la_inferencia_del_modelo), false)
    showError(getString(R.string.fallo_en_la_inferencia_del_modelo)) // Dialogo de error
}

inputTensor.close() // Cerrar tensor para liberar recursos

} catch (e: Exception) {
    updateTextStatus(getString(R.string.error_al_procesar_la_imagen), false)
    showError(getString(R.string.error_al_procesar_imagen, e.message)) // Dialogo de error
    // Considerar resetear la imagen a la original o al placeholder en caso de error grave
    updateScannedImageDisplay(bitmap)
} finally {
    // Asegurar que el progressBar se oculte si alguna ruta no lo hizo
    if (binding.progressBar.visibility == View.VISIBLE) {
        updateTextStatus(binding.textStatus.text.toString(), false) // Mantiene el mensaje actual pero oculta el loader
    }
}
}

```

Ilustración 83: Método processImage parte 2 de ScannerFragment

- El usuario puede optar por la añadir los productos de forma manual comenzando en **SelectCategoryFragment**, donde elige entre "fruta" o "verdura". Esta selección lo dirige a **SelectProductFragment**, que muestra una lista de productos básicos (obtenidos de Firestore) correspondientes a la categoría elegida. Al seleccionar un producto específico, se navega a **AddProductFragment**. Aquí, el usuario define la cantidad, la ubicación en el hogar (nevera, congelador, despensa) y la condición del producto (fresco o

podrido) mediante `PowerSpinnerView`. Al guardar, esta información se persiste en la colección `products` del usuario en `Firestore`.

- **RecipeDetailsFragment**, presenta toda la información detallada: imagen, ingredientes (en un `RecyclerView` horizontal), pasos de preparación e información nutricional (cada uno en su propio `RecyclerView`). Ambos fragments permiten guardar o quitar recetas de favoritos, interactuando con la subcolección `savedRecipes` en `Firestore`.
- **SavedRecipesFragment** se trata de la lista de las recetas favoritas del usuario, se obtienen los IDs desde `Firestore` y luego solicitando los detalles completos de estas recetas a la API para su visualización, aplicando también los filtros activos.
- **ShoppingListFragment** permite a los usuarios gestionar su lista de la compra y añadir sus productos de la compra en su inventario. Los ítems se muestran en un `RecyclerView`, con opciones para incrementar/decrementar la cantidad o eliminar productos. La adición de nuevos productos se realiza navegando al **AddShoppingListFragment**, que presenta una lista de productos básicos desde `Firestore` para facilitar la selección. Todas las modificaciones se reflejan en la colección `shopping_list` del usuario en `Firestore`.
- El **WastedProductsFragment** ofrece una visión analítica del desperdicio de alimentos. Consulta la colección `wasted_products` y, utilizando el **WastedProductsViewModel** y la librería **MPAndroidChart**, presenta gráficos de barras y circulares que muestran el desperdicio por periodo (semanal/mensual) y por tipo de producto.
- El **AlertsFragment** está diseñado para centralizar notificaciones importantes. Actualmente, muestra dos tipos de alertas en `RecyclerViews` separados: productos del inventario del usuario que están marcados como "podridos" (obtenidos de la colección `products` del usuario) y productos de temporada (obtenidos de una colección global `basic_products` en `Firestore`, filtrados según el mes actual). El usuario puede optar por eliminar los productos podridos de su inventario o añadir los productos de temporada a su lista de la compra directamente desde estas alertas.

Por último, los **adapters** como `RecipesAdapter.kt` o `UserIngredientsAdapter.kt`, básicamente se han utilizado para rellenar con listas de datos los `RecyclerViews` utilizados en los `Fragments`.

Data Binding permite conectar los datos del ViewModel con las vistas XML de forma automática. En este caso **recipes_row_layout.xml** ver Ilustración 84, representa cada receta con sus valores cargados de la API Spoonacular, asimismo para recibir los datos se ha vinculado al objeto **Result.kt**. Además, he creado un BindingAdapter personalizado **RecipesRowBinding** para cargar las imágenes de cada receta mediante la librería Glide ver Ilustración 85.

```
<data>
    <variable
        name="result"
        type="com.example.smartfreezer.models.Result" />
</data>

<androidx.cardview.widget.CardView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_margin="8dp"
    android:clickable="true"
    android:focusable="true"
    app:cardCornerRadius="8dp"
    app:cardElevation="2dp"
    android:backgroundTint="?attr/colorCardBackground">

    <androidx.constraintlayout.widget.ConstraintLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content">

        <ImageView
            android:id="@+id/recipeImage"
            android:layout_width="match_parent"
            android:layout_height="200dp"
            android:scaleType="centerCrop"
            android:contentDescription="imagen receta"
            app:layout_constraintTop_toTopOf="parent"
            app:layout_constraintStart_toStartOf="parent"
            app:layout_constraintEnd_toEndOf="parent"
            android:layout_marginBottom="8dp"
            app:loadImageFromUrl="@{result.image}" />
```

Ilustración 84: *recipes_row_layout.xml*


```

class RecipesRowBinding {
    companion object {
        @BindingAdapter("loadImageFromUrl")
        @JvmStatic
        fun loadImageFromUrl(imageView: ImageView, imageUrl: String?) {
            imageUrl?.let {
                val lastSlashIndex = it.lastIndexOf("/")
                val dotIndex = it.lastIndexOf(".")

                if (lastSlashIndex != -1 && dotIndex != -1 && dotIndex > lastSlashIndex) {
                    val name = it.substring(lastSlashIndex + 1, dotIndex)
                    val extension = it.substring(dotIndex + 1)

                    val id = name.substringBefore("-")
                    val newUrl = "https://img.spoonacular.com/recipes/${id}-550x370.${extension}"

                    imageView.load(newUrl) {
                        crossfade(600)
                        error(R.drawable.ic_error_placeholder)
                    }
                } else {
                    // URL no válida, cargar como está
                    imageView.load(it) {
                        crossfade(600)
                        error(R.drawable.ic_error_placeholder)
                    }
                }
            }
        }
    }
}

```

Ilustración 85: RecipesRowBinding

4.3.3 ViewModel

Los principales ViewModel implementados son los siguientes:

- **RecipesViewModel** tiene la responsabilidad de las consultas de las recetas desde la API, maneja los filtros de búsqueda y pagina, consulta los detalles de una receta como nutrientes e ingredientes y mantiene los estados de la repuesta de la API.

Con la función `getRecipes()` (datos para **RecipesFragment**) ver Ilustración 86 y `getRecipesDetails()` (datos para **RecipeDetailsFragment**) ver Ilustración 87 se recibe el mapa de parámetros para la API desde la función `applyQueries()` ver Ilustración 88 que selecciona los filtros y otros parámetros obligatorios, y mediante `handleRecipesResponse()` y `handleRecipesDetailsResponse()` se verifica si la respuesta fue exitosa mediante la clase **NetworkResult** ver Ilustración 89. Por último, con la función `updateFilters()` se permite actualizar los filtros seleccionados por el usuario.

```

fun getRecipes(queries: Map<String, String>) {
    _recipesResponse.value = NetworkResult.Loading()

    viewModelScope.launch {
        if (hasInternetConnection()) {
            try {
                val queryString = queries.map { "${it.key}-${it.value}" }.joinToString("&")
                val fullUrl = "https://api.spoonacular.com/recipes/complexSearch?queryString"
                Log.d("API_REQUEST_URL", fullUrl)
                val response = foodRecipesApi.getRecipes(queries)
                _recipesResponse.value = handleRecipesResponse(response)
            } catch (e: Exception) {
                _recipesResponse.value = NetworkResult.Error("Error: ${e.message}")
            }
        } else {
            _recipesResponse.value = NetworkResult.Error("No Internet Connection")
        }
    }
}

private fun handleRecipesResponse(response: Response<FoodRecipe>): NetworkResult<List<Result>> {
    return if (response.isSuccessful) {
        val data = response.body()?.results
        if (data != null && data.isNotEmpty()) {
            NetworkResult.Success(data)
        } else {
            NetworkResult.Error("No recipes found")
        }
    } else {
        NetworkResult.Error("Error: ${response.message()}")
    }
}

```

Ilustración 86: Función getRecipes de RecipesViewModel

```

fun getRecipeDetails(recipeId: Int) {
    _recipeDetailsResponse.value = NetworkResult.Loading()
    viewModelScope.launch {
        if (hasInternetConnection()) {
            try {
                val response = foodRecipesApi.getRecipeDetails(recipeId, API_KEY, true)
                _recipeDetailsResponse.value = handleRecipeDetailsResponse(response)
            } catch (e: Exception) {
                _recipeDetailsResponse.value = NetworkResult.Error("Error: ${e.message}")
            }
        } else {
            _recipeDetailsResponse.value = NetworkResult.Error("No Internet Connection")
        }
    }
}

private fun handleRecipeDetailsResponse(response: Response<RecipeDetails>): NetworkResult<RecipeDetails> {
    return if (response.isSuccessful) {
        response.body()?.let {
            NetworkResult.Success(it)
        } ?: NetworkResult.Error("No se encontraron detalles de la receta")
    } else {
        NetworkResult.Error("Error: ${response.message()}")
    }
}

```

Ilustración 87: Función getRecipeDetails de RecipesViewModel

```

fun applyQueries(): HashMap<String, String> {
    val queries = HashMap<String, String>()
    queries[QUERY_NUMBER] = "20"
    queries[QUERY_API_KEY] = API_KEY
    queries[QUERY_TYPE] = selectedType
    queries[QUERY_DIET] = selectedDiet
    queries[QUERY_INSTRUCTION_REQUIRED] = "true"
    queries[QUERY_ADD_RECIPE_INFORMATION] = "true"
    queries[QUERY_FILL_INGREDIENTS] = "true"
    queries[QUERY_ADD_NUTRITION] = "true"
    queries["offset"] = offset.toString()

    if (selectedIngredients.isNotEmpty()) {
        val ingredients = selectedIngredients.joinToString(",")
        queries["includeIngredients"] = ingredients
    }

    return queries
}

fun updateFilters(type: String, diet: String, rating: Int, ingredients: List<String>) {
    selectedType = type
    selectedDiet = diet
    selectedRating = rating
    selectedIngredients = ingredients
    offset = 0
}

```

Ilustración 88: Función de applyQueries de RecipesViewModel

```

sealed class NetworkResult<T> {
    val data: T? = null,
    val message: String? = null
} {
    class Success<T>(data: T) : NetworkResult<T>(data)
    class Error<T>(message: String?, data: T? = null) : NetworkResult<T>(data, message)
    class Loading<T> : NetworkResult<T>()
}

```

Ilustración 89: Clase NetworkResult

- **WastedProductsViewModel** se encarga de la carga y filtrado de los productos desperdiciados del usuario desde Firestore, muestra los datos semanales o mensuales según el periodo seleccionado. Procesa los datos para agruparlos por día de la semana o mes, y expone LiveData con listas para la UI. Por último, actualiza las fechas en las que se encuentra el usuario consultando.

4.4 Configuración del entorno de desarrollo

Los pasos realizados en contexto a la configuración han sido los siguientes;

1. Se creó un proyecto en Firebase Console
2. La aplicación "SmartFreezer" fue registrada en el proyecto de Firebase, proporcionando el nombre "com.example.smartfreezer"
3. Se agregó el archivo de configuración google-services.json al directorio app/. Este archivo contiene la configuración necesaria para que la aplicación se conecte a los servicios de Firebase.

4. Se añadieron los siguientes plugin y dependencias de Gradle necesarios para trabajar con todas las herramientas anteriormente mencionadas en el proyecto:

```
plugins {  
    alias(libs.plugins.android.application)  
    alias(libs.plugins.kotlin.android)  
    alias(libs.plugins.kotlin.compose)  
    id("com.google.gms.google-services")  
    id("com.google.firebase.crashlytics")  
    id("com.google.firebase.firebase-perf")  
    id("androidx.navigation.safeargs.kotlin")  
    id("com.google.dagger.hilt.android")  
    id("kotlin-kapt")  
}
```

Ilustración 90: Plugins Gradle

```
// Servicios de Firebase (autenticación y Firestore)  
implementation("com.google.firebase:firebase-auth")  
implementation("com.google.firebase:firebase-firestore")  
implementation("com.google.firebase:firebase-messaging:24.1.1")  
implementation("com.google.code.gson:gson:2.10.1")  
  
// Importar la BoM de Firebase para gestionar versiones automáticamente  
implementation(platform("com.google.firebase:firebase-bom:33.13.0"))  
  
// Dependencia de Firebase Crashlytics y Performance Monitoring  
implementation("com.google.firebase:firebase-crashlytics-ktx")  
implementation("com.google.firebase:firebase-perf")  
  
// Menu SmoothBottomBar  
implementation("com.github.ibrahimsn98:SmoothBottomBar:1.7.9")  
// Menu desplegable PowerSpinner  
implementation("com.github.skydoves:powerspinner:1.2.7")  
implementation("com.github.skydoves:powermenu:2.2.4")  
implementation("com.google.android.material:material:1.12.0")  
// Retrofit  
implementation("com.squareup.retrofit2:retrofit:2.9.0")  
implementation("com.squareup.retrofit2:converter-gson:2.9.0")  
// ViewModel + LiveData + Hilt  
implementation("androidx.lifecycle:lifecycle-viewmodel-ktx")  
implementation("androidx.lifecycle:lifecycle-livedata-ktx")  
implementation("androidx.lifecycle:lifecycle-extensions:2.2.0")  
implementation("androidx.hilt:hilt-common:1.2.0")  
implementation("androidx.lifecycle:lifecycle-viewmodel-ktx:2.9.0")  
  
// Grafico  
implementation("com.github.PhilJay:MPAndroidChart:v3.1.0")  
  
// Glide para imágenes  
implementation("com.github.bumptech.glide:glide:4.12.0")  
implementation("com.github.bumptech.glide:compiler:4.12.0")  
// Parsear html  
implementation("org.jsoup:jsoup:1.16.1")
```

```
//Shimmer
implementation("com.facebook.shimmer:shimmer:0.5.0")
//Swipe Refresh Layout
implementation("androidx.swiperefreshlayout:swiperefreshlayout:1.1.0")
//Rounded Image
implementation("com.makeramen:roundedimageview:2.3.0")
//Palette
implementation("androidx.palette:palette-ktx:1.0.0")

implementation("org.pytorch:pytorch_android:1.12.1")
implementation("org.pytorch:pytorch_android_torchvision:1.12.1")

//Timber
implementation("com.jakewharton.timber:timber:5.0.1")
implementation("com.squareup.okhttp3:okhttp:4.11.0")
implementation("com.squareup.okhttp3:logging-interceptor:4.11.0")
implementation("org.jetbrains.kotlinx:kotlinx-coroutines-android:1.7.3")
// Hilt
implementation("com.google.dagger:hilt-android:2.56.1")
implementation(libs.firebase.datatransport)
kapt("com.google.dagger:hilt-android-compiler:2.56.1")
```

Ilustración 91: Dependencias Gradle

5. Firebase se inicializa automáticamente en la aplicación al incluir el archivo `google-services.json`.
6. La clase **MyApplication** inicializa Firebase y configura el idioma y el tema (claro u oscuro) al iniciar la app, según las preferencias del usuario. Usa **SharedPreferences** para recuperar estas preferencias. Además, permite la inyección de dependencias mediante Hilt (**@HiltAndroidApp**), ver Ilustración 92
7. Por último, en **AndroidManifest.xml** declara los permisos de cámara y notificaciones, actividades y servicios utilizados por la app. También configura **MyApplication**, el tema global y especifica **MainActivity** como pantalla de inicio, ver Ilustración 93.

```
@HiltAndroidApp
class MyApplication : Application() {
    override fun onCreate() {
        super.onCreate()
        FirebaseApp.initializeApp(this)

        // Configurar el tema según las preferencias guardadas
        val sharedPrefManager = SharedPrefManager(this)
        if (sharedPrefManager.isDarkModeEnabled()) {
            AppCompatDelegate.setDefaultNightMode(AppCompatDelegate.MODE_NIGHT_YES)
        } else {
            AppCompatDelegate.setDefaultNightMode(AppCompatDelegate.MODE_NIGHT_NO)
        }
    }

    override fun attachBaseContext(base: Context) {
        val sharedPref = SharedPrefManager(base)
        val language = sharedPref.getAppLanguage()
        super.attachBaseContext(LocaleHelper.setLocale(base, language))
    }
}
```

Ilustración 92: MyApplication

```
<uses-feature
    android:name="android.hardware.camera"
    android:required="false" />

<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE" />
<uses-permission android:name="android.permission.CAMERA" />
<uses-permission android:name="android.permission.POST_NOTIFICATIONS" />

<application
    android:name=".MyApplication"
    android:allowBackup="true"
    android:configChanges="locale|layoutDirection|uiMode|screenSize|smallestScreenSize|orientation"
    android:dataExtractionRules="@xml/data_extraction_rules"
    android:fullBackupContent="@xml/backup_rules"
    android:icon="@mipmap/ic_launcher"
    android:label="@string/SmartFreezer"
    android:roundIcon="@mipmap/ic_launcher"
    android:supportRtl="true"
    android:theme="@style/Theme.SmartFreezer"
    tools:targetApi="31">

    <service
        android:name=".util.MyFirebaseMessagingService"
        android:exported="false">
        <intent-filter>
            <action android:name="com.google.firebase.MESSAGING_EVENT" />
        </intent-filter>
    </service>
```

Ilustración 93: AndroidManifest.xml

4.5 Diagrama de la arquitectura de la aplicación

En este diagrama se muestra como está compuesta la aplicación según el patrón arquitectónico MVVM ver Ilustración 94.

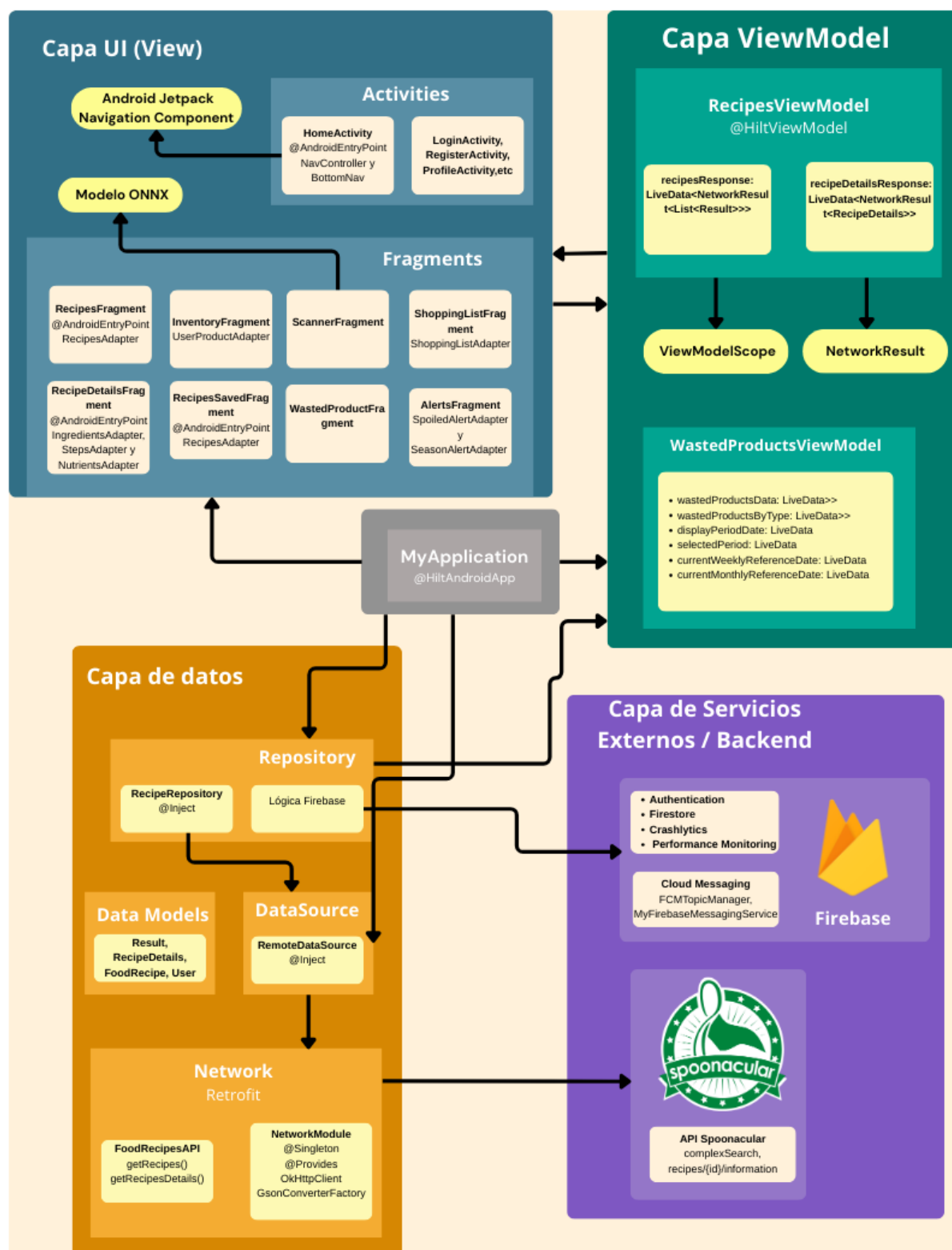


Ilustración 94: Diagrama arquitectura Smartfreezer

5. Pruebas y Evaluación de la Aplicación

Una vez completada la fase desarrollo principal del proyecto “SmartFreezer”, pasaré a explicar el proceso de pruebas y evaluación que ha sido a cabo. Se evaluará características como calidad, funcionalidad, usabilidad y rendimiento de la aplicación. Dividiré el capítulo en dos etapas la evaluación del modelo y de la aplicación.

5.1 Pruebas del Modelo de Detección

En este apartado se evaluará el desempeño del modelo dentro del **ScannerFragment**

5.1.1 Pruebas de Precisión y Rendimiento

Metodología: Se realizaron varias pruebas utilizando un conjunto de imágenes nuevas de frutas y verduras que el modelo no ha entrenado. Estas imágenes se han capturado directamente con la cámara del dispositivo en diferentes condiciones de iluminación, ángulos y distancias. Se compararon los resultados obtenidos de las detecciones con la realidad observada.

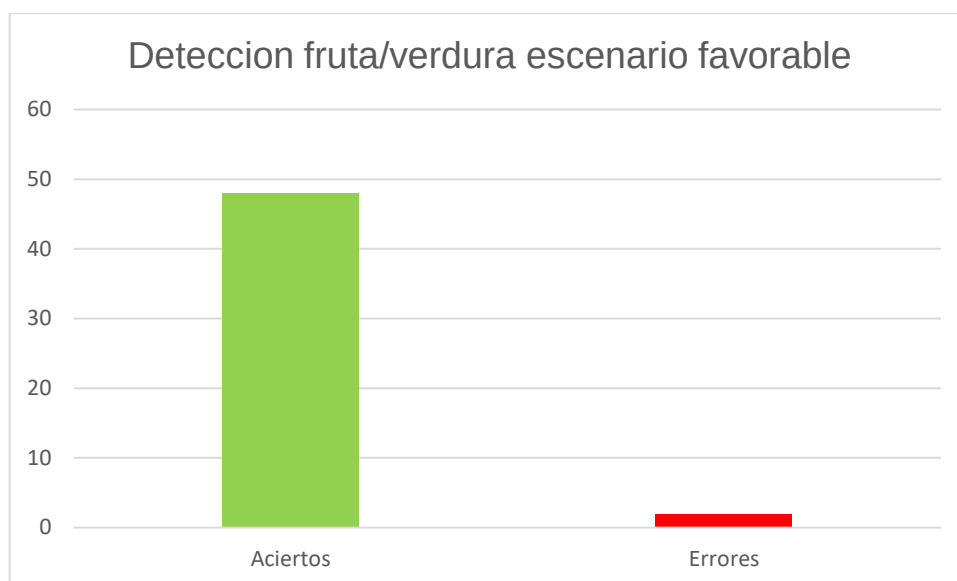
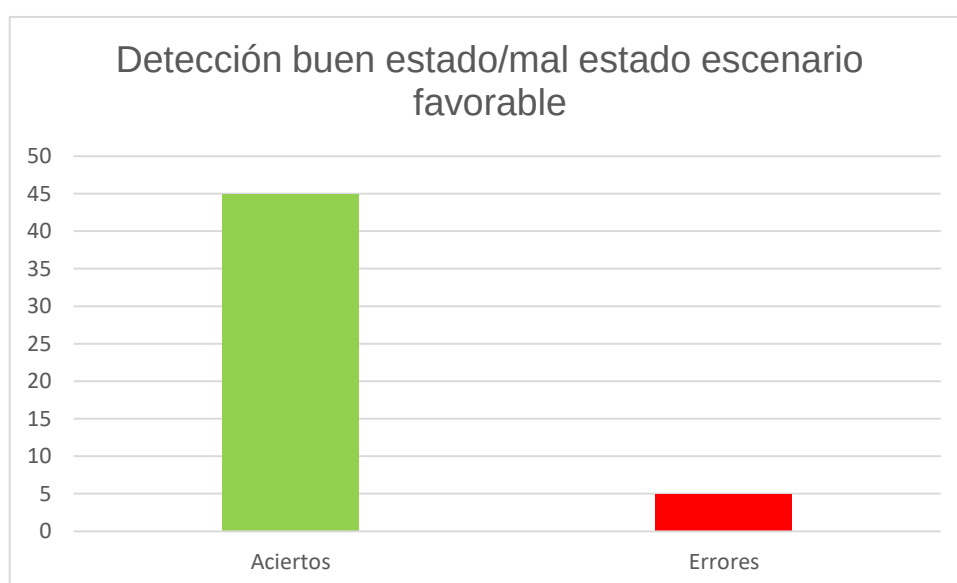
Resultados esperados: Se verifico si la precisión obtenida del entrenamiento del modelo se mantenía estable al integrarse a la aplicación móvil. Se prestaría atención a la tasa de verdaderos positivos, falsos positivos y falsos negativos para las clases.

Métricas:

- Porcentaje de detecciones correctas de tipo fruta o verdura.
- Porcentaje de clasificaciones correctas del estado del producto.
- Identificación de casos donde el modelo fallo recurrentemente.

Resultados obtenidos:

Para el escaneo de imágenes en escenarios favorables con buena iluminación, hemos obtenido un porcentaje de aciertos para ambos del 90% donde la mayoría de los casos se obtenía del escaneo un porcentaje de confianza mayor del 0.85, las siguientes graficas muestran los resultados obtenidos en este escenario:

*Ilustración 95: Resultados detección 1 escenario favorable**Ilustración 96: Resultados detección 2 escenario favorable 2*

Mientras que el escaneo de imágenes en escenarios no favorables con poca iluminación e imágenes borrosas, hemos obtenido un porcentaje de aciertos menor al escenario favorable ya que el porcentaje de aciertos ha sido de aproximadamente del 70, las siguientes graficas muestran los resultados obtenidos en este escenario:

:

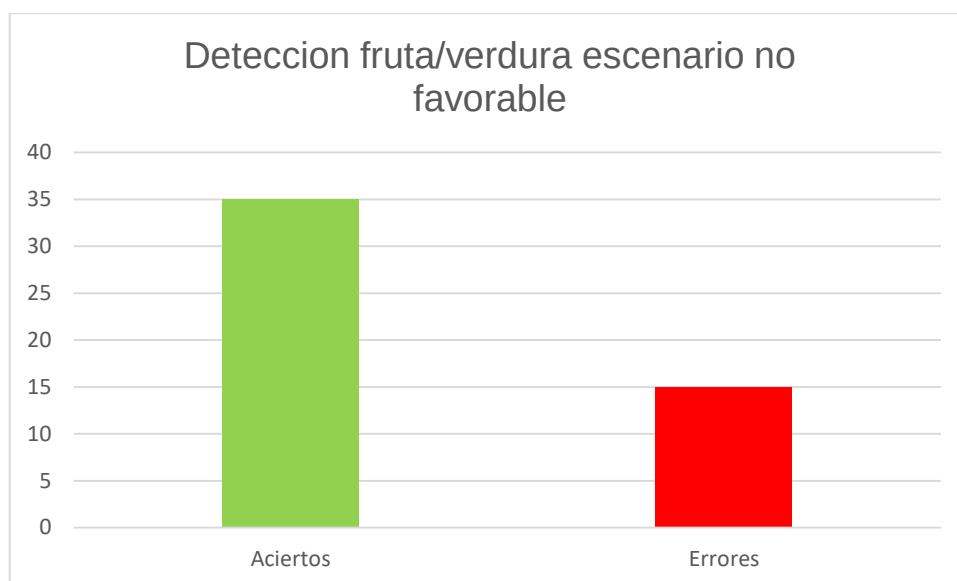


Ilustración 97: Resultados detección escenario no favorable 1

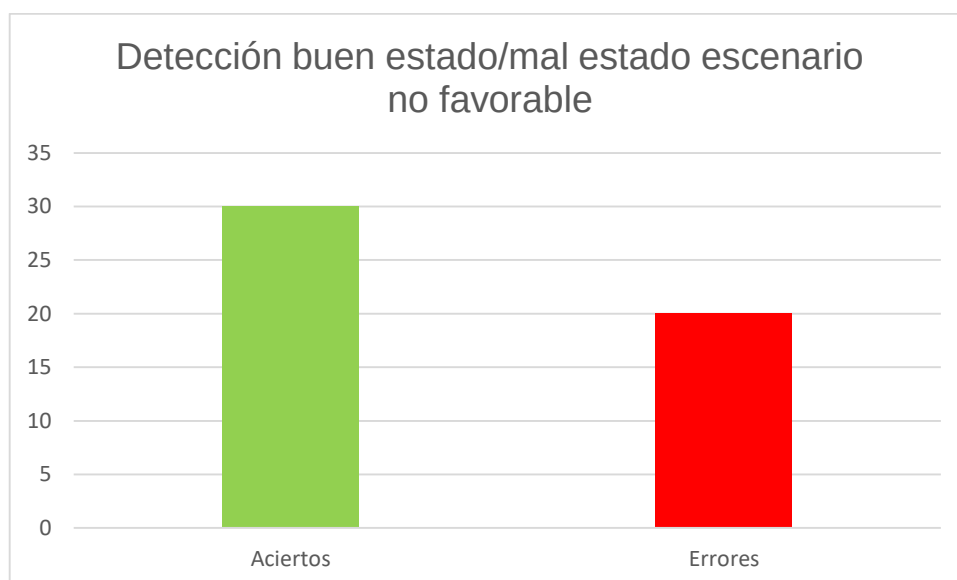


Ilustración 98: Resultados detección escenario no favorable 2

En conclusión, el modelo suele ser sensible al escenario donde se ejecuta el escaneo debido a que en la mayoría de los casos no detecta ningún objeto en la imagen. En cuanto a productos podridos, el modelo solo ha confundido de clase en frutas como la granada podrida que tiene proporciones y colores similares a la manzana.

5.1.2 Pruebas de Tiempo Medio de Escaneo

Metodología: Se midió el tiempo transcurrido desde que se captura una imagen con la cámara hasta que el **ScannerFragment** muestra los resultados de la detección.

Resultados esperados: Se busca un tiempo de respuesta rápido con el objetivo de ofrecer al usuario una experiencia de escaneo fluida.

Resultados obtenidos: Casi no se puede percibir el tiempo de respuesta del modelo ya que cuando el usuario fotografía el producto, el modelo tarda menos de 400ms en procesar la imagen. Por tanto, hemos logrado unos tiempos bastante buenos para la detección de imágenes.

5.2 Pruebas de Funcionalidad de la Aplicación

Esta fase se centró en verificar que todas las funcionalidades implementadas en "SmartFreezer" funcionan según lo esperado y cumplen con los requisitos funcionales definidos.

Verificación de Cada Funcionalidad:

- **Autenticación:** Se probaron los flujos de registro de nuevos usuarios (incluyendo el envío y la lógica de verificación de email), inicio de sesión con credenciales válidas e inválidas, cierre de sesión, recuperación de contraseña y eliminación de cuenta.
- **Gestión de Inventario:**
 - Escaneo de productos: Funcionamiento del **ScannerFragment**, precisión de la detección en la app, adición correcta al inventario desde el escáner.
 - Adición manual: Flujo completo desde **SelectCategoryFragment** hasta **AddProductFragment**, guardado correcto en Firestore.
 - Visualización y edición/eliminación de productos en **InventoryFragment**.
- **Recomendación de Recetas:**
 - Conexión con Spoonacular API: Obtención de listas de recetas y detalles.
 - Filtrado en **RecipesFragment**: Por tipo, dieta e ingredientes del usuario.
 - Visualización en **RecipeDetailsFragment**: Correcta presentación de ingredientes, pasos, información nutricional.
 - Guardado y visualización de recetas favoritas (**SavedRecipesFragment**).

- **Lista de la Compra:** Creación, incorporación de productos (**AddShoppingListFragment**), añadir lista de la compra al inventario y eliminación en **ShoppingListFragment**.
- **Estadísticas de Desperdicio:** Correcto registro de productos en **wasted_products** (especialmente desde el **ScannerFragment** si se detecta "podrido") y visualización de gráficos en **WastedProductsFragment** (**BarChart**, **PieChart**) con datos semanales/mensuales.
- **Gestión de Perfil:** Modificación de nombre, email, contraseña y preferencias dietéticas en **ProfileActivity**, y su correcta persistencia y efecto.
- **Notificaciones:** Envío y recepción de notificaciones FCM y correcta navegación al pulsar la notificación (**handleIntentExtras** en **HomeActivity**).
- **Feedback de Usuarios:** Envío y almacenamiento correcto de valoraciones y comentarios en **Firestore**.

5.3 Pruebas de Aceptación

En estas pruebas es el propio usuario quien evaluará la aplicación en cuestiones de usabilidad y experiencia de usuario.

Metodología: Se realizó pruebas con un pequeño grupo de usuarios, donde se le indicaba que debía realizar dentro de la aplicación. Tras ello, los usuarios podrían evaluar el comportamiento y aspecto de la aplicación mediante un cuestionario encontrado en **FeedbackFragment**.

En este cuestionario el usuario tenía que responder las preguntas valorándolas del 1 al 5, siendo el 5 la mejor valoración, o escribir su respuesta:

- ¿Te resultó fácil usar la app?
- ¿Te gustó el diseño de la interfaz?
- ¿La app funcionó sin errores ni bloqueos?
- ¿Te parece útil la app para tus necesidades?
- ¿La presentación de la información es clara?
- Describe cualquier error que hayas encontrado en la aplicación
- Escribe alguna sugerencia de mejora para implementar en un futuro
- Valoración general de la aplicación

Resultados obtenidos: A través de cada respuesta, se concluye lo siguiente

- Facilidad de uso: 4.5 estrellas
- Diseño de la interfaz: 5 estrellas
- Estabilidad: 5 estrellas
- Claridad de la información: 4.5 estrellas
- Utilidad: 4 estrellas
- Valoración general: 4.5 estrellas

Ningún usuario reportó algún error sufrido tras la navegación de la aplicación.

Muchos de los usuarios recomendaron mejoras que requieren de mucho tiempo de desarrollo o incluso cuestiones en relación al sistema de recetas que solo tiene soporte en inglés. Por otro lado, un usuario reportó la posibilidad de añadir los productos de su lista de compra al inventario, por tanto, esta tarea se llevó a cabo y se lanzó una nueva versión.

5.4 Pruebas No Funcionales de la Aplicación

Se comprobarán los requisitos no funcionales establecidos previamente para la aplicación midiendo el rendimiento, la fluidez de la interfaz, tiempos de respuesta y robustez ante errores. A continuación, se describen los resultados:

Prueba de mantenibilidad

Se comprobó que la arquitectura MVVM, junto con Hilt y módulos separados (Repository, ViewModel, servicios, utilidades), permitía la fácil depuración de los datos, gracias a los logs de Crashlytics, cualquier excepción inesperada queda reflejada, así permitiendo modificar la clase afectada sin modificar otras capas. Por otro lado, gracias a esta estructura la escalabilidad en la aplicación será sencilla debido a la limpia organización del software.

Prueba de seguridad

Se cumple las políticas de seguridad de autenticación de usuarios obteniendo tiempos de respuestas rápidos en los accesos a Firestore.

Prueba de rendimiento

Se evaluó el tiempo de respuesta en que RecipesFragment obtiene la información desde la API remota. Además, se comprobó el tiempo en que el modelo tarda en procesar la imagen, estas métricas se capturan desde Firebase Performance Monitoring, añadiendo trazas a las funciones principales; se inicia la traza con **start()** y con **stop()** se finaliza. Con Crashlytics, mostramos si se ha lanzado una excepción dentro de la función, por ejemplo, en **ScannerFragment** se observa la forma de añadir las métricas ver Ilustración 99.

```
private fun processImage(bitmap: Bitmap) {  
    val trace = FirebasePerformance.getInstance().newTrace("image_processing")  
    trace.start()  
  
    try {  
        // Código de la función...  
        if (detections.isNotEmpty()) {  
            trace.incrementMetric("detected_products", detections.size.toLong())  
        }  
    } catch (e: Exception) {  
        FirebaseCrashlytics.getInstance().recordException(e)  
    } finally {  
        trace.stop()  
    }  
}
```

Ilustración 99: Métricas de rendimiento para ScannerFragment

En la siguiente sacada de la consola de Firebase ver Ilustración 100, revela que las métricas donde los datos de red y UI, como el proceso de escaneo; se han obtenido unos tiempos óptimos que cumple el objetivo del mínimo retardo perceptible por el usuario. Esta herramienta es muy útil ya que nos sirve para monitorear todo el rendimiento de la app.



Ilustración 100: Firebase Performance Monitoring

Por último, con la herramienta de Crashlytics podremos identificar la excepción. La consola de Firebase nos muestra todo tipo de información, además con la

implementación de IA nos explica el problema del código y como arreglarlo ver Ilustración 101.

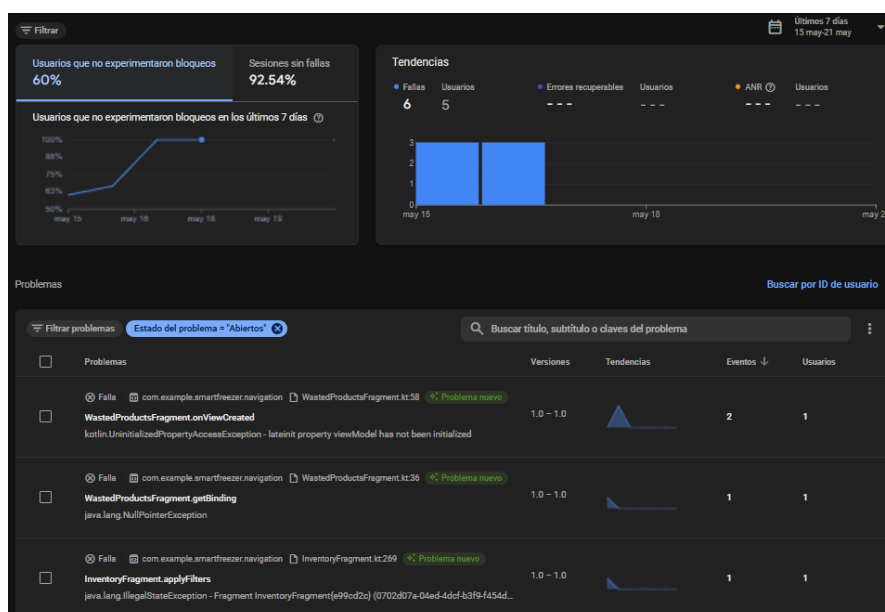


Ilustración 101: Consola Crashlytics

Prueba de accesibilidad

Se probó el cambio al tema oscuro y la configuración de idioma con resultado claramente perceptibles en toda la interfaz, sin embargo, al trabajar con API externa remota, no se ha podido configurar el idioma de las recetas.

Prueba de Usabilidad

Mediante pruebas empíricas se ha conseguido un diseño responsive, gracias a la combinación de **ConstraintLayout** y el uso de tamaños dp y sp. Además, los distintos mensajes de error, los efectos visuales de carga de datos y el efecto de pulsación de los elementos táctiles tiene como resultado garantizar una aplicación intuitiva.

Prueba de portabilidad

La aplicación esta soportada en la mayoría de las versiones Android y en diversos tamaños de pantalla. El uso de themes/ garantiza la apariencia global de la aplicación.

6. Resultados y discusión

6.1 Cumplimiento de objetivos

Una vez desarrollado y probado el producto software, vemos que el proyecto alcanzó los objetivos generales de gestión de inventario y concienciación de la reducción del desperdicio alimentario mediante un control y seguimientos de los productos que el usuario dispone en su hogar. Se implementaron los requisitos relacionados a la autenticación segura, almacenamiento en Firestore, integración con la API de Spoonacular para las recetas, escaneo de productos, estadísticas de desperdicio y listado de la compra. Cada una de estas funcionalidades fueron verificadas mediante pruebas de funcionalidad y rendimiento, demostrando que cumplen con los plazos de respuesta y criterios de usabilidad definidos.

6.2 Presentación de la Aplicación Final

SmartFreezer ofrece las siguientes funcionalidades:

- Listado de la compra, con posibilidad de gestionar y marcar los productos como comprados para asimismo agregarse al inventario.
- Inventario interactivo, con filtros de búsqueda y por características como ubicación, categoría y condición. Además, de mostrar cada producto y modificar sus características.
- Recomendación de recetas, con la total disposición de filtrar por ingredientes disponibles del usuario, por tipo de dieta (vegetariana, vegana, sin lactosa y sin gluten) y por tipo de plato (postre, bebida, ensalada y plato principal). Además, se ofrece los detalles de cada receta como descripción, ingredientes utilizados, detalle nutricionales e instrucciones. El usuario tiene la posibilidad de guardar sus recetas a favoritos.
- Estadísticas de desperdicio, mostrando el desperdicio en un gráfico de barras o circular tanto en un periodo semanal como mensual.
- Escaneo y registro de productos mediante un modelo detección y clasificación de objetos.
- Perfil de usuario, ajustes de idioma y tema oscuro/claro.

- Alertas sobre un producto en temporada o un producto en mal estado con posibilidad de eliminar el producto podrido del inventario o de agregar el producto en temporada a la lista de la compra.
- Notificaciones con sugerencias de recetas según las preferencias dietéticas del usuario.

6.3 Discusión de los resultados obtenidos

Las fortalezas que nos ofrece la aplicación son las siguientes:

- Arquitectura limpia (MVVM + Hilt) que facilita la mantenibilidad y extensión del programa
- Monitoreo de rendimiento y errores en tiempo real a través de la consola de Firebase
- Interfaz accesible, fluida e intuitiva.

Limitaciones:

- Dependencia total de conexión a Internet para recetas y consultas de datos (Firestore).
- Sistema API de recetas con tan solo soporte inglés
- Poca precisión en algunas clases del modelo donde se necesitaba un entrenamiento con más datos de entrada (Guava, Grape, etc).

Problemas y Soluciones

A lo largo del desarrollo surgieron varios retos donde se tuvo que escoger diversas estrategias:

- Problemas CRUD en Firestore (crear/leer(actualizar/borrar) que ocurren al mismo tiempo en distintas partes de la app, por ejemplo, para añadir un producto de la lista de la compra al inventario antes debo comprobar si existe un producto de mismo nombre para evitar duplicados.

Solución: Se integró una consulta que comprobaba el nombre del producto. Si el producto ya existía, se actualiza el campo de cantidad y si no existía, se creaba el documento.

- Al implementar la funcionalidad multilenguaje con **recreate()** en **SettingActivity** cada cambio de idioma forzaba a reiniciar toda la aplicación, perdiendo la navegación actual y bloqueando el estado del ajuste de tema claro/oscuro. Además, al alternar entre claro y oscuro una vez reiniciada la aplicación, el idioma se cambiaba automáticamente. También se recibía problema de excepción debido que cambiar el tema claro/oscuro reiniciaba la actividad y recrea los fragmentos. Por tanto, si me encontraba en una llamada a Firestore, la respuesta llegaba cuando el fragmento ya estaba destruido y recreado.

Solución: Se evitó llamar a **recreate()** en el switch de cambio de tema y se creó **BaseActivity** con la que las demás actividades heredan de ella, en donde el método **attachBaseContext()** lee el idioma de **SharedPrefManager** y aplica la configuración antes de cargar cada Activity, de modo que cualquier recreación conserve la Locale con el idioma seleccionado. Además, en los Fragments se aplicó técnicas de manejo seguro del ciclo de vida como verificar propiedades antes de usar vistas o declarar variables de view binding como nullable.

- Durante el entrenamiento del modelo surgió el problema de requerir anotaciones con bounding boxes de cada imagen. Hacer estas anotaciones manualmente resulta muy laborioso con miles de imágenes.

Solución: Se desarrolló el script en Python **segment_generator.py** con el que mediante la librería de OpenCV se segmentaba automáticamente cada objeto y se extraía sus contornos a través de leer cada imagen en escala de grises y aplicar desenfoque y umbralización adaptativa. Como resultado se obtenía el archivo JSON en formato COCO con cada anotación de cada imagen.

7. Conclusiones y Trabajo futuro

El proyecto da lugar a una solución móvil completa y práctica para ayudar a las personas en la gestión de sus alimentos en casa. Combina funciones como el escaneo de productos, control de inventario, lista de la compra, alertas, notificaciones y recetas personalizadas, todo en una misma aplicación. Gracias al uso de una arquitectura moderna y las herramientas de Firebase, el desarrollo fue ágil, seguro y fácil de mantener, consiguiendo un buen rendimiento y una experiencia fluida para el usuario.

A nivel personal, el proyecto demostró un total desafío debido a que tanto como aprender el lenguaje Kotlin y entender el mundo de las redes CNN ha implicado una curva de aprendizaje bastante pronunciado, no es así, que ha sido un camino muy interesante aprender sobre estas tecnologías y poder aplicar conocimientos aprendidos durante la carrera a este trabajo. Conocimientos como la metodología del software, el diseño de las interfaces de usuario y los diversos algoritmos de tratamiento digital de imágenes han sido presentes en este proyecto. La adopción de una metodología ágil ha sido vital en la planificación y en el desarrollo ya que la presencia de errores e inconsistencias han estado presentes durante gran parte del camino, donde en muchas etapas se ha tenido que borrar todo e ingeniarse algo totalmente nuevo. Me ha resultado muy reconfortante como para cada pequeño problema se ha logrado aplicar una solución efectiva. Otro punto de interés ha sido Firebase ya que ha sido un descubrimiento muy satisfactorio al comprobar cómo esta herramienta facilita tanto el sistema backend como la depuración de la aplicación. Por último, este proyecto me ha ayudado a descubrir mi alcance para aprender y aplicar nuevas tecnologías, esto ha generado en mí una motivación para continuar mi camino como futuro ingeniero informático.

7.1 Líneas de Trabajo Futuro

Habiendo cumplido los objetivos previstos para la aplicación, estas mejoras tan solo significarían una mejora del producto en un futuro. Es por ello, que las mejoras que me gustaría implementar en la aplicación significarían un tiempo de desarrollo significativo. Me gustaría añadir:

Optimizar el acceso a los datos las recetas mediante Caché:

Aprovechando la arquitectura MVVM y el Patrón Repositorio implementado en **RecipeRepository**, se podría implementar una mejora significativa introduciendo una capa de caché para los datos obtenidos a través de la API externa de Spoonacular. Actualmente, cada vez que se solicitan recetas o sus detalles, se realiza una llamada de red. Esta estrategia se podría implementar utilizando una base de datos local como Room. Es entonces que **RecipeRepository** podría obtener los datos de la caché local, y si no estuvieran esos datos disponibles, entonces se realizaría la llamada a la API. Esta técnica de base de datos local también se podría aprovechar para los datos obtenidos de Firestore, ofreciendo una aplicación con funcionalidad offline.

Otra mejora sugerida por un usuario fue implementar la funcionalidad de que, a partir de una receta seleccionada por el usuario, el sistema pudiera analizar automáticamente que ingredientes de la receta ya posee y cuales le hacen falta.

Por último, la mejora más ambiciosa es poder trabajar con cualquier producto de supermercado, transformando “SmartFreezer” a un gestor de alimentos para el hogar. Para ello, es claro que se necesitaría de mucho hardware y un gran dataset con muchas imágenes y anotaciones de productos de un supermercado.

Bibliografía

1. Ministerio de Agricultura, Pesca y Alimentación. [En línea] [Citado el: 15 de 05 de 25.] https://www.mapa.gob.es/gl/alimentacion/temas/desperdicio/informedesperdiciohogares2022/anualfinal_tcm37-659643.pdf.
2. Firebase. [En línea] [Citado el: 15 de 05 de 25.] <https://firebase.google.com/products/firestore?hl=es-419>.
3. Spoonacular. [En línea] [Citado el: 15 de 05 de 25.] <https://spoonacular.com/food-api>.
4. SCRUM. [En línea] [Citado el: 15 de 05 de 25.] <https://scrumguides.org/>.
5. Figma. [En línea] [Citado el: 15 de 05 de 25.] <https://www.figma.com/es-es/>.
6. Xataka. [En línea] [Citado el: 15 de 05 de 25.] <https://www.xataka.com/basics/que-github-que-que-le-ofrece-a-desarrolladores>.
7. Developers Android. [En línea] [Citado el: 15 de 05 de 25.] <https://developer.android.com/studio/intro?hl=es-419>.
8. KeepCooding. [En línea] [Citado el: 16 de 05 de 25.] <https://keepcoding.io/blog/que-es-pycharm-y-como-utilizarlo/>.
9. AWS Amazon. [En línea] [Citado el: 16 de 05 de 25.] <https://aws.amazon.com/es/what-is/restful-api/>.
10. UNEP. [En línea] [Citado el: 16 de 05 de 25.] <https://www.unep.org/news-and-stories/press-release/un-17-all-food-available-consumer-levels-wasted>.
11. FAO. [En línea] [Citado el: 16 de 05 de 25.] https://www.fao.org/investment-centre/latest/news/detail/New-UN-Report-finds-828-million-people-going-hungry-in-a-world-of-plenty/?utm_source=chatgpt.com.
12. Too Good To Go. [En línea] [Citado el:] <https://www.toogoodtogo.com/es/blog/reduce-desperdicio-alimentos-too-good-to-go>.
13. Samsung Newsroom. [En línea] [Citado el: 16 de 05 de 25.] <https://news.samsung.com/co/con-tecnologia-puedes-reducir-el-desperdicio-de-alimentos-en-el-hogar>.

14. Codemotion. [En línea] [Citado el: 17 de 05 de 25.] <https://www.codemotion.com/magazine/es/inteligencia-artificial/deep-learning-y-redes-neuronales-una-guia-completa>.
15. Plain Concepts. [En línea] [Citado el: 17 de 05 de 25.] <https://www.plainconcepts.com/es/convolutional-neural-network-guia/>.
16. Madrid, Universidad Politecnica de. [En línea] [Citado el: 17 de 05 de 25.] https://dcain.etsin.upm.es/~carlos/bookAA/05.7_RRNN_Convoluciones_CIFAR_10_INFORMATIVO.html.
17. Ultralytics. [En línea] [Citado el: 17 de 05 de 25.] <https://docs.ultralytics.com/es/models/yolov8>.
18. Roboflow. [En línea] [Citado el: 17 de 05 de 25.] <https://blog.roboflow.com/what-is-yolov8/>.
19. Developer Android. [En línea] [Citado el: 18 de 05 de 25.] <https://developer.android.com/kotlin?hl=es-419>.
20. Scoreapps. [En línea] [Citado el: 18 de 05 de 25.] <https://scoreapps.com/blog/android-studio/>.
21. Developer Android. [En línea] [Citado el: 18 de 05 de 25.] <https://developer.android.com/studio/projects?hl=es-419>.
22. Conasa. [En línea] [Citado el: 18 de 05 de 25.] <https://conasa.grupocibernos.com/blog/ventajas-del-backend-as-a-service-baas>.
23. Firebase. [En línea] [Citado el: 18 de 05 de 25.] <https://firebase.google.com/docs/auth?hl=es-419>.
24. Firebase. [En línea] [Citado el: 18 de 05 de 25.] <https://firebase.google.com/docs/firestore/rtdb-vs-firestore?hl=es-419>.
25. Firebase. [En línea] [Citado el: 18 de 05 de 25.] <https://firebase.google.com/docs/cloud-messaging?hl=es-419>.
26. Firebase. [En línea] [Citado el: 18 de 05 de 25.] <https://firebase.google.com/docs/crashlytics?hl=es-419>.
27. Firebase. [En línea] [Citado el: 18 de 05 de 25.] <https://firebase.google.com/docs/perf-mon?hl=es-419>.

28. Kaggle. [En línea] [Citado el: 19 de 05 de 25.] <https://www.kaggle.com/code/osamaabobakr/fruit-and-vegetable-disease-healthy-vs-rotten/input>.
29. Aprender Machine Learning. [En línea] [Citado el: 19 de 05 de 25.] <https://www.aprendemachinelarning.com/deteccion-de-objetos-con-python-yolo-keras-tutorial/>.
30. Toolify. [En línea] [Citado el: 19 de 05 de 25.] <https://www.toolify.ai/es/ai-news-es/todo-sobre-anotaciones-de-imagenes-1119620>.
31. Ultralytics. [En línea] [Citado el: 19 de 05 de 25.] <https://docs.ultralytics.com/es/datasets/detect/#ultralytics-yolo-format>.
32. GitHub Ultralytics. [En línea] [Citado el: 20 de 05 de 25.] <https://github.com/ultralytics/ultralytics/issues/5800>.
33. StackOverFlow. [En línea] [Citado el: 20 de 05 de 25.] <https://stackoverflow.com/questions/75927857/how-to-convert-coco-json-to-yolov8-segmentation-format>.
34. NVIDIA. [En línea] [Citado el: 20 de 05 de 25.] https://developer.nvidia.com/cuda-downloads?target_os=Windows.
35. Pytorch. [En línea] [Citado el: 21 de 05 de 25.] <https://pytorch.org/get-started/locally/>.
36. StackOverFlow. [En línea] [Citado el: 21 de 05 de 25.] <https://stackoverflow.com/questions/20554074/sklearn-omp-error-15-initializing-libiomp5md-dll-but-found-mk2iomp5md-dll-a>.
37. Innovatiana. [En línea] [Citado el: 22 de 05 de 25.] <https://es.innovatiana.com/post/understand-confusion-matrix-in-ai>.
38. Ultralytics. [En línea] [Citado el: 22 de 05 de 25.] <https://docs.ultralytics.com/es/modes/export/#usage-examples>.
39. OpenWebinars. [En línea] [Citado el: 22 de 05 de 25.] <https://openwebinars.net/blog/python-en-ia-aplicaciones-practicas-y-bibliotecas-clave/>.

Anexo A

Manual de Usuario

A.1 Descripción

En este anexo se presentan las diversas funcionalidades de *SmartFreezer*, acompañadas de una breve explicación de cada una. Este anexo tiene como objetivo ser una guía práctica para el usuario, facilitando la comprensión y uso de la aplicación.

A.2 Manual de usuario SmartFreezer

Pantalla de inicio donde el usuario selecciona el botón vamos allá para comenzar a interactuar con la aplicación ver Ilustración 102.

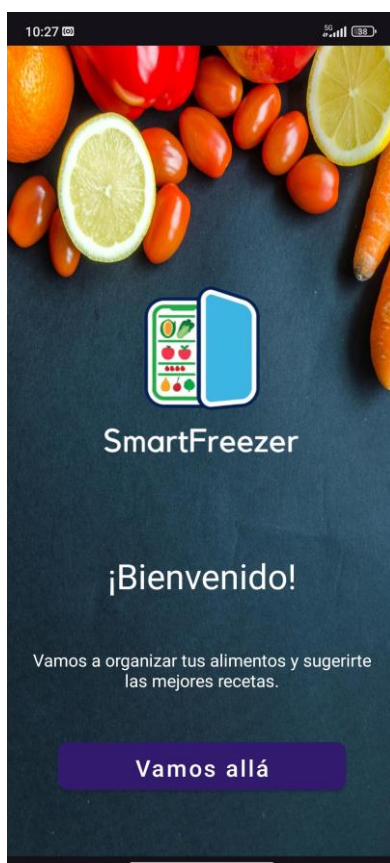


Ilustración 102: Pantalla inicio

Pantalla donde el usuario inicia sesión, si las credenciales son incorrectas o el usuario no ha verificado su correo electrónico se informará a través de mensajes de error ver

Ilustración 103. Además, el usuario podrá restablecer su contraseña en caso de olvido ver Ilustración 104.



Ilustración 103: Pantalla inicio sesión



Ilustración 104: Dialogo restablecer contraseña

Pantalla donde el usuario podrá registrar un usuario en la aplicación, debiendo antes aceptar los términos y condiciones y verificar su correo electrónico ver Ilustración 105.

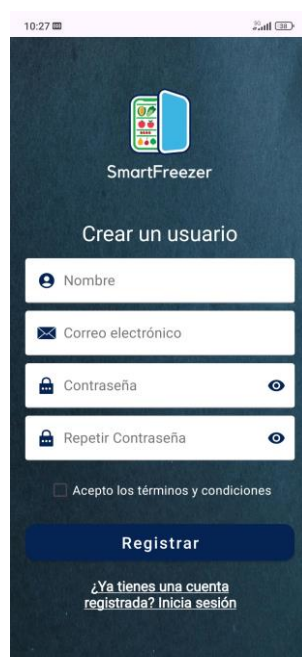


Ilustración 105: Pantalla registrar

Pantalla principal donde el usuario visualiza sus productos en el inventario, pudiendo agregar, eliminar y modificar productos, además de filtrar por su ubicación y por sus características ver Ilustración 106.



Ilustración 106: Pantalla inventario

Si el usuario desea agregar un producto tan solo debe seleccionar el botón flotante (+) de la pantalla inventario, y de esa forma accederá a la pantalla de selección de categoría ver Ilustración 107, que una vez escogida la categoría el sistema transportará al usuario a seleccionar un producto ver Ilustración 108, y finalmente, tras seleccionar el producto, se revela el formulario para modificar las características deseadas para la adición del producto ver Ilustración 109. El resultado del proceso se muestra en la pantalla inventario

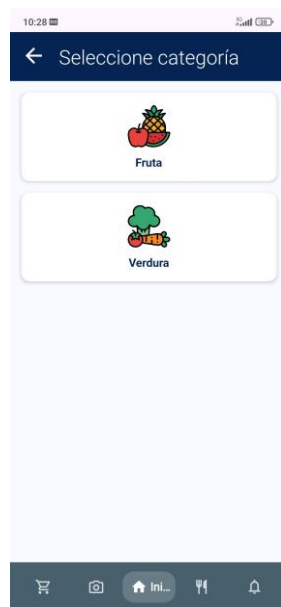


Ilustración 107: Pantalla seleccionar categoría



Ilustración 108: Pantalla seleccionar producto inventario

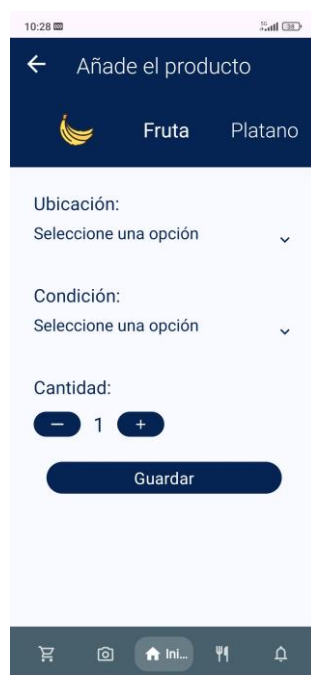


Ilustración 109: Pantalla añadir producto al inventario



Ilustración 110: Pantalla inventario con producto

Para modificar las características de los productos del inventario, se debe pulsar pulsado encima de la casilla del producto para revelar una ventana flotante sobre el producto ver Ilustración 111, si el usuario decide mantener pulsado la casilla se abrirá un dialogo para eliminar el producto del inventario ver.



Ilustración 111: Pantalla ventana producto del inventario



Ilustración 112: Pantalla dialogo para eliminar producto del inventario

Si el usuario decidiera pulsar a Historial desde la pantalla Inventario, el sistema navegaría hacia la pantalla donde el usuario puede consultar sus desperdicios tanto

semanales como mensuales, a través de un gráfico de barras ver Ilustración 113 o un gráfico circular ver Ilustración 114.



Ilustración 113: Pantalla desperdicios gráfico de barras



Ilustración 114: Pantalla desperdicio gráfico circular

Pantalla donde el usuario gestiona su lista de la compra ver Ilustración 115, para agregar producto el usuario deberá pulsar el botón flotante (+) donde el sistema navegará al usuario a la pantalla de seleccionar producto para añadir en la lista de la compra ver Ilustración 116. Una vez añadidos los productos el usuario podrá modificar la cantidad de producto o eliminarlos ver Ilustración 117 , el usuario tiene la posibilidad de transferir esos productos al inventario a través del botón “Añadir lista al inventario” donde aparecerá un diálogo de confirmación.

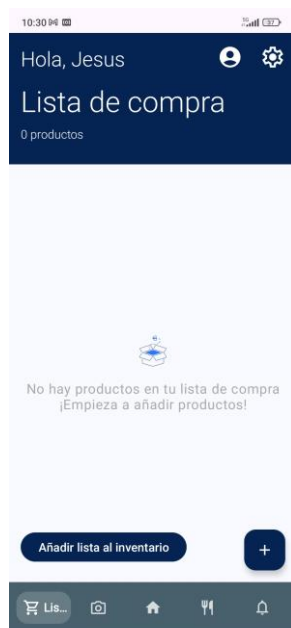


Ilustración 115: Pantalla lista de la compra vacía



Ilustración 116: Pantalla seleccionar producto lista de compra

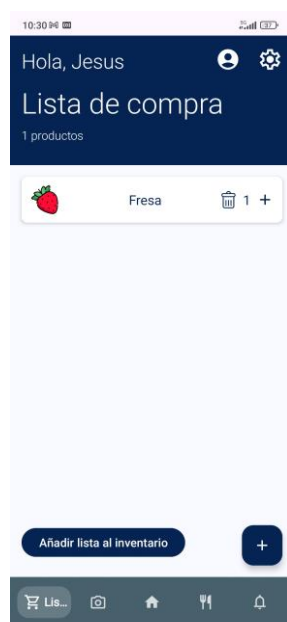


Ilustración 117: Pantalla lista de compra con productos



Ilustración 118: Dialogo de confirmación para mover lista de compra al inventario

Pantalla donde el usuario podrá observar las distintas recetas disponibles ver Ilustración 119, donde el usuario puede actualizar nuevas recetas a través del botón flotante de dos flechas circundantes o también puede realizar el gesto de deslizar la pantalla hacia arriba suavemente. Otra posibilidad que tiene el usuario es poder guardar recetas a través del botón que aparece encima de la imagen de cada receta,

estas recetas se pueden consultar navegando a “Recetas guardadas”. Por último, el usuario tiene la opción de filtrar sus recetas a sus preferencias pulsando el botón filtrar, que abrirá una ventana flotante con los distintos filtros y con un botón para aplicar esos filtros ver Ilustración 120.



Ilustración 119: Pantalla recetas disponibles



Ilustración 120: Filtros para Recetas

Pantalla donde el usuario consulta sus recetas favoritas ver Ilustración 121, con posibilidad de filtrar sus recetas para optimizar la búsqueda o de eliminar todas sus recetas a través del botón flotante con icono de contenedor, este botón abrirá un dialogo de confirmación ver Ilustración 122.

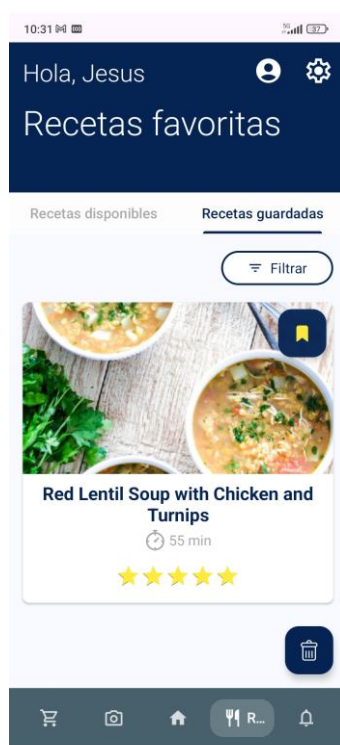


Ilustración 121: Pantalla recetas guardadas

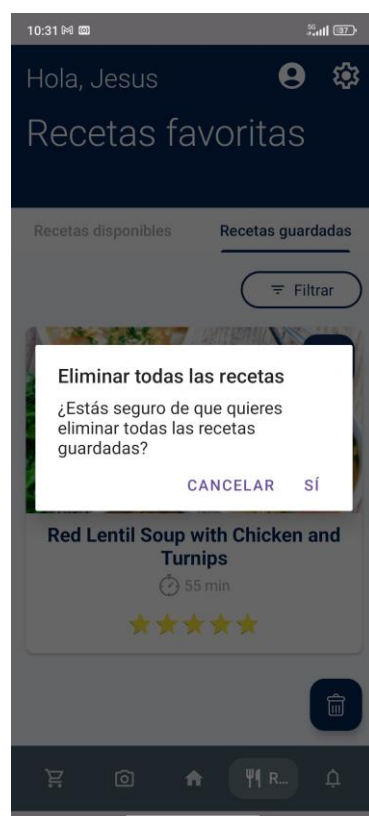


Ilustración 122: Diálogo de confirmación para eliminar todas las recetas

Pantalla en la que el usuario observa sus productos que se encuentran en mal estado con la opción de poder eliminarlos del inventario a través del icono contenedor, esto abrirá un mensaje de confirmación ver Ilustración 124. También, se muestra las diversas alertas de productos en temporada donde el usuario podrá añadir esos productos a su lista de compra ver Ilustración 123.



Ilustración 123: Pantalla alertas



Ilustración 124: Diálogo de confirmación de alertas

Pantalla donde el usuario configura su información personal. Para acceder a esta ventana el usuario tan solo debe seleccionar el botón con forma de persona de la cabecera ver Ilustración 125.



Ilustración 125: Pantalla perfil

Pantalla para configurar las distintas opciones que ofrece la aplicación como cambiar el tema a oscuro, configurar el idioma o desactivar las notificaciones, además de conocer información acerca de la aplicación ver Ilustración 126.



Ilustración 126: Pantalla configuración

Como resultado, se configura toda la aplicación, ver ejemplo Ilustración 127

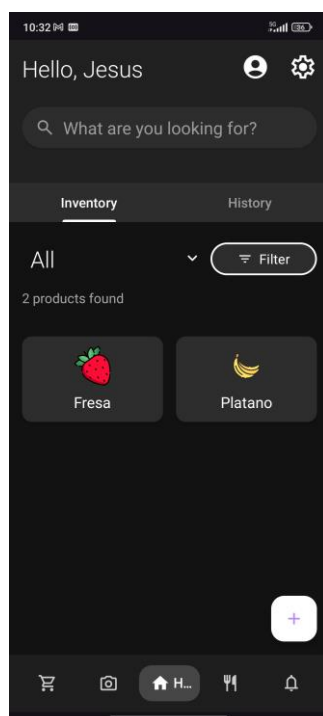


Ilustración 127: Pantalla tema oscuro con idioma inglés

Pantalla donde el usuario escanea manualmente sus productos, si el usuario selecciona el botón “escanear producto” y tiene los permisos de cámara activados, se abrirá la cámara de su teléfono donde tendrá que fotografiar el producto a escanear ver Ilustración 128.

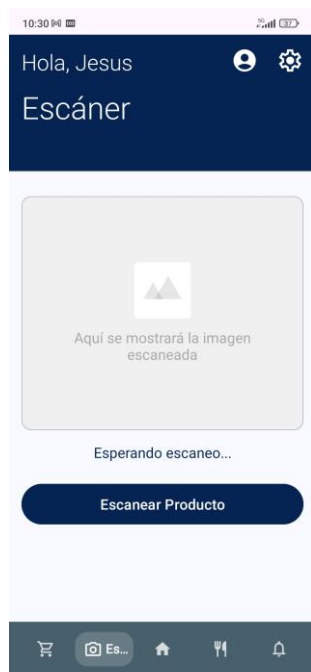


Ilustración 128: Pantalla escáner

Como resultado, el usuario tendrá el producto escaneado y tendrá la opción de poder agregar el producto escaneado a su inventario mediante un dialogo donde se muestra las características obtenidas del escaneo ver Ilustración 129.



Ilustración 129: Pantalla producto escaneado agregar a inventario

En la actualidad, la gestión eficiente de productos perecederos es un desafío en muchos hogares, lo que conlleva al desperdicio de alimentos y compras innecesarias. Gracias a *SmartFreezer* brindará a los usuarios una solución innovadora que optimiza la organización de frutas y verduras mediante un sistema de reconocimiento basado en redes neuronales. El objetivo de esta aplicación, es reducir el malgasto, mejorar la planificación de compras y proporcionar información interesante de cada alimento, facilitando así un consumo más eficiente y sostenible. Además, integrará funcionalidades como notificaciones de compras, información nutricional, recomendación de recetas con los productos disponibles, etc.

Esta aplicación ha sido desarrollada principalmente mediante el lenguaje de programación de código abierto *Kotlin*, preferido por muchos desarrolladores de Android. Por otro lado, se ha integrado la función de detección y reconocimiento de varias frutas y verduras mediante un modelo previamente entrenado por el modelo de Deep Learning *YOLOv8* "*You Only Look Once*". Cabe mencionar el uso del servicio de Backend-as-a-Service (BaaS) *Firebase* tanto para el sistema de autenticación de usuario, como sistema de notificaciones y uso de base de datos NoSQL.

Esta idea surge de la amplia demanda de dispositivos inteligentes en el hogar, por ejemplo, un frigorífico inteligente que solucione todas las peticiones del usuario para su compra y conservación de alimentos de forma automática. Sin embargo, *SmartFreezer* recoge todas estas características sin la inversión monetaria de un dispositivo nuevo al hogar.